

claude-windows / 985cc286-9e1e-4217-87c3-2d418bc4fa9c

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\985cc286-9e1e-4217-87c3-2d418bc4fa9c\subagents\agent-aabf47f43ed8cf751.jsonl`
- SHA-256: `ff82cd6d7966cacb8d190b8035bec3f9936fe3a936e754b239adf05b52aa9498`
- Source modified: `2026-06-14T10:28:39+00:00`
- Imported at: `2026-07-05T16:48:25+00:00`
- project: `subagents`
- session_id: `985cc286-9e1e-4217-87c3-2d418bc4fa9c`

Transcript

1. user (2026-06-14T10:10:01.167Z)

Build AND merge the ablation-layer Phase-0 per scratch/ABLATION_FRAMEWORK_PLAN.md. Repo cwd C:/Users/avidu/Projects/badminton-highlight-indexer. ISOLATED worktree: `git -C ... fetch origin` then `git -C ... worktree add C:/Users/avidu/Projects/bhi-ablation -b feat/ablation-layer origin/master`.

READ first: scratch/ABLATION_FRAMEWORK_PLAN.md (the design), backend/eval/experiment.py, eval_stats.py, fusion_compare.py, per_user_gate.py, segmentation_metrics.py, fusion_golden.py (the golden feature JSON shape), and scratch/quality_report.py (the reportlab pattern to reuse for PDF).

BUILD backend/eval/ablation.py ? a leave-one-signal-out (LOSO) + leave-one-GROUP-out (LOGO) ablation RUNNER that is a PURE DRIVER over experiment.compare + eval_stats (NO new scoring/stats math). For a corpus + a signal inventory (groups: shuttle, gate_a [structural], person, audio; + columns), for each signal/group report the marginal ?F1 (full set vs full-minus-that-signal) with bootstrap 95% CI + paired sign test + C4 over-seg ratio. Gate-A ablation = toggling the min_crossings gate (not a feature column). Honor the structural flag (Gate-A is reported but never 'recommend-remove' on no-lift). Emit a structured AblationReport + a sorted markdown/ASCII table. CLI: `python -m backend.eval.ablation --features-dir output [--pdf <path>]`.

LITMUS TEST (owner's ask): run on the 6 golden feature JSONs (output/*_golden_features.json), the runner MUST reproduce the known Gate-A result (marginal ?F1 ~+0.074, 6/6 wins, p~0.031, within tolerance). Add a test asserting this reproduction (validates the framework against the hand-measured truth). If the golden JSONs are absent in CI, the test must skip cleanly (and run when present).

PDF EXPORT (owner's ask): add a reportlab exporter rendering the AblationReport to a clean PDF report on ablation / feature-enablement ? a table of each signal's marginal contribution + CI + sign test + a verdict column (clears-bar / suggestive / inert / structural-protected). Wire `--pdf <path>`.

HEALTHY COVERAGE + MORE CI TESTS (explicit owner ask): comprehensive tests/test_ablation.py ? LOSO/LOGO correctness on deterministic synthetic candidates (GPU-free), the structural-guard exemption, honest-stats fields present, a valid non-trivial PDF is produced (assert via pypdf page count > 0), plus the golden litmus reproduction (skip-if-absent). Aim for high coverage of ablation.py.

CONSTRAINT: do NOT modify docs/QUALITY_ITERATIONS.md or eval_baselines/experiment_leaderboard.json (the Gate-A-flip agent owns those ? avoid conflicts); tests must use a TMP leaderboard, never the real file. A short docs/ABLATION_LAYER.md note is OK if minimal.

VERIFY: ruff + mypy + pytest (your tests + the eval suite) green. MERGE on green: `gh pr create --base master`; verify ALL 4 CI lanes; `gh pr merge <n> --squash --delete-branch`; cleanup worktree+branch+prune. Branch-safe, explicit paths, `Co-Authored-By: Claude Opus 4.8 (1M context) <noreply@anthropic.com>`.

Return: PR number/URL, files, the LITMUS result (did it reproduce Gate-A +0.074?), a sample ablation table, the PDF path, test count + coverage, merge status.

2. assistant (2026-06-14T10:10:03.531Z)

I'll start by reading the plan and the key files to understand the design, then build the ablation layer in an isolated worktree.

3. user (2026-06-14T10:10:04.946Z)

```
1      # Ablation framework ? quantifying each signal's marginal contribution over time
2
3      **Status:** DESIGN ? owner-gated, NOT started. A peer to
4      [`EXPERIMENT_HARNESS.md`](../docs/EXPERIMENT_HARNESS.md) and
5      [`ANALYZERS_AND_RECONCILERS.md`](../docs/ANALYZERS_AND_RECONCILERS.md). Docs-only here;
each
6      phase below is a separate, owner-gated, default-OFF PR.
7
8      > **One-line intent (owner).** A *systematic, repeatable* version of the one-off
ablation: run
9      > it and it EXCLUDES a single signal, measures whether quality dropped, and ? tracked
across the
10     > EVOLUTION of quality ? tells us QUANTIFIABLY whether an old signal still earns its
keep or has
11     > become dead weight. The ablation table IS the empirical backbone of the paper-coauthor
aim
12     > ([`MEMORY` paper-coauthor-aim], honest measured results only).
13
14     ---
15
16     ## 0. TL;DR
17
18     The harness already computes a B?A lift honestly (LOVO + bootstrap CI + paired sign
test,
19     `experiment.compare` + `eval_stats`). **An ablation is just a structured family of those
20     comparisons** where B = "full signal set" and each A = "full set MINUS one signal" ? so
the
21     ?F1 it reports *is* that signal's **marginal contribution**. We add (1) a leave-one-
signal-out
22     (LOSO) / leave-one-group-out (LOGO) **runner** over the golden corpus that emits a per-
signal
23     ?F1 table (CI + sign test), (2) a **relevance ledger** that appends each run as a
timestamped
24     row so a signal's contribution is a time series ? a signal trending to ?0 (CI spanning
0, sign
25     test n.s. across several iterations) is flagged as candidate dead weight, (3)
**interaction
26     handling** (leave-PAIR-out + backward elimination, optional Shapley) so two-cheap-alone
/
27     useful-together signals aren't wrongly cut, (4) honest small-N reuse of `eval_stats`
with a
28     **no-over-claim contract**, (5) integration with the leaderboard +
`QUALITY_ITERATIONS.md`, and
29     (6) a per-user generalization ("signal X not relevant FOR THIS USER") that reuses
`per_user_gate`.
30
31     **Nothing here is new scoring math.** It is a *driver* over `experiment.compare` +
`eval_stats`
32     + `segmentation_metrics`, exactly as `compare` was a driver over
`metrics`/`calibration`.
33
34     ---
35
36     ## 1. What "a signal" is (the unit of ablation)
37
```

```

38     A signal = **one feature column** (or a named group of columns from one producer) that
the scorer
39     consumes. The repo already treats signals as columns
([`ANALYZERS_AND_RECONCILERS.md`](../docs/ANALYZERS_AND_RECONCILERS.md))
40     $1/$4): `experiment.predict` reads `feature_names` out of persisted `stats` and
gates/weights them.
41     So **ablating a signal = removing its column(s) from a `Variant`'s `feature_names`**
(and, for a
42     gate-style signal like Gate-A, zeroing `min_crossings`). No code branch; no re-
detection.
43
44     The signal inventory today (the things the runner ablates), grouped by producer:
45
46     | Group (producer) | Signal columns | Source |
47     |---|---|---|
48     | `shuttle` | `duration`, `peak_velocity`, `mean_velocity`, `active_ratio` |
`distill_local.FEATURE_NAMES` |
49     | `gate_a` (net-crossings) | `net_crossings` (as a gate `min_crossings`, AND/or a
column) | `rally_gate` / `experiment._gate_mask` |
50     | `person` | `players_in_court`, `two_sided_motion`, `mean_player_motion`,
`in_court_ratio`, `shuttle_player_colocation` | `fusion_features.PERSON_FEATURE_NAMES` |
51     | `audio` | `audio_hit_count`, `audio_hit_rate`, `audio_hit_strength_mean` |
`audio_features.AUDIO_FEATURE_NAMES` |
52     |  $*(future)*$  `service_fault`, `warmup`, `shuttle_state` | `>__<key>` (+
`_confidence`) | `ANALYZERS_AND_RECONCILERS.md` $3 |
53
54     The runner takes a **signal registry**: an ordered list of `Signal(name, group, columns,
55     gate_knob=None)` records. Two ablation granularities fall out of one registry:
56
57     - **Leave-one-signal-out (LOSO)** ? remove ONE column at a time. Fine-grained ("is
58     `shuttle_player_colocation` pulling its weight?").
59     - **Leave-one-GROUP-out (LOGO)** ? remove a whole producer's columns at once ("is the
entire
60     `person` cue still relevant, or has the shuttle+audio set subsumed it?"). This is the
owner's
61     "is this old SIGNAL still relevant" question at the cue granularity, and it is what
the
62     `QUALITY_ITERATIONS` person-vs-audio rows are already informally doing.
63
64     > **Naming note (don't confuse two LOGOs).** `eval_stats.grouped_folds` is leave-one-
VIDEO-group-out
65     > over the EVAL units (group by match) ? the honesty axis. This doc's LOGO is leave-one-
SIGNAL-group-out
66     > over the FEATURE axis. Both run together: each signal-group ablation is measured
*with* video-grouped
67     > LOVO underneath. Keep the terms "signal-LOSO/LOGO" vs "video-LOVO/LOGO" distinct in
code + tables.
68
69     ---
70
71     ## 2. Marginal contribution ? the core measurement
72
73     For a full signal set `S` and a signal `s`, the **marginal contribution** of `s` is:
74
75     ```
76     ?(s) = score(full set S) - score(S \ {s})          # leave-one-out drop
77     ```
78
79     measured as **?F1** (primary), with **F1@k + segment-count-ratio** alongside (C4 ? over-
seg that
80     tIoU hides). Operationally this is exactly:
81
82     ```python
83     full = Variant(name="full",          feature_names=ALL_COLUMNS, min_crossings=2, ...)
84     without_s = Variant(name=f"?{s.name}", feature_names=[c for c in ALL_COLUMNS if c not in

```

```

s.columns],
85         min_crossings=(0 if s.gate_knob=="min_crossings" else 2))
86     res = compare(golden_videos, variant_a=without_s, variant_b=full, honest_stats=True)
87     #         A = ablated (smaller), B = full ? result["honest"]["paired_delta_f1"] is
?(s),
88     #         a per-video-paired ?F1 with bootstrap CI + exact sign test, LOVO-honest if
learned.
89     ...
90
91     Reading `compare`'s `honest` block gives, per signal, for free:
92     - `paired_delta_f1.mean_delta` ? the marginal ?F1 (B?A = full?ablated);
93     - `paired_delta_f1.ci_low/ci_high` + `ci_excludes_zero` ? is the drop real at n=6?
94     - `paired_delta_f1.sign_test` ? did removing `s` hurt on a MAJORITY of videos?
95     - the C4 segmentation block ? did removing `s` fragment/merge rallies even if F1 barely
moved?
96
97     A signal is earning its keep when removing it produces a drop whose CI excludes 0
AND the
98     sign test agrees ( ? the majority of videos got worse) ? the same bar `per_user_gate` /
99     promotion use, applied in reverse. A signal is candidate dead weight when removal is
a wash
100     (`mean_delta ? 0`, CI spans 0, sign test n.s.) across multiple iterations ($4 ? one
wash is
101     noise at n=6, a trend is signal).
102
103     > ? LOSO is necessary but not sufficient. Leave-one-out attributes the unique
contribution
104     > of `s` given everything else present. It MISSES redundancy and synergy: two signals
each useless
105     > alone but jointly useful both show ?0 LOSO drop (each is covered by the other ? "drop
one, the
106     > other compensates"); and a signal can look free under LOSO yet be the linchpin of a
pair. $5 adds
107     > leave-PAIR-out + backward elimination to recover this. Report LOSO as the screening
pass, never
108     > as the final verdict to delete a signal.**
109
110     ---
111
112     ## 3. The runner (Phase 0)
113
114     `backend/eval/ablation.py` ? a pure driver, no new scoring math. Public surface:
115
116     ```python
117     @dataclass(frozen=True)
118     class Signal:
119         name: str                # "person.colocation" | "gate_a" | "audio"
120         group: str                # "person" | "gate_a" | "audio" | "shuttle"
121         columns: Tuple[str, ...]  # feature columns this signal contributes
122         gate_knob: Optional[str] = None # e.g. "min_crossings" for the Gate-A threshold
signal
123
124     @dataclass
125     class AblationConfig:
126         full: Variant            # the full-set reference variant (B)
127         signals: List[Signal]     # the inventory to ablate
128         granularity: str = "signal" # "signal" (LOSO) | "group" (LOGO)
129         iou: float = 0.5
130         honest: bool = True       # LOVO for learned variants
131         seed: int = 0             # threaded into eval_stats bootstrap (determinism)
132
133     def ablate_one(videos, cfg, signal_or_group) -> Dict:            # one row: ?(s) + CIs +
sign test + C4
134     def run_ablation(videos, cfg) -> AblationReport:                # all signals ? a sorted
table

```

```

135     ``
136
137     `run_ablation` loops the inventory, builds `without_s` by **subtracting columns /
zeroing gates**
138     from `cfg.full`, calls `compare(videos, without_s, cfg.full)`, and collects each
`honest` block
139     into an `AblationReport`. **It reuses `compare` verbatim** ? so learned variants are
LOVO-honest,
140     threshold/calibration stay inside the fold (C2), and the C1+C4 honest stats come for
free.
141
142     **Output table** (sorted by marginal ?F1 desc ? the contribution ranking):
143
144     ``
145     Ablation report ? full set = {shuttle, gate_a, person, audio} . n=6 . IoU 0.5 .
LOVO-honest
146     signal                ?F1(full?ablated)    95% CI                sign test    CI?0    verdict
147     gate_a                +0.074                [+0.042, +0.104]    6W/0L p=.031    yes      EARNs KEEP
148     audio                  +0.041                [+0.054, +0.137]    4W/2L p=.69     no       uncertain
(keep, watch)
149     shuttle                +0.301                [ ? ]                6W/0L                yes      EARNs KEEP
(load-bearing)
150     person                 ?0.021                [+0.165, +0.162]    2W/4L p=.69     no       CANDIDATE
DEAD WEIGHT
151     ``
152
153     (The `gate_a`/`person` numbers above are the already-measured `QUALITY_ITERATIONS`
2026-06-14
154     values, shown to anchor the format ? the runner reproduces them as a regression check on
day one.)
155
156     **Reuses, no reinvention:** `experiment.compare`/`evaluate_variant`/`predict`,
157     `eval_stats.paired_delta_ci`/`paired_sign_test`/`mean_with_ci`, `segmentation_metrics`,
158     `record_experiment` (each ablation row is just an experiment record),
`load_video_candidates`
159     (the DB?`VideoCandidates` bridge, owner-side). The golden corpus is the 6 matches in
160     `output/human_lovo_manifest.json` (`QUALITY_ITERATIONS` $"Golden corpus").
161
162     **Determinism + small-N honesty:** the bootstrap seed is threaded; n is carried on every
row;
163     no row is allowed to print a verdict stronger than the stats support ($6 contract).
164
165     ---
166
167     ## 4. The Signal-Relevance Ledger (a time series of marginal contribution)
168
169     The single most novel piece for the owner's "is this OLD signal still relevant?"
question. Today
170     the leaderboard records *pairwise A/B experiments*; it does NOT track a *signal's
contribution as
171     the system evolves*. The ledger does.
172
173     **File:** `eval_baselines/signal_relevance_ledger.json` (tracked, baseline-sized ? same
posture as
174     `experiment_leaderboard.json`; NOT a service, per `EXPERIMENT_HARNESS` $9). One **run**
appends one
175     block; one **row per signal** inside it:
176
177     ```json
178     {
179         "timestamp": "2026-06-21T?",
180         "label_set_hash": "50d98ffbc370",          // ties a contribution to the exact golden
labels (gt drift guard)
181         "full_set_hash": "253f4709b4c4",          // spec_hash of the full Variant at this
iteration

```

```

182     "full_set_signals": ["shuttle", "gate_a", "person", "audio"],
183     "num_videos": 6,
184     "granularity": "group",
185     "rows": [
186         {"signal": "gate_a", "group": "gate_a", "delta_f1": 0.074, "ci_low": 0.042, "ci_high": 0.104,
187          "ci_excludes_zero": true, "sign": {"wins": 6, "losses": 0, "p_value": 0.031},
188     "f1_at_overlaps_delta": {?}, "segment_count_ratio_delta": -0.67, "n": 6, "verdict": "earns_keep"},
189         {"signal": "person", ? , "verdict": "candidate_dead_weight"}
190     ]
191 }
192 ```
193
194     **Why it's a time series, not a snapshot.** As the full set evolves (a new cue lands, a
195     label is
196     fixed, the corpus grows), each signal's marginal contribution is RE-MEASURED *against
197     the new full
198     set*. So `gate_a`'s row in run-1 and run-7 are comparable points on a curve. A reader
199     (or a tiny
200     report tool) can ask:
201
202     - ***"Is `person` still dead weight?"** ? plot `delta_f1` for `person` across runs.
203     Persistently
204     ?0 with CI spanning 0 ? flag `STALE` (candidate for removal / archival).
205     - ***"Did `gate_a` decay when audio landed?"** ? a once-strong signal whose marginal
206     contribution
207     *shrinks* as a newer signal subsumes it = quantified **redundancy onset** (the newer
208     signal now
209     carries what the old one used to). This is the dead-weight question answered with a
210     number.
211     - ***"Did corpus growth resolve audio?"** ? `audio`'s CI should TIGHTEN as n grows; a
212     contribution
213     that was "uncertain (CI spans 0) at n=6" becoming "earns keep (CI clears 0) at n=20"
214     is the
215     honest promotion trigger ? and the inverse (still spanning 0 at large n) is honest
216     demotion.
217
218     **Staleness flag (quantified dead-weight rule).** A signal is flagged `STALE_CANDIDATE`
219     when, over
220     the **last K runs** (K?3, configurable) at a comparable n: every run's `ci_excludes_zero
221     == false`
222     AND the sign test never reaches `p ? p_threshold` AND the rolling-mean `delta_f1` is
223     within
224     `[??, +?]'` (? small, e.g. 0.01). That is "removal never measurably hurt, repeatedly" ?
225     the
226     defensible, reproducible version of "this old signal is dead weight." Flagging is
227     **report-only**;
```

the owner decides whether to retire the column (retiring = drop it from the `full` Variant; rollback

= add it back ? same default-OFF / config-flip discipline). **A structurally-protected signal is

NEVER auto-flagged** (see S6 / `per\_user\_gate` structural rule: a guard earns its keep through

failure modes the golden corpus may not contain ? e.g. Gate-A on held-shuttle FPs).

A `tools/relevance\_report.py` (or a notebook cell) renders the ledger as a small markdown/ASCII

sparkline table for `QUALITY\_ITERATIONS.md` ? the ablation table *is* the paper's empirical backbone,

maintained automatically instead of by hand.

5. Redundancy & interaction ? when LOSO lies

```

226     Three named failure modes of naive leave-one-out, and the handling for each (cost-
ordered cheap?dear):
227
228     | Failure mode | What LOSO shows | What's true | Handler |
229     |---|---|---|---|
230     | **Redundancy** (two signals carry the same info) | each LOSO ? ? 0 (the other
compensates) | together they're load-bearing; dropping BOTH tanks F1 | **leave-PAIR-out** +
**backward elimination** |
231     | **Synergy / XOR** (useless alone, useful together, e.g. `shuttle_in_air AND
person_two_sided`) | each LOSO ? ? 0 (or even +) | the PAIR is the contribution; a linear
arbiter can't even see it | **leave-pair-out** + an **interaction column** (C7) |
232     | **Suppressor** (a signal helps only by correcting another) | LOSO ? small | removing
it *with* its partner changes the story | **backward elimination** path-dependence note |
233
234     ** (a) Leave-pair-out (LPO). ** For the suspect set (signals whose LOSO ? spans 0), also
run
235     `compare(full ? {s_i, s_j}, full)` for each pair. If a pair's joint drop's CI clears 0
while *both*
236     singletons' did not ? **redundant-but-jointly-useful**: keep both, do NOT cut either on
LOSO
237     evidence. Cost:  $O(k^2)$  comparisons vs  $O(k)$  for LOSO ? bounded ( $k \geq 4$  groups ? 6 pairs),
and only
238     run on the LOSO-ambiguous subset, so cheap in practice. Gate it behind a flag
(`--pairs`); default
239     LOSO-only.
240
241     ** (b) Backward elimination (greedy, honest). ** The decision procedure that *uses* the
marginal
242     numbers: starting from the full set, repeatedly drop the signal whose removal is the
**least
243     harmful** (most-positive / least-negative ?F1 with CI NOT clearing 0), re-measure the
new full set,
244     repeat until every remaining signal's removal measurably hurts (CI clears 0 + sign
agrees). The
245     surviving set is the **honest minimal sufficient signal set**. This is the principled
answer to "trim
246     dead weight" ? and it is path-dependent by construction (it re-measures after each
drop), so it
247     catches redundancy that one-shot LOSO can't. Cost:  $O(k^2)$  worst case; report the
elimination *path*
248     (order + ? at each step) in the ledger so the trim is auditable, not a black box.
249
250     ** (c) Shapley-style attribution (optional, owner-gated, higher cost). ** The
theoretically-fair
251     "average marginal contribution over all subset orderings" ? the only attribution that is
order- and
252     redundancy-fair. Exact Shapley is  $O(2^k)$  full-set evaluations; at  $k \geq 6$  signal
GROUPS this is
253      $16 \times 64$  evals ? entirely affordable offline** (re-scoring is cheap; detection is already
persisted).
254     So: **exact Shapley over GROUPS** is in scope as an opt-in (`--shapley`), Monte-Carlo
Shapley only if
255      $k$  ever grows past ~10 columns. Report it as a *complement* to backward elimination
(Shapley says "fair
256     credit"; backward elimination says "what's safely droppable"), with the same CI honesty
? each subset
257     eval is itself a LOVO mean, so a Shapley value carries propagated small-N uncertainty
and must not be
258     over-claimed at  $n=6$ .
259
260     **Cost trade-off summary (what to run when):**
261     - **Always:** LOSO + LOGO (Phase 0) ?  $O(k)$ , the screening table.
262     - **On LOSO-ambiguous signals:** leave-pair-out ( $O(k^2)$  on a subset) ? catches
redundancy/synergy.
263     - **For a trim decision:** backward elimination ( $O(k^2)$ ) ? the minimal-set procedure +

```

```

audit path.
264 - **For paper-grade fair credit:** exact Shapley over groups ( $0(2^k)$ ,  $k \geq 6$ ) ? opt-in.
265 - **Interaction recovery for the scorer itself:** add hand-crossed product columns (C7)
so the linear
266 arbiter can *represent* a synergy LPO surfaced ? then that interaction is itself an
ablatable signal.
267
268 ---
269
270 ## 6. Honest small-N handling (the non-negotiable contract)
271
272 The whole point is honest verdicts at  $n=6$ , where a single mean lies. Reuse `eval_stats`
and
273 **inherit its discipline** ; add NO new statistics. The contract every ablation row
obeys:
274
275 1. **Never a bare mean.** Every  $\phi(s)$  ships with a bootstrap 95% CI + the exact paired
sign test
276 (`paired_delta_ci`). A row with no CI is a bug.
277 2. **A signal "earns its keep" only when** removing it drops F1 with **CI excluding 0
AND sign test
278 agreeing** (majority of videos worse,  $p \leq p_{\text{threshold}}$ ) ? the `per_user_gate` bar,
in reverse.
279 3. **"Dead weight" needs a TREND, not one run.** A single wash at  $n=6$  is *insufficient
evidence*, not
280 proof of irrelevance. The `STALE_CANDIDATE` flag (§4) requires  $k \geq 3$  consistent runs ?
this is the
281 honest version of "remove it." Until then the verdict is literally `uncertain`, never
`irrelevant`.
282 4. **Structural signals are exempt from auto-demotion** (mirror `per_user_gate`
`structural=True`): a
283 guard whose value is in failure modes the golden corpus may lack (Gate-A vs held-
shuttle FPs) is
284 marked `structural` in the registry and **never flagged dead weight on "no measured
lift."** Its
285 ledger row records the (possibly  $\phi_0$ ) golden ? honestly but tags
`structural_protected: true`.
286 5. **Group by match for the eval folds** (video-LOVO via `compare`'s honest path) ?
never a held-out
287 window from the same video; the small corpus's LOOCV anti-correlation bias (C1) is
why CIs, not
288 means, gate.
289 6. **Multiplicity caveat.** Ablating  $k$  signals =  $k$  simultaneous tests; at  $k \geq 4$  we report
raw p-values
290 but **note the family-wise risk** in the ledger header (a Holm/BH correction is a
one-liner to add
291 when  $k$  grows ? flagged, not yet applied, to avoid over-engineering at  $k=4$ ).
292 7. **No over-claim, ever** (attribution working agreement): the runner emits
`earns_keep` /
293 `uncertain` / `candidate_dead_weight(trend)` / `structural_protected` ? it MUST NOT
print
294 "irrelevant" or "proven useless." Negative/uncertain rows are KEPT on purpose (the
honest backbone).
295
296 ---
297
298 ## 7. Integration with the existing machinery
299
300 - **Experiment harness / leaderboard.** Each `ablate_one` is a `compare` call ? already
301 `record_experiment`-able. An ablation run optionally appends its pairwise rows to
302 `experiment_leaderboard.json` (provenance) AND its per-signal summary to the new
303 `signal_relevance_ledger.json` (the time series). Zero new persistence primitives ?
both are
304 baseline-sized JSON, the `EXPERIMENT_HARNESS` §9 posture.
305 - **`QUALITY_ITERATIONS.md`.** The relevance-report renderer drops an updated ablation

```

```

table into the
306     living log at each run; the prose entry (hypothesis / measured / lesson) stays human-
written. The
307     person=no-lift / audio=promising rows already there are the first two ledger points ?
the runner
308     reproduces them as a regression check.
309     - **Analyzers/reconcilers (`ANALYZERS_AND_RECONCILERS.md`).** Every new analyzer signal
310     (`service_fault__?`, `warmup__in_warmup`, derived `shuttle_state`) registers in the
**same signal
311     inventory** and is ablated by the same runner the moment it persists a column ? so
"does this new
312     cue earn its keep, and does it stay earning it?" is answered uniformly. A reconciler-
as-Variant
313     (§6 of that doc) is ablatable too (full?reconciler).
314     - **Promotion gate.** Ablation is the *complement* of promotion: promotion asks "does
ADDING B beat
315     A?"; ablation asks "does REMOVING s from the full set hurt?". Same
`compare`/`eval_stats` engine,
316     opposite direction. Backward elimination (§5b) is the natural *de-promotion* procedure
the gate
317     lacks today.
318     - **CI lane.** A `run_ablation` smoke test (synthetic candidate rows, deterministic)
runs in the
319     GPU-free lane like the rest of the harness; a `--check-no-regression` mode asserts the
day-one
320     table matches the recorded `QUALITY_ITERATIONS` numbers within tolerance (drift guard
on the math).
321
322     ---
323
324     ## 8. Per-user ablation (ties to the personalization gate)
325
326     The owner's "motion cue not relevant FOR THIS USER" is per-user ablation, and it
composes cleanly:
327     the runner's eval folds are videos; restrict them to **one user's videos** and you get
that user's
328     per-signal marginal contribution. The decision then reuses
`per_user_gate.should_promote_for_user`
329     *in reverse*:
330
331     ```python
332     # Per-user relevance of signal s for user U: A = U's videos with s removed, B = U's
videos full set.
333     a = evaluate_variant(U_videos, without_s).per_video_f1      # ablated
334     b = evaluate_variant(U_videos, full).per_video_f1          # full
335     dec = should_promote_for_user(a_scores=b, b_scores=a, structural=s.structural, min_n=?)
336     # Read as: is REMOVING s a (per-user) win? If yes (and not structural) ? s is dead
weight FOR U ?
337     # personalize that user's served Variant to the smaller set. If structural ? keep
regardless (Rule 1).
338     ```
339
340     This inherits all of `per_user_gate`'s safety: the **min-N floor** (refuse to
personalize off 1?2
341     videos), the **CI-excludes-0 + sign-test** bar, and ? critically ? the **structural-
guard exemption**
342     (a user whose corpus simply lacks held-shuttle FPs must NOT get Gate-A removed; the
guard stays ON
343     regardless of that user's measured ?). Output is a **per-user relevance ledger** keyed
by user_id,
344     the personalization analog of §4, feeding the same flag-gated, fall-back-to-baseline
promotion path.
345     This is owner-gated (personalization is product/strategy) and arrives after the global
ledger proves out.
346

```

```

347 ---
348
349 ## 9. Phased, PR-sized build plan
350
351 Engineering (E) = good-engineering-fix, mergeable on verified-green CI per the merging
working
352 agreement. Owner-gated (O) = product/strategy/personalization, needs explicit approval.
353
354 **Phase 0 ? the leave-one-signal-out runner + results table (E):**
355
356 - **PR-A0 ? Signal inventory + `ablation.py` LOSO/LOGO runner.**
`Signal`/`AblationConfig`/
357 `AblationReport` + `ablate_one` + `run_ablation`, a pure driver over
`experiment.compare` (no new
358 scoring math). Hard-code the current 4-group inventory; `structural` flag on `gate_a`.
Unit-tested
359 on synthetic candidate rows (deterministic, GPU-free). **DoD:** reproduces the
recorded
360 `QUALITY_ITERATIONS` person/gate_a numbers from a fixture within tolerance.
361 - **PR-A1 ? Results table + leaderboard wiring.** Sorted ASCII/markdown ablation table;
append each
362 row to `experiment_leaderboard.json` via `record_experiment`; a `--check-no-
regression` mode. CLI
363 `python -m backend.eval.ablation --manifest output/human_lovo_manifest.json`.
364
365 **Phase 1 ? the time-series relevance ledger (E for the mechanism; O to retire any
signal):**
366
367 - **PR-L0 ? `signal_relevance_ledger.json` writer** (append one block/run;
`label_set_hash` +
368 `full_set_hash` provenance). (E)
369 - **PR-L1 ? `tools/relevance_report.py`** renders the ledger as a per-signal sparkline
table +
370 computes the `STALE_CANDIDATE` flag (K-run trend rule, §4); drops the table into
371 `QUALITY_ITERATIONS.md`. Report-only. (E)
372 - **PR-L2 ? act on a flag (retire / archive a signal).** Owner-gated: removing a column
from the
373 `full` Variant is a product call (rollback = re-add). (O)
374
375 **Phase 2 ? interaction analysis (E for the analysis; opt-in flags):**
376
377 - **PR-I0 ? leave-pair-out** on the LOSO-ambiguous subset (`--pairs`),
redundancy/synergy table. (E)
378 - **PR-I1 ? backward elimination** procedure + audited elimination-path record ? minimal
sufficient
379 set. (E)
380 - **PR-I2 ? exact Shapley over groups** (`--shapley`, k?6) + C7 interaction-column
support so a
381 surfaced synergy is itself ablatable. (E, paper-track)
382
383 **Phase 3 ? per-user ablation (O ? personalization is product/strategy):**
384
385 - **PR-U0 ? per-user relevance runner** reusing `per_user_gate` in reverse + the
structural exemption;
386 per-user relevance ledger keyed by user_id; feeds the flag-gated, fall-back-to-
baseline path. (O)
387
388 Each PR: ONE work item, off `origin/master`, default-OFF, gated by the no-regression
invariant +
389 `run_regression.py`. Phases 1?3 are independent of each other given Phase 0 and can be
reprioritized.
390
391 ---
392
393 ## 10. Verification

```

```

394
395     - **Offline / dev-container (no GPU/torch):** `run_ablation` deterministic on synthetic
candidate
396     rows; the day-one table reproduces the recorded `QUALITY_ITERATIONS` person/gate_a
numbers (math
397     drift guard); leave-pair-out on a constructed redundant pair (both singletons span 0,
the pair
398     clears 0) and a constructed synergy pair behave as designed; the `STALE_CANDIDATE`
trend rule fires
399     on a 3-run synthetic ledger and stays silent on a 1-wash ledger; structural signals
never flag;
400     ledger round-trips JSON with stable hashes; no row ever prints a verdict stronger than
its stats.
401     - **Machine-side (owner, GPU/WSL):** persist the full signal set once on the 6 golden
videos
402     (`fusion_golden.py` substrate already exists), run `run_ablation` LOVO, confirm the
per-signal ?F1
403     matches a manual `compare(without_s, full)` spot-check; run backward elimination and
sanity-check
404     the surviving set against intuition (shuttle + gate_a survive; person is the
elimination candidate).
405     - **Docs:** the ledger table renders into `QUALITY_ITERATIONS.md`; this plan graduates
to `docs/` and
406     is cross-linked from `EXPERIMENT_HARNESS.md` / `ANALYZERS_AND_RECONCILERS.md` when
greenlit.
407
408     ---
409
410     ## 11. Open design questions (for the owner)
411
412     1. **Granularity default ? column (LOSO) or producer-group (LOGO)?** The owner's "is
this old
413     *signal* still relevant" reads at the GROUP level (person/audio/shuttle), but column-
level catches
414     a single dead column inside a useful group. Default to LOGO for the relevance
verdict, expose LOSO
415     for diagnosis? Or always report both?
416     2. **Dead-weight trend threshold (K, ?, p).** K?3 consecutive washes at comparable n,
|rolling ?|<?,
417     sign never significant ? are these the right knobs, and should the flag require n to
be *non-
418     decreasing* across the K runs (so corpus shrinkage can't fake staleness)?
419     3. **Re-measure-on-every-iteration cost.** The ledger's value is re-measuring every
signal against
420     each new full set. On the persisted golden substrate this is cheap; do we want it on
EVERY harness
421     run (auto), or a separate explicit `make ablate` cadence (e.g. per merged cue +
weekly)?
422     4. **Shapley scope.** Exact Shapley over ?6 groups is affordable ? is paper-grade fair-
credit
423     attribution worth the extra surface now, or deferred until the corpus is large enough
that its
424     small-N uncertainty isn't itself the dominant caveat?
425     5. **Multiplicity correction.** Apply Holm/BH across the k simultaneous ablation tests
now (more
426     honest, slightly more conservative ? may mask a real signal at n=6), or report raw p
with a
427     family-wise caveat until k grows?
428     6. **Per-user trigger.** Per-user ablation needs ? `min_n` videos/user ? do enough users
have that,
429     and is per-user signal-retirement (vs only per-user signal-*addition*) even desired,
or should
430     personalization stay add-only for safety?
431

```

4. user (2026-06-14T10:10:06.810Z)

```
7c6498f docs(quality): personalization Phase-0 checkpoint ? eval/safety rails + locked
structural-guard decision (#145)
4be16e1 docs(personalization): add PERSONALIZATION_PLAN ? hybrid per-user feature + locked owner
decisions (#144)
1a98ee8 feat(eval): per-user no-regression guard ? never make a user worse on a retrain/upgrade
(#143)
```

5. assistant (2026-06-14T10:10:13.317Z)

I have the plan. Now let me set up the isolated worktree and read the key source files in parallel.

6. user (2026-06-14T10:10:15.194Z)

```
Preparing worktree (new branch 'feat/ablation-layer')
branch 'feat/ablation-layer' set up to track 'origin/master'.
HEAD is now at 7c6498f docs(quality): personalization Phase-0 checkpoint ? eval/safety rails +
locked structural-guard decision (#145)
---EXIT:0---
7c6498f docs(quality): personalization Phase-0 checkpoint ? eval/safety rails + locked
structural-guard decision (#145)
```

7. assistant (2026-06-14T10:10:18.646Z)

Worktree is set up at `C:/Users/avidu/Projects/bhi-ablation` on `feat/ablation-layer` off `origin/master`. Now let me read the key source files.

8. user (2026-06-14T10:10:19.541Z)

```
1      """Experiment harness ? A/B + shadow + side-by-side for rally detection.
2
3      This is the thin variant/experiment layer that `docs/EXPERIMENT_HARNESS.md`
4      identifies as the one missing piece: ~80% of the machinery (scoring, LOVO, the
5      no-regression gate, the pinnable classifier artifact, the persisted-candidate
6      substrate) already exists ? this module composes it. No new scoring math lives
7      here*; everything defers to:
8
9      - `backend.eval.metrics`      ? `score`, `match_intervals`, `interval_iou`
10     - `backend.eval.training`      ? `label_candidates` (y by the same IoU matcher)
11     - `backend.eval.classifier`    ? `LogisticRegression` (the pinnable JSON model)
12     - `backend.eval.gemini_refine` ? `merge_overlaps` (the blessed window merge)
13     - `backend.eval.regression`    ? `gt_hash` (label-set fingerprint)
14     - `backend.eval.calibration`    ? `pct_improvement`
15
16     The thesis (`EXPERIMENT_HARNESS.md` §2): separate the expensive, risky DETECTION
17     (per-frame signals ? run once on the GPU/WSL box, persisted into
18     `candidate_segments.stats`) from the cheap, swappable DECISION (given those
19     signals, where are the rallies). A `Variant` is a declarative re-scoring recipe;
20     given persisted candidate features it produces `List[Interval]` deterministically,
21     with no GPU and zero production risk. So most experiments are pure offline
22     re-scoring of stored data and never touch the served pipeline.
23
24     Three modes (`EXPERIMENT_HARNESS.md` §5):
25     - `compare()`      ? labeled A/B vs golden labels: per-video + LOVO-honest
26       aggregate deltas. Mirrors `distill_local`'s honest train-on-others/predict-
27       held-out loop, but variant-parameterised.
28     - `compare_unlabeled()` ? SxS on NEW footage with no ground truth: where do two
29       variants agree / diverge (A-only, B-only, agreed). The divergences are what a
30       human spot-checks (and what two highlight reels would show side by side).
31     - the promotion gate stays `run_regression.py` + `regression_baseline.json`
32       (exit 0/1/3); rollback = flip the variant/config, never `git revert`.
33
```

```

34 Pure + offline + unit-tested. The only DB-touching helper is
35 `load_video_candidates`, which reads persisted candidates via
36 `Database.query_candidate_segments`.
37 """
38 from __future__ import annotations
39
40 import json
41 import logging
42 import os
43 from dataclasses import dataclass, field
44 from datetime import datetime, timezone
45 from hashlib import sha1
46 from typing import Any, Callable, Dict, List, Optional, Sequence
47
48 from backend.eval.calibration import pct_improvement
49 from backend.eval.classifier import LogisticRegression
50 from backend.eval.gemini_refine import merge_overlaps
51 from backend.eval.metrics import Interval, match_intervals, score
52 from backend.eval.regression import gt_hash
53 from backend.eval.training import label_candidates
54 from backend.eval import eval_stats as _stats          # C1: CIs + paired sign test
55 from backend.eval import segmentation_metrics as _segm  # C4: F1@k + segment-count
ratio
56 from backend.eval.score_calibration import fit_isotonic, fit_platt  # C2: calibrate cue
scores
57
58 logger = logging.getLogger(__name__)
59
60 # Where a comparison run appends one reproducible record. Tracked location (NOT under
61 # the gitignored output/) so the leaderboard survives a clean checkout, mirroring
62 # regression.DEFAULT_BASELINE_PATH.
63 DEFAULT_LEADERBOARD_PATH = os.path.join("eval_baselines", "experiment_leaderboard.json")
64 _METRICS = ("precision", "recall", "f1", "mean_iou")
65
66
67 class ExperimentError(ValueError):
68     """A variant cannot be evaluated as configured (e.g. a learned variant asked to
69     score an unlabeled video with no pinned model, or a gate referencing an absent
70     feature)."""
71
72
73 # ----- Variant
74
75
76 @dataclass
77 class Variant:
78     """A declarative re-scoring recipe ? NOT a code branch (`EXPERIMENT_HARNESS.md` §4).
79
80     A variant turns one video's persisted candidate windows + features into rally
81     intervals. Every knob defaults to the control behaviour (no gate, no learned
82     filter), so a variant that merely carries a new, disabled cue is byte-identical
83     to control ? the core no-regression guarantee.
84
85     Fields:
86     - ``segmenter`` / ``config_overrides`` / ``cue_flags`` ? informational provenance
87       for the leaderboard and for selecting the served path (the existing
88       ``segmenter_registry`` + ``IndexingConfig`` do the actual selection in production).
89       New cues are recorded here default-OFF.
90     - ``min_crossings`` ? decision-gate Gate A: keep only windows whose persisted
91       ``net_crossings`` feature is  $\geq$  this. 0 = off. This is the eval-only gate from
92       ``rally_gate.py`` expressed as a re-scoring knob (kills service-feed / held-
shuttle
93       windows ? see ``MULTI_SIGNAL_FUSION_PLAN.md` §3.1).
94     - ``min_players`` ? decision-gate Gate B (the future person cue): keep windows
95       whose persisted ``players_in_court`` is  $\geq$  this. 0 = off (no-op until the person

```

```

96         cue persists that feature).
97     - ``classifier_model`` ? path to a frozen `LogisticRegression` JSON. When set,
98       `compare()` scores videos with the SHIPPED model as-is (no per-fold training);
99       required for `compare_unlabeled` on a learned variant.
100    - ``feature_names`` ? the persisted features the learned filter consumes. When set
101      WITHOUT ``classifier_model``, `compare()` trains a fresh model per LOVO fold
102      (honest) ? the recipe, not a pinned artifact.
103    - ``threshold`` ? classifier probability cutoff. ``merge_gap`` ? post-filter
104      overlap-merge gap (seconds), the same `gemini_refine.merge_overlaps` default.
105    """
106
107    name: str
108    segmenter: str = "wasb_hybrid"
109    config_overrides: Dict[str, Any] = field(default_factory=dict)
110    cue_flags: Dict[str, bool] = field(default_factory=dict)
111    min_crossings: int = 0
112    min_players: int = 0
113    classifier_model: Optional[str] = None
114    feature_names: Optional[List[str]] = None
115    threshold: float = 0.5
116    merge_gap: float = 1.0
117    # C2 / threshold-in-LOVO (default OFF ? unchanged behaviour). When set, the
operating
118    # point is chosen on the TRAINING fold, never the held-out one ? fixing the
first-A/B
119    # failure where a fixed 0.5 zeroed out a small-candidate video.
120    tune_threshold: bool = False          # pick the F1-best threshold on the train fold
121    calibrate: Optional[str] = None       # None | "platt" | "isotonic" ? calibrate
proba first
122
123    def is_learned(self) -> bool:
124        """True if this variant applies a learned classifier (pinned model or
recipe)."""
125        return self.classifier_model is not None or bool(self.feature_names)
126
127    def to_dict(self) -> dict:
128        return {
129            "name": self.name,
130            "segmenter": self.segmenter,
131            "config_overrides": self.config_overrides,
132            "cue_flags": self.cue_flags,
133            "min_crossings": self.min_crossings,
134            "min_players": self.min_players,
135            "classifier_model": self.classifier_model,
136            "feature_names": self.feature_names,
137            "threshold": self.threshold,
138            "merge_gap": self.merge_gap,
139            "tune_threshold": self.tune_threshold,
140            "calibrate": self.calibrate,
141        }
142
143    @classmethod
144    def from_dict(cls, d: dict) -> "Variant":
145        known = {f for f in cls.__dataclass_fields__} # type: ignore[attr-defined]
146        return cls(**{k: v for k, v in d.items() if k in known})
147
148    def spec_hash(self) -> str:
149        """Stable 12-char hash of the recipe ? identifies the variant in the leaderboard
150        and makes a result reproducible. The pinned **control** variant always hashes
the
151        same, so a regression is always traceable to a recipe change, never to drift."""
152        blob = json.dumps(self.to_dict(), sort_keys=True, default=str)
153        return sha1(blob.encode()).hexdigest()[:12]
154
155

```

```

156 #: The pinned, frozen baseline: candidates as detected, merged, no gate, no learned
157 #: filter. Always the comparison reference; "rollback" = make this the served variant.
158 CONTROL = Variant(name="control")
159
160
161 # ----- persisted candidates
162
163
164 @dataclass
165 class VideoCandidates:
166     """One video's persisted detection, ready for offline re-scoring.
167
168     ``intervals`` are the candidate windows; ``features`` is column-major
169     (``feature_name -> per-candidate value``, each list aligned to ``intervals``);
170     ``gts`` are golden labels (None for new/unlabeled footage). Build one from the DB
171     with ``load_video_candidates``, or directly in tests.
172     """
173
174     name: str
175     intervals: List[Interval]
176     features: Dict[str, List[float]] = field(default_factory=dict)
177     gts: Optional[List[Interval]] = None
178
179
180 def _feature_column(vc: VideoCandidates, name: str) -> List[float]:
181     """Persisted feature column, defaulting missing/short columns to 0.0 (matches
182     ``training.extract_yolo_features``, which lets a sparse/old row still vectorise)."""
183     col = vc.features.get(name)
184     n = len(vc.intervals)
185     if col is None:
186         logger.warning("variant references feature %r absent from %s ? treating as 0.0",
187                        name, vc.name)
188         return [0.0] * n
189     if len(col) != n:
190         raise ExperimentError(
191             f"feature {name!r} has {len(col)} values but {vc.name} has {n} candidates")
192     return [float(x) for x in col]
193
194
195 def _feature_matrix(vc: VideoCandidates, feature_names: Sequence[str]) ->
List[List[float]]:
196     cols = [_feature_column(vc, name) for name in feature_names]
197     return [list(row) for row in zip(*cols)] if cols else [[] for _ in vc.intervals]
198
199
200 def _gate_mask(variant: Variant, vc: VideoCandidates) -> List[bool]:
201     """Boolean keep-mask from the decision gates (Gate A net-crossings, Gate B
202     players)."""
203     n = len(vc.intervals)
204     mask = [True] * n
205     if variant.min_crossings > 0:
206         col = _feature_column(vc, "net_crossings")
207         mask = [m and (col[i] >= variant.min_crossings) for i, m in enumerate(mask)]
208     if variant.min_players > 0:
209         col = _feature_column(vc, "players_in_court")
210         mask = [m and (col[i] >= variant.min_players) for i, m in enumerate(mask)]
211     return mask
212
213
214 def predict(variant: Variant, vc: VideoCandidates,
215            model: Optional[LogisticRegression] = None,
216            threshold: Optional[float] = None,
217            calibrator: Optional[Callable[[float], float]] = None) -> List[Interval]:
218     """Variant prediction: gate by persisted features, optionally filter by a learned
219     model, then merge. Pure and deterministic.

```

```

219
220 ``model`` (an already-fitted `LogisticRegression`) is supplied by `compare` per LOVO
221 fold; if omitted and the variant pins ``classifier_model``, that frozen model is
222 loaded. A learned variant with neither a supplied nor a pinned model raises ? there
223 is nothing to score with. ``threshold``/``calibrator`` (both fitted on the TRAINING
224 fold) override the variant defaults when the C2 / threshold-in-LOVO path supplies
them.
225 """
226 mask = _gate_mask(variant, vc)
227 if variant.feature_names:
228     if model is None:
229         if variant.classifier_model is None:
230             raise ExperimentError(
231                 f"variant {variant.name!r} is learned but has no model to predict "
232                 f"with (pass a fitted model or set classifier_model)")
233             model = LogisticRegression.load(variant.classifier_model)
234             proba = [float(p) for p in model.predict_proba(_feature_matrix(vc,
variant.feature_names))]
235             if calibrator is not None:
236                 proba = [calibrator(p) for p in proba]
237                 thr = variant.threshold if threshold is None else threshold
238                 mask = [m and (proba[i] >= thr) for i, m in enumerate(mask)]
239             kept = [iv for iv, keep in zip(vc.intervals, mask) if keep]
240             return merge_overlaps(kept, gap_tol=variant.merge_gap)
241
242
243 def _calibrate_and_tune(variant: Variant, model: LogisticRegression,
244                        train_videos: Sequence[VideoCandidates], iou: float
245                        ) -> "tuple[Optional[Callable[[float], float]],
Optional[float]]":
246     """Fit a calibrator and/or pick the operating threshold on the TRAINING fold only
247     (C2 + threshold-in-LOVO). Returns ``(calibrator, threshold)`` ? either may be None
248     (use the variant default). Honest: never looks at the held-out video.
249     """
250     calibrator: Optional[Callable[[float], float]] = None
251     if variant.calibrate:
252         raw: List[float] = []
253         y: List[int] = []
254         for vc in train_videos:
255             raw.extend(float(p) for p in
256                         model.predict_proba(_feature_matrix(vc, variant.feature_names or
[])))
257             y.extend(label_candidates(vc.intervals, vc.gts or [], iou))
258         if variant.calibrate == "platt":
259             calibrator = fit_platt(raw, y)
260         elif variant.calibrate == "isotonic":
261             calibrator = fit_isotonic(raw, y)
262         else:
263             raise ExperimentError(f"unknown calibrate={variant.calibrate!r} (use
'platt'/'isotonic')")
264         threshold: Optional[float] = None
265         if variant.tune_threshold:
266             best_t, best_f1 = variant.threshold, -1.0
267             for k in range(1, 20):
268                 t = round(0.05 * k, 2)
269                 f1s = [score(predict(variant, vc, model=model, threshold=t,
calibrator=calibrator),
270                             vc.gts or [], iou).f1 for vc in train_videos]
271                 mean_f1 = sum(f1s) / len(f1s) if f1s else 0.0
272                 if mean_f1 > best_f1:
273                     best_f1, best_t = mean_f1, t
274             threshold = best_t
275         return calibrator, threshold
276

```

```

277
278 # ----- variant scoring
279
280
281 def _train(variant: Variant, videos: Sequence[VideoCandidates], iou: float) ->
LogisticRegression:
282     """Fit the variant's classifier on the pooled candidates of ``videos`` (labels via
the
283     evaluator's own IoU matcher). The honest-LOVO training step from `distill_local`. """
284     X: List[List[float]] = []
285     y: List[int] = []
286     for vc in videos:
287         if vc.gts is None:
288             raise ExperimentError(f"cannot train: {vc.name} has no golden labels")
289             X.extend(_feature_matrix(vc, variant.feature_names or []))
290             y.extend(label_candidates(vc.intervals, vc.gts, iou))
291     if not X:
292         raise ExperimentError("cannot train a learned variant on zero candidates")
293     return LogisticRegression().fit(X, y, variant.feature_names)
294
295
296 def evaluate_variant(videos: Sequence[VideoCandidates], variant: Variant,
297                     iou: float = 0.5, honest: bool = True) -> Dict[str, Any]:
298     """Score a variant on a labeled corpus. Returns per-video Scores + predictions + a
299     mean aggregate.
300
301     Prediction policy:
302     - **static variant** (no learned filter): predict each video directly ? no
training,
303     so no leakage; per-video scoring is already honest.
304     - **learned, pinned model** (`classifier_model` set): score every video with the
305     SHIPPED model. ? optimistic if those videos were in its training set ? use this
to
306     report "how the shipped artifact does here", not for the promotion decision.
307     - **learned recipe** (`feature_names`, no pinned model) + `honest=True`: LOVO
?
308     train on the OTHER videos, predict the held-out one. The number to trust.
309     - learned recipe + `honest=False`: train on all, predict each (overfit; sanity
only).
310
311     """
312     for vc in videos:
313         if vc.gts is None:
314             raise ExperimentError(f"{vc.name} has no golden labels; use
compare_unlabeled")
315
316     per_video: List[Dict[str, Any]] = []
317     predictions: Dict[str, List[Interval]] = {}
318
319     recipe_lovo = bool(variant.feature_names) and variant.classifier_model is None
320     pinned_model = (LogisticRegression.load(variant.classifier_model)
321                     if variant.classifier_model is not None else None)
322     all_model = None
323     if recipe_lovo and not honest:
324         all_model = _train(variant, videos, iou)
325
326     for i, vc in enumerate(videos):
327         cal: Optional[Callable[[float], float]] = None
328         thr: Optional[float] = None
329         if recipe_lovo and honest:
330             others = [v for j, v in enumerate(videos) if j != i]
331             if not others:
332                 raise ExperimentError("LOVO needs >=2 videos; pass honest=False or a
pinned model")
333             model: Optional[LogisticRegression] = _train(variant, others, iou)
334             if variant.tune_threshold or variant.calibrate: # operating point from

```

```

the train fold
334             assert model is not None                                # _train never returns
None
335             cal, thr = _calibrate_and_tune(variant, model, others, iou)
336         elif recipe_lovo:                                           # honest=False ? one model over all
videos
337             model = all_model
338         else:                                                         # static variant or pinned model
339             model = pinned_model
340             preds = predict(variant, vc, model=model, threshold=thr, calibrator=cal)
341             assert vc.gts is not None                                # guarded at function top
342             sc = score(preds, vc.gts, iou)
343             per_video.append({"video": vc.name, "num_pred": len(preds), **{m: getattr(sc, m)
for m in _METRICS}})
344             predictions[vc.name] = preds
345
346         aggregate = {m: (sum(p[m] for p in per_video) / len(per_video) if per_video else
0.0)
347                     for m in _METRICS}
348     return {
349         "variant": variant.name,
350         "spec_hash": variant.spec_hash(),
351         "is_learned": variant.is_learned(),
352         "honest_lovo": recipe_lovo and honest,
353         "per_video": per_video,
354         "aggregate": aggregate,
355         "predictions": predictions,
356     }
357
358
359     def _ci_block(eval_v: Dict[str, Any]) -> Dict[str, Any]:
360         """Per-metric mean + bootstrap CI over the per-video scores (C1 ? never a bare
mean)."""
361         return {m: _stats.mean_with_ci([p[m] for p in eval_v["per_video"]]) for m in
_METRICS}
362
363
364     def _segmentation_block(videos: Sequence[VideoCandidates], eval_v: Dict[str, Any]) ->
Dict[str, Any]:
365         """Mean F1@k + segment-count ratio across videos (C4 ? the over-segmentation tIoU
hides)."""
366         gts_by_name = {v.name: (v.gts or []) for v in videos}
367         overlaps = _segm.DEFAULT_OVERLAPS
368         f1k: Dict[float, List[float]] = {ov: [] for ov in overlaps}
369         ratios: List[float] = []
370         for name, preds in eval_v.get("predictions", {}).items():
371             gts = gts_by_name.get(name, [])
372             got = _segm.f1_at_overlaps(preds, gts, overlaps)
373             for ov in overlaps:
374                 f1k[ov].append(got[ov])
375             ratios.append(_segm.segment_count_ratio(preds, gts))
376
377         def _mean(xs: List[float]) -> float:
378             return sum(xs) / len(xs) if xs else 0.0
379         out: Dict[str, Any] = {"f1@{int(ov * 100)}": _mean(vals) for ov, vals in
f1k.items()}
380         out["segment_count_ratio"] = _mean(ratios)
381         return out
382
383
384     def honest_summary(videos: Sequence[VideoCandidates], eval_a: Dict[str, Any],
385                       eval_b: Dict[str, Any]) -> Dict[str, Any]:
386         """C1 + C4 honest reporting layered over a compare() pair (additive,
EXPERIMENT_HARNESS $6).
387

```

```

388     ``per_video`` lists are video-aligned (``evaluate_variant`` iterates ``videos`` in
order),
389     so ``paired_delta_f1`` pairs B?A by video ? the promotion-grade view: a lift is real
when
390     the ?F1 CI clears 0 *and* the sign test agrees, not when a bare mean ticked up.
391     """
392     a_f1 = [p["f1"] for p in eval_a["per_video"]]
393     b_f1 = [p["f1"] for p in eval_b["per_video"]]
394     return {
395         "variant_a_ci": _ci_block(eval_a),
396         "variant_b_ci": _ci_block(eval_b),
397         "paired_delta_f1": _stats.paired_delta_ci(a_f1, b_f1),
398         "segmentation": {"variant_a": _segmentation_block(videos, eval_a),
399                         "variant_b": _segmentation_block(videos, eval_b)},
400     }
401
402
403     def compare(videos: Sequence[VideoCandidates], variant_a: Variant, variant_b: Variant,
404                 iou: float = 0.5, honest: bool = True, honest_stats: bool = True) ->
Dict[str, Any]:
405         """Offline labeled A/B (``EXPERIMENT_HARNESS.md`` §5.1). Scores both variants on the
same
406         golden corpus and reports the **B ? A** deltas, per video and in aggregate.
407
408         The deltas use the existing ``metrics.score`` (per video) +
``calibration.pct_improvement``;
409         learned variants are scored LOVO-honestly. The result is a self-contained record fit
for
410         ``record_experiment`` ? variant specs + hashes + the label-set fingerprint travel with
it.
411
412         ``honest_stats`` (default True) **adds** a ``"honest"`` block ? per-metric bootstrap
CIs, a
413         paired ?F1 CI + exact sign test (C1), and F1@k + segment-count ratio (C4) ?
*alongside* the
414         existing bare-mean fields (additive: nothing existing changes). Set False for the
legacy
415         shape. This is the measurement-honesty wiring (``ANALYZERS_AND_RECONCILERS.md``
§13/C1+C4): a
416         bare LOVO mean at n?6 is not trustworthy on its own.
417         """
418         if len(videos) < 1:
419             raise ExperimentError("compare needs at least one labeled video")
420         eval_a = evaluate_variant(videos, variant_a, iou, honest)
421         eval_b = evaluate_variant(videos, variant_b, iou, honest)
422
423         delta = {m: eval_b["aggregate"][m] - eval_a["aggregate"][m] for m in _METRICS}
424         delta_pct = {m: pct_improvement(eval_a["aggregate"][m], eval_b["aggregate"][m]) for
m in _METRICS}
425
426         by_video_a = {p["video"]: p for p in eval_a["per_video"]}
427         per_video_delta = [
428             {"video": p["video"], **{m: p[m] - by_video_a[p["video"]][m] for m in _METRICS}}
429             for p in eval_b["per_video"]]
430     ]
431
432     # One fingerprint over the whole label set ? detects "the deltas were measured
against
433     # different labels" the same way regression.gt_hash guards the no-regression
baseline.
434     all_gts: List[Interval] = [iv for vc in videos for iv in (vc.gts or [])]
435
436     result: Dict[str, Any] = {
437         "iou": iou,
438         "label_set_hash": gt_hash(all_gts),

```

```

439         "num_videos": len(videos),
440         "variant_a": eval_a,
441         "variant_b": eval_b,
442         "delta": delta,
443         "delta_pct": delta_pct,
444         "per_video_delta": per_video_delta,
445     }
446     if honest_stats:
447         result["honest"] = honest_summary(videos, eval_a, eval_b)
448     return result
449
450
451 # ----- SxS on unlabeled video
452
453
454 def compare_unlabeled(video: VideoCandidates, variant_a: Variant, variant_b: Variant,
455                      agree_iou: float = 0.5) -> Dict[str, Any]:
456     """Side-by-side on NEW footage with no ground truth (`EXPERIMENT_HARNESS.md` §5.2).
457
458     With no labels there is no lift to measure ? instead this surfaces where the two
459     variants **agree vs diverge**, which is exactly what a human reviews (and what two
460     highlight reels would show side by side):
461
462     - ``agreed`` ? windows both variants found (matched at ``agree_iou``);
463     - ``a_only`` ? A fired, B suppressed (B's added precision, if A was over-firing);
464     - ``b_only`` ? B fired, A did not (B's new detections ? real recall or new FPs:
465       the human / a later golden label adjudicates).
466
467     Both variants must be predictable without labels: a learned variant therefore needs
468     a
469     pinned ``classifier_model`` (you cannot train on an unlabeled video). Reuses
470     ``metrics.match_intervals`` ? no new matching logic.
471
472     """
473     preds_a = predict(variant_a, video)
474     preds_b = predict(variant_b, video)
475     mr = match_intervals(preds_a, preds_b, agree_iou)
476
477     agreed = [{"a": preds_a[m.pred_index], "b": preds_b[m.gt_index], "iou": m.iou}
478              for m in mr.matches]
479     agree_ious = [m.iou for m in mr.matches]
480     a_only = [preds_a[i] for i in mr.unmatched_pred_indices]
481     b_only = [preds_b[i] for i in mr.unmatched_gt_indices]
482
483     def _dur(ivs: List[Interval]) -> float:
484         return sum(e - s for s, e in ivs)
485
486     return {
487         "video": video.name,
488         "agree_iou": agree_iou,
489         "variant_a": variant_a.name,
490         "variant_b": variant_b.name,
491         "num_a": len(preds_a),
492         "num_b": len(preds_b),
493         "num_agreed": len(agreed),
494         "mean_agree_iou": (sum(agree_ious) / len(agree_ious)) if agree_ious else 0.0,
495         "duration_a": _dur(preds_a),
496         "duration_b": _dur(preds_b),
497         "agreed": agreed,
498         "a_only": a_only,
499         "b_only": b_only,
500     }
501
502 # ----- leaderboard / DB

```

```

503
504 def record_experiment(result: Dict[str, Any], path: str = DEFAULT_LEADERBOARD_PATH,
505                      timestamp: Optional[str] = None) -> Dict[str, Any]:
506     """Append a compact, reproducible record of a `compare()` result to the JSON
507     leaderboard (`EXPERIMENT_HARNESS.md` §6: experiments are records). Generalises
508     `regression_baseline.json`; deliberately the size of a baseline file, not a service
509     (§9). Returns the row written. `timestamp` is injectable for deterministic tests.
510     """
511     row: Dict[str, Any] = {
512         "timestamp": timestamp or datetime.now(timezone.utc).isoformat(),
513         "iou": result.get("iou"),
514         "label_set_hash": result.get("label_set_hash"),
515         "num_videos": result.get("num_videos"),
516         "variant_a": {"name": result["variant_a"]["variant"],
517                     "spec_hash": result["variant_a"]["spec_hash"],
518                     "aggregate": result["variant_a"]["aggregate"]},
519         "variant_b": {"name": result["variant_b"]["variant"],
520                     "spec_hash": result["variant_b"]["spec_hash"],
521                     "aggregate": result["variant_b"]["aggregate"]},
522         "delta": result.get("delta"),
523     }
524     # Record the honest ?F1 (CI + sign test) when present, so the leaderboard carries
525     # the
526     # promotion-grade evidence, not just the bare-mean delta (additive ? C1).
527     honest = result.get("honest") or {}
528     if honest.get("paired_delta_f1"):
529         row["honest_delta_f1"] = honest["paired_delta_f1"]
530     entries: List[Dict[str, Any]] = []
531     if os.path.isfile(path):
532         with open(path, encoding="utf-8") as f:
533             entries = json.load(f)
534     entries.append(row)
535     os.makedirs(os.path.dirname(path) or ".", exist_ok=True)
536     with open(path, "w", encoding="utf-8") as f:
537         json.dump(entries, f, indent=2)
538     logger.info("recorded experiment %s vs %s (?f1=%0.3f) -> %s",
539               row["variant_a"]["name"], row["variant_b"]["name"],
540               (result.get("delta") or {}).get("f1", 0.0), path)
541     return row
542
543 def load_video_candidates(db: Any, video_id: str, *, detector: Optional[str] = None,
544                          gts: Optional[List[Interval]] = None,
545                          feature_names: Sequence[str] = ()) -> VideoCandidates:
546     """Bridge persisted detection ? `VideoCandidates` via
547     `Database.query_candidate_segments`.
548     ``feature_names`` selects which persisted features to lift out of each row's
549     ``stats``
550     JSON (``"duration"`` is derived from the window bounds). This is the only DB-
551     touching
552     function; the comparison itself is pure and offline. Exercised owner-side on a real
553     DB,
554     but works against any `Database` (including an in-memory one in tests).
555     """
556     rows = db.query_candidate_segments(video_id, detector=detector)
557     intervals: List[Interval] = [(float(r["start_time"]), float(r["end_time"])) for r in
558     rows]
559     features: Dict[str, List[float]] = {}
560     for name in feature_names:
561         if name == "duration":
562             features[name] = [float(r["end_time"]) - float(r["start_time"]) for r in
563             rows]
564         else:
565             features[name] = [float((r.get("stats") or {}).get(name, 0.0)) for r in

```

```

rows]
561         return VideoCandidates(name=video_id, intervals=intervals, features=features,
gts=gts)
562

```

9. user (2026-06-14T10:10:19.556Z)

```

1      """Honest small-N evaluation statistics (course-corrections C1 + C3).
2
3      The ~6-match golden corpus + a regularised logistic arbiter is exactly the regime where
a
4      single leave-one-video-out *mean* lies: leave-one-out anti-correlates each fold's
training
5      and held-out label means at small N, and one lucky/unlucky fold swings the headline. The
6      ``compare``/LOVO machinery already produces **per-video held-out scores** ? this module
7      turns those into an honest summary:
8
9      - **C1 ? never a single mean.** ``mean_with_ci`` / ``bootstrap_ci`` report a confidence
10     interval; ``paired_sign_test`` and ``paired_delta_ci`` grade a B?A improvement by how
11     many *videos* it wins on (the distribution-free test the repo already leans on:
12     "sign test needs n ? ~n=10/9-wins for p<0.05"), so the promotion gate can't promote
noise.
13     - **C3 ? honest stacking.** ``out_of_fold_predictions`` / ``grouped_folds`` give a
learned
14     producer's *out-of-fold* outputs so feeding them to the arbiter doesn't leak (group by
15     match, never a held-out window from the same video).
16
17     See ``docs/ANALYZERS_AND_RECONCILERS.md`` §13/C1+C3. All **additive**, pure (stdlib
only),
18     deterministic (bootstrap takes an explicit ``seed``) ? nothing here changes existing
19     ``compare``/``calibration.leave_one_video_out`` behaviour; callers opt in.
20     """
21     from __future__ import annotations
22
23     import math
24     import random
25     import statistics
26     from typing import Any, Callable, Dict, List, Optional, Sequence, Tuple
27
28     Interval = Tuple[float, float]
29     FitPredict = Callable[[List[int], List[int]], List[Any]]
30
31
32     def bootstrap_ci(values: Sequence[float], confidence: float = 0.95,
33                     n_boot: int = 2000, seed: int = 0) -> Tuple[float, float]:
34         """Percentile bootstrap CI for the mean of ``values``. Deterministic given ``seed``.
35
36         ``n<=1`` returns a degenerate ``(v, v)`` / ``(0, 0)`` interval ? there is no spread
to
37         resample. With n=6 (our corpus) this is the honest "how wide could the mean be?"
band.
38         """
39         vals = [float(v) for v in values]
40         n = len(vals)
41         if n == 0:
42             return (0.0, 0.0)
43         if n == 1:
44             return (vals[0], vals[0])
45         rng = random.Random(seed)
46         means: List[float] = []
47         for _ in range(max(1, n_boot)):
48             means.append(sum(vals[rng.randrange(n)] for _ in range(n)) / n)
49         means.sort()
50         alpha = (1.0 - confidence) / 2.0
51         last = len(means) - 1

```

```

52     lo = means[max(0, int(math.floor(alpha * last)))]
53     hi = means[min(last, int(math.ceil((1.0 - alpha) * last)))]
54     return (lo, hi)
55
56
57 def mean_with_ci(values: Sequence[float], confidence: float = 0.95,
58                 n_boot: int = 2000, seed: int = 0) -> Dict[str, float]:
59     """`{mean, std, n, ci_low, ci_high, confidence}` ? report this, never a bare mean
(C1)."""
60     vals = [float(v) for v in values]
61     n = len(vals)
62     mean = statistics.mean(vals) if vals else 0.0
63     std = statistics.pstdev(vals) if n > 1 else 0.0
64     lo, hi = bootstrap_ci(vals, confidence, n_boot, seed)
65     return {"mean": mean, "std": std, "n": float(n),
66           "ci_low": lo, "ci_high": hi, "confidence": confidence}
67
68
69 def paired_sign_test(deltas: Sequence[float], eps: float = 0.0) -> Dict[str, float]:
70     """Two-sided exact sign test over per-fold deltas (B ? A).
71
72     ``wins`` = folds where B beat A by more than ``eps``; ``losses`` the reverse; ties
73     excluded. ``p_value`` is the exact two-sided binomial tail under p=0.5 ? the
74     distribution-free "did B win on enough *videos*" check.
75     """
76     wins = sum(1 for d in deltas if d > eps)
77     losses = sum(1 for d in deltas if d < -eps)
78     ties = len(deltas) - wins - losses
79     n_eff = wins + losses
80     if n_eff == 0:
81         p = 1.0
82     else:
83         k = min(wins, losses)
84         tail = sum(math.comb(n_eff, j) for j in range(k + 1)) * (0.5 ** n_eff)
85         p = min(1.0, 2.0 * tail)
86     return {"wins": float(wins), "losses": float(losses), "ties": float(ties),
87           "n_effective": float(n_eff), "p_value": p}
88
89
90 def paired_delta_ci(a_values: Sequence[float], b_values: Sequence[float],
91                    confidence: float = 0.95, n_boot: int = 2000, seed: int = 0,
92                    eps: float = 0.0) -> Dict[str, Any]:
93     """Paired (by fold) B ? A summary: mean delta + bootstrap CI + the sign test (C1).
94
95     ``a_values``/``b_values`` are per-fold held-out scores for variants A and B, aligned
96     by
97     fold (same video order). A promotion is honest when the delta CI clears 0 *and* the
98     sign
99     test agrees ? not when a single mean ticked up.
100     """
101     n = min(len(a_values), len(b_values))
102     deltas = [float(b_values[i]) - float(a_values[i]) for i in range(n)]
103     summary = mean_with_ci(deltas, confidence, n_boot, seed)
104     return {
105         "mean_delta": summary["mean"],
106         "ci_low": summary["ci_low"],
107         "ci_high": summary["ci_high"],
108         "n": float(n),
109         "ci_excludes_zero": (summary["ci_low"] > 0.0) or (summary["ci_high"] < 0.0),
110         "sign_test": paired_sign_test(deltas, eps),
111     }
112
113 def grouped_folds(groups: Sequence[Any]) -> List[Tuple[List[int], List[int]]]:
114     """Leave-one-group-out splits ``(train_idx, test_idx)`` ? group by match, the honest

```

```

unit.
114
115     The field's name for "LOVO by match" is leave-one-group-out (LOGO). Group order is
116     first-seen (deterministic). Groups with no complement (a single group) yield no
fold.
117     """
118     seen: List[Any] = []
119     for g in groups:
120         if g not in seen:
121             seen.append(g)
122     n = len(groups)
123     folds: List[Tuple[List[int], List[int]]] = []
124     for g in seen:
125         test = [i for i in range(n) if groups[i] == g]
126         train = [i for i in range(n) if groups[i] != g]
127         if test and train:
128             folds.append((train, test))
129     return folds
130
131
132     def out_of_fold_predictions(groups: Sequence[Any], fit_predict: FitPredict) ->
List[Optional[Any]]:
133         """Out-of-fold predictions for honest stacking (C3).
134
135         For each leave-one-group-out fold, ``fit_predict(train_idx, test_idx)`` trains on
the
136         other groups and returns predictions for ``test_idx`` (in that order); results are
scattered back to a list aligned to the input items. So a *learned* producer's
137         outputs,
138         when used as arbiter features, never include in-sample predictions ? the leakage
path
139         that inflates measured lift on a tiny corpus. Items in a singleton group stay
``None``
140         (needs >= 2 groups to predict).
141         """
142         n = len(groups)
143         oof: List[Optional[Any]] = [None] * n
144         for train_idx, test_idx in grouped_folds(groups):
145             preds = fit_predict(train_idx, test_idx)
146             for pos, ti in enumerate(test_idx):
147                 if pos < len(preds):
148                     oof[ti] = preds[pos]
149         return oof
150

```

10. assistant (2026-06-14T10:10:24.076Z)

Now let me read the remaining key files in parallel: fusion_compare.py, per_user_gate.py, segmentation_metrics.py, fusion_golden.py, and the quality_report.py reportlab pattern.

11. user (2026-06-14T10:10:24.943Z)

```

1     """Pure-CPU LOVO comparison: shuttle-only vs shuttle+person learned fusion on the golden
set.
2
3     The fusion-P3 measurement. Loads the per-video feature JSONs written by
`fusion_golden.py`
4     and runs the SHIPPED experiment harness (`experiment.compare`, honest leave-one-video-
out) ?
5     no new scoring/labeling math. The two variants differ ONLY by `feature_names` (the
harness's
6     designed seam): the person features are extra columns the learned LogisticRegression
weights,
7     so the held-out ?F1 is an honest answer to "does the person cue help?" ? not a hunch.
8

```

```

9      Run (no GPU):
10      python -m backend.eval.fusion_compare --features-dir output --record
11      """
12      from __future__ import annotations
13
14      import argparse
15      import glob
16      import json
17      import os
18      from typing import Any, Dict, List
19
20      from backend.eval import experiment
21      from backend.eval.eval_stats import paired_delta_ci
22      from backend.eval.fusion_golden import PERSON_FEATURE_NAMES, SHUTTLE_FEATURE_NAMES
23
24
25      def delta_stats(res: Dict[str, Any], metric: str = "f1") -> Dict[str, Any]:
26          """Paired (by video) B?A summary for one metric: mean delta + bootstrap CI + sign
test
27          (eval_stats, C1 ? "never a bare mean"). Aligns the two variants' per-video held-out
scores
28          by video name, so the headline ?F1 comes with how-wide + did-it-win-on-enough-
videos."""
29          by_a = {p["video"]: p[metric] for p in res["variant_a"]["per_video"]}
30          by_b = {p["video"]: p[metric] for p in res["variant_b"]["per_video"]}
31          vids = [p["video"] for p in res["variant_a"]["per_video"]]
32          return paired_delta_ci([by_a[v] for v in vids], [by_b[v] for v in vids])
33
34
35      def load_videos(features_dir: str) -> List[experiment.VideoCandidates]:
36          """Load every ``*_golden_features.json`` into a VideoCandidates (intervals + the 10
37          feature columns + golden gts). The harness derives labels from gts itself."""
38          videos: List[experiment.VideoCandidates] = []
39          for path in sorted(glob.glob(os.path.join(features_dir, "*_golden_features.json"))):
40              with open(path, encoding="utf-8") as f:
41                  d = json.load(f)
42                  cands = d["candidates"]
43                  intervals = [(float(c["start"]), float(c["end"])) for c in cands]
44                  features: Dict[str, List[float]] = {}
45                  for fn in SHUTTLE_FEATURE_NAMES:
46                      features[fn] = [float(c["shuttle"][fn]) for c in cands]
47                  for fn in PERSON_FEATURE_NAMES:
48                      features[fn] = [float(c["person"][fn]) for c in cands]
49                  gts = [(float(a), float(b)) for a, b in d["gts"]]
50                  videos.append(experiment.VideoCandidates(name=d["name"], intervals=intervals,
51                                                              features=features, gts=gts))
52          return videos
53
54
55      def compare(videos: List[experiment.VideoCandidates], iou: float = 0.5,
56                  threshold: float = 0.5) -> Dict[str, Any]:
57          shuttle_only = experiment.Variant(name="shuttle_only",
feature_names=list(SHUTTLE_FEATURE_NAMES),
58                  threshold=threshold)
59          fusion = experiment.Variant(name="shuttle_person_fusion",
60                  feature_names=list(SHUTTLE_FEATURE_NAMES) +
list(PERSON_FEATURE_NAMES),
61                  threshold=threshold)
62          return experiment.compare(videos, shuttle_only, fusion, iou=iou, honest=True)
63
64
65      def format_result(res: Dict[str, Any], iou: float) -> str:
66          a, b, d = res["variant_a"]["aggregate"], res["variant_b"]["aggregate"], res["delta"]
67          ds = delta_stats(res)
68          st = ds["sign_test"]

```

```

69         lines = [
70             f"=== Fusion P3 LOVO (n={res['num_videos']} golden videos, IoU>={iou}, honest)
===",
71             f"  shuttle-only : F1={a['f1']:.3f}  P={a['precision']:.3f}  R={a['recall']:.3f}
IoU={a['mean_iou']:.3f}",
72             f"  +person      : F1={b['f1']:.3f}  P={b['precision']:.3f}  R={b['recall']:.3f}
IoU={b['mean_iou']:.3f}",
73             f"  ?F1 = {d['f1']:+.3f}  ({res['delta_pct']['f1']:+.1f}%)
?P={d['precision']:+.3f}  ?R={d['recall']:+.3f}",
74             f"  paired ?F1 {ds['mean_delta']:+.3f}  [95% CI {ds['ci_low']:+.3f},
{ds['ci_high']:+.3f}]  "
75             f"| sign test {int(st['wins'])}W/{int(st['losses'])}L (p={st['p_value']:.3f})  "
76             f"| CI clears 0: {ds['ci_excludes_zero']}",
77             "  per-video ?F1:",
78         ]
79         for pv in res["per_video_delta"]:
80             lines.append(f"      {pv['video']:<26s} ?F1={pv['f1']:+.3f}")
81         return "\n".join(lines)
82
83
84     def main() -> None:
85         ap = argparse.ArgumentParser(description="LOVO fusion lift: shuttle-only vs
shuttle+person (golden set).")
86         ap.add_argument("--features-dir", default="output")
87         ap.add_argument("--iou", type=float, default=0.5)
88         ap.add_argument("--record", action="store_true", help="append the result to the
experiment leaderboard")
89         args = ap.parse_args()
90
91         videos = load_videos(args.features_dir)
92         if len(videos) < 2:
93             raise SystemExit(f"need >=2 *_golden_features.json in {args.features_dir} (found
{len(videos)})")
94         res = compare(videos, iou=args.iou)
95         print(format_result(res, args.iou))
96         if args.record:
97             row = experiment.record_experiment(res)
98             print(f"recorded -> eval_baselines/experiment_leaderboard.json
({row['timestamp']}")
99
100
101     if __name__ == "__main__":
102         main()
103

```

12. user (2026-06-14T10:10:25.418Z)

```

1     """Per-user promotion gate ? the safety rail for flag-gated personalization.
2
3     The owner adopted personalization as a flag-gated feature on a generalization-
first
4     baseline: a cue/variant B may be flipped ON for one user only when that user's own
5     held-out evidence honestly clears the same small-N bar the global eval uses ? otherwise
we
6     fall back to the shipped baseline (variant A). This module is the decision logic for
that
7     flip; it is pure (no I/O) and reuses ``backend.eval.eval_stats`` for every statistic
8     (``paired_delta_ci`` + ``paired_sign_test``) so the math is never reinvented.
9
10    The counter-case this guards against (and its mitigations, implemented here):
11
12    1. Minimum-N floor. Personalizing off 1-2 of a user's videos is the small-N-lies
regime
13    (see ``eval_stats`` C1). Refuse to promote below ``min_n`` videos ? point estimates
from

```

```

14         too few held-out videos are noise, not signal.
15     2. **Honest lift, not a single mean.** Promote only when the paired ?F1 95% bootstrap CI
16         **excludes 0** *and* the paired sign test agrees (majority of videos win, p <=
threshold).
17         This is the global promotion bar applied per user.
18     3. **Structural guards are never disabled on "no lift".** A *structural* guard (e.g.
Gate-A
19         net-crossings ? the held-shuttle false-positive killer) earns its keep through
failure
20         modes a given user's corpus may not contain yet. Measuring "no per-user lift" on a
corpus
21         that simply lacks those failure cases must NEVER turn the guard OFF. For
`structural=True`
22         the decision is therefore **always "keep ON"**, independent of the measured delta.
23
24         Below the floor / insufficient evidence ? fall back to the shipped baseline (the
conservative
25         default), never an unsafe promotion. All additive, pure, deterministic (the bootstrap
seed is
26         threaded through). Nothing here changes existing eval behaviour; callers opt in.
27         ""
28         from __future__ import annotations
29
30         from dataclasses import dataclass, field
31         from typing import Any, Dict, Sequence
32
33         from backend.eval.eval_stats import paired_delta_ci, paired_sign_test
34
35         __all__ = ["PromotionDecision", "should_promote_for_user"]
36
37
38     @dataclass
39     class PromotionDecision:
40         """Structured result of a per-user promotion decision.
41
42         - ``promote`` ? flip variant B ON for this user (only ever ``True`` for a non-
structural
43         cue that cleared the full bar).
44         - ``keep_on`` ? the cue/guard should be ON for this user. For a structural guard
this is
45         always ``True`` (it is never disabled on "no lift"); for a non-structural cue it
equals
46         ``promote``.
47         - ``reason`` ? short human-readable explanation of the decision.
48         - the remaining fields surface the evidence (n, mean delta, CI bounds and the two
tests)
49         so a caller can log/audit exactly why a user was or wasn't personalized.
50         ""
51
52         promote: bool
53         keep_on: bool
54         reason: str
55         n: int
56         mean_delta: float
57         ci_low: float
58         ci_high: float
59         ci_excludes_zero: bool
60         sign_test: Dict[str, float] = field(default_factory=dict)
61         floor_met: bool = False
62         structural_protected: bool = False
63
64         def as_dict(self) -> Dict[str, Any]:
65             """Plain-dict view (for JSON logging / telemetry)."""
66             return {
67                 "promote": self.promote,

```

```

68         "keep_on": self.keep_on,
69         "reason": self.reason,
70         "n": self.n,
71         "mean_delta": self.mean_delta,
72         "ci_low": self.ci_low,
73         "ci_high": self.ci_high,
74         "ci_excludes_zero": self.ci_excludes_zero,
75         "sign_test": self.sign_test,
76         "floor_met": self.floor_met,
77         "structural_protected": self.structural_protected,
78     }
79
80
81     def should_promote_for_user(
82         a_scores: Sequence[float],
83         b_scores: Sequence[float],
84         *,
85         structural: bool = False,
86         min_n: int = 5,
87         p_threshold: float = 0.05,
88         confidence: float = 0.95,
89         n_boot: int = 2000,
90         seed: int = 0,
91         eps: float = 0.0,
92     ) -> PromotionDecision:
93         """Decide whether to flip variant B ON for ONE user from that user's held-out
94         scores.
95         ``a_scores`` / ``b_scores`` are the user's **per-video** held-out F1 (or any score)
96         for
97         variant A (control / shipped baseline) and variant B (with the cue), aligned by
98         video
99         (same order). The decision rules (faithful to the module docstring):
100         1. ``structural=True`` ? the guard is **never disabled on "no lift"**: return
101         ``keep_on=True, promote=False`` regardless of the measured per-user delta. Its
102         value is
103         in failure modes the user's corpus may not yet contain.
104         2. Otherwise promote **iff ALL** hold ? the minimum-N floor (``n >= min_n``), the
105         paired
106         ?F1 bootstrap CI **excludes 0**, and the paired sign test agrees (majority of
107         videos
108         win, ``p_value <= p_threshold``). Any miss ? fall back to the shipped baseline.
109
110         Statistics come straight from ``eval_stats`` (``paired_delta_ci`` /
111         ``paired_sign_test``);
112         deterministic given ``seed``. Returns a :class:`PromotionDecision`.
113         """
114         n = min(len(a_scores), len(b_scores))
115         floor_met = n >= min_n
116
117         # --- Evidence (reuse eval_stats; never reinvent the statistics)
118         -----
119         if n == 0:
120             # No paired videos at all ? nothing to test. Surface an empty-but-honest result.
121             sign_test: Dict[str, float] = paired_sign_test([], eps)
122             mean_delta, ci_low, ci_high, ci_excludes_zero = 0.0, 0.0, 0.0, False
123         else:
124             delta = paired_delta_ci(
125                 a_scores, b_scores,
126                 confidence=confidence, n_boot=n_boot, seed=seed, eps=eps,
127             )
128             mean_delta = float(delta["mean_delta"])
129             ci_low = float(delta["ci_low"])
130             ci_high = float(delta["ci_high"])

```

```

125         ci_excludes_zero = bool(delta["ci_excludes_zero"])
126         sign_test = delta["sign_test"]
127
128     sign_p = float(sign_test.get("p_value", 1.0))
129     sign_wins = float(sign_test.get("wins", 0.0))
130     sign_losses = float(sign_test.get("losses", 0.0))
131     # The sign test "agrees" with a promotion only if it is significant AND the majority
132     # leans the *winning* way (more B-wins than B-losses) ? a significant net *loss*
must
133     # never satisfy the promotion bar.
134     sign_agrees = (sign_p <= p_threshold) and (sign_wins > sign_losses)
135
136     # --- Rule 1: structural guards are never disabled on "no lift"
    -----
137     if structural:
138         return PromotionDecision(
139             promote=False,          # nothing to "promote" ? it ships ON by design
140             keep_on=True,           # ALWAYS kept ON, regardless of measured per-user
lift
141             reason=(
142                 "structural guard kept ON regardless of measured per-user lift "
143                 "(value is in failure modes this user's corpus may not yet contain)"
144             ),
145             n=n,
146             mean_delta=mean_delta,
147             ci_low=ci_low,
148             ci_high=ci_high,
149             ci_excludes_zero=ci_excludes_zero,
150             sign_test=sign_test,
151             floor_met=floor_met,
152             structural_protected=True,
153         )
154
155     # --- Rule 2: minimum-N floor (refuse before even looking at lift)
    -----
156     if not floor_met:
157         return PromotionDecision(
158             promote=False,
159             keep_on=False,          # fall back to the shipped baseline (variant A)
160             reason=(
161                 f"below minimum-N floor (n={n} < min_n={min_n}); "
162                 "falling back to shipped baseline"
163             ),
164             n=n,
165             mean_delta=mean_delta,
166             ci_low=ci_low,
167             ci_high=ci_high,
168             ci_excludes_zero=ci_excludes_zero,
169             sign_test=sign_test,
170             floor_met=False,
171             structural_protected=False,
172         )
173
174     # --- Rule 2 (cont.): honest lift ? CI excludes 0 AND sign test agrees
    -----
175     if ci_excludes_zero and sign_agrees:
176         promote = True
177         reason = (
178             f"promoted: n={n} >= {min_n}, ?F1 95% CI "
179             f"[{ci_low:.4f}, {ci_high:.4f}] excludes 0, "
180             f"sign test {int(sign_wins)}-{int(sign_losses)} (p={sign_p:.4f} <=
{p_threshold})"
181         )
182     else:
183         promote = False

```

```

184         misses = []
185         if not ci_excludes_zero:
186             misses.append(f"?F1 95% CI [{ci_low:.4f}, {ci_high:.4f}] spans 0")
187         if not sign_agrees:
188             misses.append(
189                 f"sign test does not agree ({int(sign_wins)}-{int(sign_losses)}), "
190                 f"p={sign_p:.4f}")
191         )
192         reason = (
193             "insufficient evidence (" + "; ".join(misses) + "); "
194             "falling back to shipped baseline"
195         )
196
197     return PromotionDecision(
198         promote=promote,
199         keep_on=promote,          # non-structural: keep_on tracks promote exactly
200         reason=reason,
201         n=n,
202         mean_delta=mean_delta,
203         ci_low=ci_low,
204         ci_high=ci_high,
205         ci_excludes_zero=ci_excludes_zero,
206         sign_test=sign_test,
207         floor_met=True,
208         structural_protected=False,
209     )
210

```

13. user (2026-06-14T10:10:25.794Z)

```

1     """Segmentation-aware metrics (course-correction C4) ? make over-segmentation visible.
2
3     Temporal IoU (`metrics.score`) is nearly blind to *fragmentation*: a single rally
4     chopped into three predicted pieces, or two rallies merged across a dead-time gap, can
5     still post a moderate mean-IoU. The temporal-action-segmentation / -localization
6     literature grades those failure modes with **F1@k** (segmental F1 at several overlap
7     thresholds) and **mAP@tIoU** (ranked, confidence-aware average precision over a tIoU
8     sweep). Reporting these alongside temporal-IoU is what lets a reconciler's split/merge/
9     tighten win actually show up ? and stops an IoU-chasing rule from shredding rally
10    structure invisibly. See ``docs/ANALYZERS_AND_RECONCILERS.md`` §13/C4.
11
12    This is **additive**: new functions over the existing ``metrics`` matcher; nothing here
13    changes ``score``/``compare``/LOVO behaviour. Pure (stdlib only), deterministic, no I/O
14    ?
15
16    so it runs in the GPU-free / numpy-free lanes too.
17
18    Conventions match ``metrics``: an ``Interval`` is a ``(start, end)`` tuple of seconds.
19    A ``ScoredInterval`` is ``(interval, confidence)`` ? the confidence is whatever the
20    scorer
21    emits per predicted rally (e.g. the classifier probability), used only to *rank* for AP.
22
23    Note on the segmental *edit* score: the classic TAS edit/Levenshtein score operates on
24    the
25    ordered sequence of segment *labels* and is degenerate for a single action class (every
26    rally carries the same label), so it is intentionally omitted ? F1@k + mAP@tIoU + the
27    segment-count ratio are the over-segmentation-sensitive metrics for this binary setting.
28    """
29
30    from __future__ import annotations
31
32    from typing import Dict, List, Sequence, Tuple
33
34    from backend.eval.metrics import Interval, interval_iou, score
35
36    #: A predicted interval plus the confidence used to rank it for average precision.
37    ScoredInterval = Tuple[Interval, float]

```

```

33
34 #: Default overlap thresholds for F1@k ? the TAS convention F1@{10,25,50}.
35 DEFAULT_OVERLAPS: Tuple[float, ...] = (0.1, 0.25, 0.5)
36 #: Default tIoU sweep for mean average precision ? the action-detection convention.
37 DEFAULT_TIOUS: Tuple[float, ...] = (0.3, 0.4, 0.5, 0.6, 0.7)
38
39
40 def f1_at_overlaps(preds: List[Interval], gts: List[Interval],
41                   overlaps: Sequence[float] = DEFAULT_OVERLAPS) -> Dict[float, float]:
42     """Segmental F1 at each overlap threshold (`{overlap: f1}`) ? the TAS F1@k.
43
44     Reuses the canonical greedy one-to-one matcher, so a fragmented prediction (3 pieces
45     for 1 rally) scores extra false positives and a merged prediction misses a rally ?
46     both drag F1 down where temporal-IoU alone would not flinch.
47     """
48     return {ov: score(preds, gts, iou_threshold=ov).f1 for ov in overlaps}
49
50
51 def average_precision(scored_preds: Sequence[ScoredInterval], gts: List[Interval],
52                       iou_threshold: float = 0.5) -> float:
53     """Average precision at a single tIoU (all-points / VOC-2010 interpolation).
54
55     Predictions are ranked by descending confidence; each is a true positive iff it
56     claims a still-unmatched ground-truth rally at IoU >= `iou_threshold` (highest
57     available IoU wins, one GT per prediction). Returns AP in `[0, 1]`. `0.0` when
58     there are no ground-truth rallies or no predictions reach the threshold.
59     """
60     n_gt = len(gts)
61     if n_gt == 0 or not scored_preds:
62         return 0.0
63
64     order = sorted(range(len(scored_preds)),
65                   key=lambda i: (-scored_preds[i][1], i)) # conf desc, stable by
index
66     matched_gt = set()
67     precisions: List[float] = []
68     recalls: List[float] = []
69     cum_tp = 0
70     cum_fp = 0
71     for i in order:
72         interval, _conf = scored_preds[i]
73         best_iou, best_gi = 0.0, -1
74         for gi, g in enumerate(gts):
75             if gi in matched_gt:
76                 continue
77             iou = interval_iou(interval, g)
78             if iou >= iou_threshold and iou > best_iou:
79                 best_iou, best_gi = iou, gi
80         if best_gi >= 0:
81             matched_gt.add(best_gi)
82             cum_tp += 1
83         else:
84             cum_fp += 1
85         precisions.append(cum_tp / (cum_tp + cum_fp))
86         recalls.append(cum_tp / n_gt)
87
88     # Monotone precision envelope (take the max precision to the right), then integrate
89     # over recall steps: AP = sum_k (recall_k - recall_{k-1}) * precision_env_k.
90     for i in range(len(precisions) - 2, -1, -1):
91         precisions[i] = max(precisions[i], precisions[i + 1])
92     ap = 0.0
93     prev_recall = 0.0
94     for p, r in zip(precisions, recalls):
95         if r > prev_recall:
96             ap += p * (r - prev_recall)

```

```

97         prev_recall = r
98     return ap
99
100
101     def mean_average_precision(scored_preds: Sequence[ScoredInterval], gts: List[Interval],
102                               iou_thresholds: Sequence[float] = DEFAULT_TIOUS) -> float:
103         """Mean of ``average_precision`` over a tIoU sweep (the standard mAP@tIoU)."""
104         tious = list(iou_thresholds)
105         if not tious:
106             return 0.0
107         return sum(average_precision(scored_preds, gts, t) for t in tious) / len(tious)
108
109
110     def segment_count_ratio(preds: List[Interval], gts: List[Interval]) -> float:
111         """Predicted-to-GT segment-count ratio ? a crude fragmentation diagnostic.
112
113         ``> 1`` hints over-segmentation (more predicted pieces than rallies), ``< 1`` hints
114         merging/misses. Only meaningful with ground truth present; returns ``0.0`` if
115         ``gts``
116         is empty (use F1@k / mAP for the graded picture).
117         """
118         if not gts:
119             return 0.0
120         return len(preds) / len(gts)
121
122     def segmentation_report(scored_preds: Sequence[ScoredInterval], gts: List[Interval],
123                             overlaps: Sequence[float] = DEFAULT_OVERLAPS,
124                             iou_thresholds: Sequence[float] = DEFAULT_TIOUS) -> Dict[str,
125 object]:
126         """One-call bundle: F1@k + mAP@tIoU + per-tIoU AP + count ratio.
127
128         ``scored_preds`` carry confidences for AP; F1@k uses the bare intervals. Returns a
129         plain dict (JSON-friendly, string keys) so it can ride next to a ``compare`` result
130         or land in the experiment leaderboard.
131         """
132         preds = [iv for iv, _conf in scored_preds]
133         return {
134             "f1_at_overlaps": {f"{ov:g}": v for ov, v in f1_at_overlaps(preds, gts,
135 overlaps).items()},
136             "ap_at_tiou": {f"{t:g}": average_precision(scored_preds, gts, t) for t in
137 iou_thresholds},
138             "mean_ap": mean_average_precision(scored_preds, gts, iou_thresholds),
139             "segment_count_ratio": segment_count_ratio(preds, gts),
140             "num_pred": len(preds),
141             "num_gt": len(gts),
142         }

```

14. assistant (2026-06-14T10:10:32.274Z)

Now let me read fusion_golden.py (the feature JSON shape) and the quality_report.py reportlab pattern.

15. user (2026-06-14T10:10:33.241Z)

```

1     """Offline golden-set person-feature extractor ? the fusion-P3 measurement's GPU step.
2
3     Mirrors `distill_local`'s CSV-driven flow but ADDS the ~5 person-cue features per
4     shuttle
5     candidate window, by running `PersonDetector` over the ORIGINAL video. It reuses the
6     EXISTING golden trajectory CSVs (so **no WASB re-run**), needs **no Gemini and no DB**,
7     and
8     writes one small JSON per video. The LOVO comparison (`fusion_compare.py`) then runs
9     100% on

```

```

7      CPU offline. So the GPU touches exactly one thing ? person detection ? and the lift is
8      measured by the shipped experiment harness; person features enter only as feature
columns the
9      learned scorer weights (no special-casing, default-OFF until the golden ?F1 justifies
them).
10
11      Run (on the GPU box):
12      python -m backend.eval.fusion_golden --manifest output/human_lovo_manifest.json
--out-dir output
13
14      Each output `output/<name>_golden_features.json` is the replayable feature substrate ?
re-run
15      `fusion_compare` with different variants/thresholds forever without re-detecting on the
GPU.
16      """
17      from __future__ import annotations
18
19      import argparse
20      import json
21      import os
22      import time
23      from typing import Any, Dict, List, Optional
24
25      from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
26      from backend.pipeline.detectors import rally_gate
27      from backend.pipeline.detectors import fusion_features as ff
28      from backend.eval import distill_local
29      from backend.eval.calibrate_wasb import load_golden_gts
30      from backend.eval.training import label_candidates
31      from backend.utils.run_telemetry import RunTelemetry
32
33      # The 5 fps/resolution-invariant shuttle features distill_local already computes.
34      SHUTTLE_FEATURE_NAMES = list(distill_local.FEATURE_NAMES)
35      # The 5 person-cue features (fusion_features.PERSON_FEATURE_NAMES).
36      PERSON_FEATURE_NAMES = list(ff.PERSON_FEATURE_NAMES)
37
38
39      def _derive_video_path(spec: Dict[str, Any]) -> str:
40          """Original MP4 for a manifest entry ? explicit `video_path`, else derived from the
41          golden labels path (`X.rallies.csv` -> `X.MP4`, the golden-set naming
convention)."""
42          if spec.get("video_path"):
43              return str(spec["video_path"])
44          labels = str(spec["labels"])
45          if labels.endswith(".rallies.csv"):
46              return labels[: -len(".rallies.csv")] + ".MP4"
47          raise ValueError(f"{spec.get('name')}: no video_path and labels {labels!r} isn't
*.rallies.csv")
48
49
50      def extract_video(spec: Dict[str, Any], detector: Optional[Any] = None,
51                      frame_skip: int = 3, iou: float = 0.5,
52                      log_every_sec: float = 15.0,
53                      telemetry: Optional[Any] = None) -> Dict[str, Any]:
54          """One golden video -> a feature record (shuttle + person features per candidate
window).
55
56          Returns `{name, fps, frame_width,
candidates: [{start, end, shuttle{}, person{}, label}], gts}`.
57          `detector` (anything with `detections_over_windows`) is injectable for GPU-free
tests;
58          when None a real `PersonDetector` is built (loads torchvision on first use).
59          `telemetry` (a `RunTelemetry` or None) records per-frame velocity + machine-health
snapshots
60          from the detection progress callback so a hard kill leaves a forensic runlog.

```

```

61     """
62     fps, fw = float(spec["fps"]), float(spec["frame_width"])
63     traj = str(spec["trajectory"])
64     points = TrackNetRunner.parse_trajectory_csv(traj)
65     intervals, x_shuttle = distill_local.build_candidates(traj, fps, fw)
66     gts = load_golden_gts(str(spec["labels"]))
67     labels = label_candidates(intervals, gts, iou)
68     # Net axis for the person two-sided split ? reuse rally_gate's camera-agnostic
estimate
69     # (parity with the shuttle net-crossing gate) rather than hard-coding an
orientation.
70     axis, net_px, _span = rally_gate.estimate_play_axis(points)
71     video_path = _derive_video_path(spec)
72
73     if detector is None:
74         from backend.pipeline.detectors.person_detector import PersonDetector
75         detector = PersonDetector({"indexing": {"use_gpu": True}})
76
77     # Single-pass person detection over ALL candidate windows (open the video ONCE, read
78     # forward ? no per-window seek). Then compute features per window (pure, no GPU).
79     n = len(intervals)
80     win_frames = [(round(s * fps), round(e * fps)) for (s, e) in intervals]
81     print(f"[fusion_golden]   {spec['name']}: {n} windows ? person-detecting in one
pass...", flush=True)
82     t0 = time.monotonic()
83     state = {"last": t0}
84
85     def _progress(done: int, total: int) -> None:
86         if telemetry is not None:
87             telemetry.snapshot(done, total, phase=spec["name"])
88             now = time.monotonic()
89             if now - state["last"] >= log_every_sec or done >= total:
90                 el = now - t0
91                 rate = done / el if el > 0 else 0.0
92                 eta = (total - done) / rate if rate > 0 else 0.0
93                 pct = 100 * done // total if total else 0
94                 print(f"[fusion_golden]   {spec['name']}: detect frame {done}/{total}
({pct}%) "
95                     f"| {el:.0f}s elapsed | ETA {eta:.0f}s", flush=True)
96                 state["last"] = now
97
98     per_window = detector.detections_over_windows(video_path, win_frames, frame_skip,
99                                                    progress_cb=_progress)
100
101     candidates: List[Dict[str, Any]] = []
102     for i, ((s, e), xs) in enumerate(zip(intervals, x_shuttle)):
103         det = per_window[i]
104         det_fw = det.get("frame_width") or fw
105         det_fh = det.get("frame_height") or 0.0
106         # Normalise the (pixel) net coord to 0-1 on the play axis, matching the foot-
point
107         # normalisation fusion_features uses (x/width, y/height).
108         if axis == "x":
109             net_norm = net_px / det_fw if det_fw > 0 else 0.5
110         else:
111             net_norm = net_px / det_fh if det_fh > 0 else 0.5
112         sf, ef = win_frames[i]
113         shuttle_in = [p for p in points if sf <= p.frame <= ef]
114         person = ff.compute_person_features(det["frames"], shuttle_in, det_fw, det_fh,
115                                             court_polygon=None, net=(axis, net_norm))
116         candidates.append({
117             "start": float(s), "end": float(e),
118             "shuttle": {k: float(v) for k, v in zip(SHUTTLE_FEATURE_NAMES, xs)},
119             "person": {k: float(v) for k, v in person.items()},
120             "label": int(labels[i]),

```

```

121         })
122     return {
123         "name": spec["name"], "fps": fps, "frame_width": fw,
124         "candidates": candidates,
125         "gts": [[float(a), float(b)] for a, b in gts],
126     }
127
128
129     def run(manifest_path: str, out_dir: str, frame_skip: int = 3, iou: float = 0.5,
130            fresh: bool = False, smallest_first: bool = False) -> List[str]:
131         with open(manifest_path, encoding="utf-8") as f:
132             specs = json.load(f)
133         if smallest_first:
134             # Process small videos first for quick feedback (the slow full match lands
135             # the LOVO result is order-independent. Proxy size by the trajectory CSV (one
136             # row/frame).
137             specs = sorted(specs, key=lambda s: (os.path.getsize(s["trajectory"])
138                                                if os.path.isfile(s.get("trajectory", ""))
139                                                else 0))
140         os.makedirs(out_dir, exist_ok=True)
141         # Run-telemetry black box: a long GPU run can be OS-killed with no traceback, so
142         # keep a tiny
143         # rotating runlog of velocity + machine health for the post-mortem. Constructing it
144         # must never
145         # break extraction, so guard the build and fall back to no telemetry on any error.
146         telem: Optional[RunTelemetry]
147         try:
148             telem = RunTelemetry(os.path.join(out_dir, "run-telemetry.jsonl"),
149                                label="fusion_golden")
150         except Exception: # noqa: BLE001 - telemetry wiring must never break the run it
151             # observes
152             telem = None
153         written: List[str] = []
154         for vi, spec in enumerate(specs, 1):
155             out = os.path.join(out_dir, f"{spec['name']}_golden_features.json")
156             # Resumable: skip a video whose JSON already exists (a re-run picks up where a
157             # prior
158             # run left off instead of re-detecting). --fresh forces a redo.
159             if not fresh and os.path.isfile(out):
160                 print(f"[fusion_golden] ({vi}/{len(specs)}) {spec['name']}: already
161                 extracted "
162                     f"-> {out} (use --fresh to redo)", flush=True)
163                 written.append(out)
164                 continue
165             print(f"[fusion_golden] ({vi}/{len(specs)}) {spec['name']}: extracting...",
166                  flush=True)
167             t0 = time.monotonic()
168             if telem is not None:
169                 telem.event("video_start", spec["name"])
170             try:
171                 data = extract_video(spec, frame_skip=frame_skip, iou=iou, telemetry=telem)
172             except Exception as e: # noqa: BLE001 - record machine state at death, then re-
173                 raise
174             if telem is not None:
175                 telem.event("error", spec["name"], err=str(e))
176                 raise
177             n_pos = sum(c["label"] for c in data["candidates"])
178             with open(out, "w", encoding="utf-8") as f:
179                 json.dump(data, f, indent=2)
180             if telem is not None:
181                 telem.event("video_done", spec["name"],
182                           windows=len(data["candidates"]), positives=n_pos)
183             print(f"[fusion_golden] ({vi}/{len(specs)}) {spec['name']}: DONE in
184             {time.monotonic() - t0:.0f}s "

```

```

174         f"| {len(data['candidates'])} windows ({n_pos} positive) |
{len(data['gts'])} golden rallies "
175         f"-> {out}", flush=True)
176         written.append(out)
177         return written
178
179
180     def main() -> None:
181         ap = argparse.ArgumentParser(description="Extract golden-set shuttle+person features
(fusion P3 GPU step).")
182         ap.add_argument("--manifest", default="output/human_lovo_manifest.json")
183         ap.add_argument("--out-dir", default="output")
184         ap.add_argument("--frame-skip", type=int, default=3)
185         ap.add_argument("--iou", type=float, default=0.5)
186         ap.add_argument("--fresh", action="store_true", help="re-extract even if a JSON
already exists")
187         ap.add_argument("--smallest-first", action="store_true",
188                         help="process smaller videos first for quick feedback (LOVO is
order-independent)")
189         args = ap.parse_args()
190         run(args.manifest, args.out_dir, args.frame_skip, args.iou, fresh=args.fresh,
191             smallest_first=args.smallest_first)
192
193
194     if __name__ == "__main__":
195         main()
196

```

16. user (2026-06-14T10:10:33.403Z)

File does not exist. Note: your current working directory is C:\Users\avidu\Projects\badminton-highlight-indexer.

17. assistant (2026-06-14T10:10:36.893Z)

The quality_report.py isn't in the worktree. Let me find it and check the feature JSON shape and which features include `net\_crossings` and audio.

18. user (2026-06-14T10:10:37.463Z)

scratch\quality_report.py

19. user (2026-06-14T10:10:37.928Z)

scratch\quality_report.py

20. user (2026-06-14T10:10:39.475Z)

```

C:/Users/avidu/Projects/badminton-highlight-
indexer/output/adarsh_avi_singles_golden_features.json
C:/Users/avidu/Projects/badminton-highlight-indexer/output/gbaaddy_golden_features.json
C:/Users/avidu/Projects/badminton-highlight-indexer/output/kushagra_singles_golden_features.json
C:/Users/avidu/Projects/badminton-highlight-
indexer/output/largetest_doubles_golden_features.json
C:/Users/avidu/Projects/badminton-highlight-
indexer/output/mahadevpura_singles_golden_features.json
C:/Users/avidu/Projects/badminton-highlight-indexer/output/testlarge_short_golden_features.json
---
C:/Users/avidu/Projects/badminton-highlight-indexer/output/human_lovo_manifest.json

```

21. assistant (2026-06-14T10:10:55.193Z)

6 golden feature JSONs exist in the main repo's `output/`. The worktree won't have them (output/ is gitignored). Let me read quality_report.py for the reportlab pattern, inspect a golden JSON's

shape, and check the FEATURE_NAMES / audio / net_crossings sources.

22. user (2026-06-14T10:10:56.112Z)

```
1      """Generate a paper-style quality report PDF for the rally-detection system.
2
3      Self-contained: uses reportlab (Platypus) + matplotlib. Produces a polished
4      multi-page PDF at scratch/Rally_Detection_Quality_Report.pdf. All numbers are
5      verified facts pulled from the codebase (docs/QUALITY_ITERATIONS.md and the
6      eval_baselines/*.json leaderboard records); nothing here is invented.
7
8      Run: python scratch/quality_report.py
9      """
10
11     import os
12
13     import matplotlib
14
15     matplotlib.use("Agg")
16     import matplotlib.pyplot as plt
17     from reportlab.lib import colors
18     from reportlab.lib.enums import TA_CENTER, TA_JUSTIFY, TA_LEFT
19     from reportlab.lib.pagesizes import A4
20     from reportlab.lib.styles import ParagraphStyle, getSampleStyleSheet
21     from reportlab.lib.units import cm, mm
22     from reportlab.pdfbase import pdfmetrics
23     from reportlab.pdfbase.ttfonts import TTFont
24     from reportlab.platypus import (
25         BaseDocTemplate,
26         Frame,
27         Image,
28         NextPageTemplate,
29         PageBreak,
30         PageTemplate,
31         Paragraph,
32         Spacer,
33         Table,
34         TableStyle,
35     )
36
37     HERE = os.path.dirname(os.path.abspath(__file__))
38     PDF_PATH = os.path.join(HERE, "Rally_Detection_Quality_Report.pdf")
39     CHART_PATH = os.path.join(HERE, "_fusion_ablation_chart.png")
40
41     # -----
42     # Fonts: register a serif Unicode TTF (Times New Roman) so the report reads
43     # like a real journal paper, and glyphs like delta, >=, +/-, x still render.
44     # Falls back to Arial, then Helvetica, only if Times is missing.
45     # -----
46     FONT_REGULAR = "Helvetica"
47     FONT_BOLD = "Helvetica-Bold"
48     FONT_ITALIC = "Helvetica-Oblique"
49
50     # Times New Roman family (preferred: serif, academic-journal acceptable).
51     _times_reg_candidates = [
52         r"C:/Windows/Fonts/times.ttf",
53         os.path.join(HERE, "times.ttf"),
54     ]
55     _times_bd_candidates = [
56         r"C:/Windows/Fonts/timesbd.ttf",
57         os.path.join(HERE, "timesbd.ttf"),
58     ]
59     _times_it_candidates = [
60         r"C:/Windows/Fonts/timesi.ttf",
61         os.path.join(HERE, "timesi.ttf"),
```

```

62     ]
63     _times_bi_candidates = [
64         r"C:/Windows/Fonts/timesbi.ttf",
65         os.path.join(HERE, "timesbi.ttf"),
66     ]
67
68     # Arial fallback (only used if Times is unavailable).
69     _arial_candidates = [
70         os.path.join(HERE, "arial.ttf"),
71         r"C:/Windows/Fonts/arial.ttf",
72     ]
73     _arialbd_candidates = [
74         r"C:/Windows/Fonts/arialbd.ttf",
75         os.path.join(HERE, "arialbd.ttf"),
76     ]
77     _ariali_candidates = [
78         r"C:/Windows/Fonts/ariali.ttf",
79         os.path.join(HERE, "ariali.ttf"),
80     ]
81
82
83     def _first_existing(paths):
84         for p in paths:
85             if os.path.exists(p):
86                 return p
87         return None
88
89
90     def _register_family(reg_paths, bd_paths, it_paths, bi_paths=None):
91         """Register a 4-variant TTF family under the logical name "UI".
92
93         Returns True on success. The family is mapped so inline <b>/<i> tags
94         resolve to the bold/italic variants.
95         """
96         reg = _first_existing(reg_paths)
97         if not reg:
98             return False
99         pdfmetrics.registerFont(TTFont("UI", reg))
100         bd = _first_existing(bd_paths)
101         it = _first_existing(it_paths)
102         bi = _first_existing(bi_paths or [])
103         pdfmetrics.registerFont(TTFont("UI-Bold", bd if bd else reg))
104         pdfmetrics.registerFont(TTFont("UI-Italic", it if it else reg))
105         pdfmetrics.registerFont(TTFont("UI-BoldItalic", bi if bi else (bd if bd else reg)))
106         pdfmetrics.registerFontFamily(
107             "UI",
108             normal="UI",
109             bold="UI-Bold",
110             italic="UI-Italic",
111             boldItalic="UI-BoldItalic",
112         )
113         return True
114
115
116     # Prefer Times New Roman (serif); fall back to Arial only if Times is missing.
117     try:
118         if _register_family(
119             _times_reg_candidates,
120             _times_bd_candidates,
121             _times_it_candidates,
122             _times_bi_candidates,
123         ) or _register_family(
124             _arial_candidates,
125             _arialbd_candidates,
126             _ariali_candidates,

```

```

127         ):
128             FONT_REGULAR = "UI"
129             FONT_BOLD = "UI-Bold"
130             FONT_ITALIC = "UI-Italic"
131     except Exception as exc: # pragma: no cover - defensive
132         print("Font registration fell back to Helvetica:", exc)
133         FONT_REGULAR, FONT_BOLD, FONT_ITALIC = (
134             "Helvetica",
135             "Helvetica-Bold",
136             "Helvetica-Oblique",
137         )
138
139     UNICODE_OK = FONT_REGULAR == "UI"
140     DELTA = "?" if UNICODE_OK else "Delta" # Greek capital delta
141     GE = "?" if UNICODE_OK else ">="
142     PM = "±" if UNICODE_OK else "+/-"
143     TIMES = "x" if UNICODE_OK else "x"
144     ARROW = "?" if UNICODE_OK else "->"
145
146     # Palette ? minimal & muted. ONE accent only (dark navy) for section
147     # headings and thin rules; everything else is black or restrained grey.
148     # No crimson, no conditional green/red number coloring.
149     NAVY = colors.HexColor("#1F3A5F") # the single accent (headings + thin rules)
150     STEEL = NAVY # subheadings reuse the same navy (no second accent)
151     BLACK = colors.HexColor("#000000")
152     DARKGREY = colors.HexColor("#333333")
153     MIDGREY = colors.HexColor("#666666")
154     LIGHT = colors.HexColor("#F0F0F0") # very light grey for restrained fills
155     ROW_ALT = colors.HexColor("#F7F7F7") # extremely faint alternating shade
156     HAIRLINE = colors.HexColor("#BFBFBF") # thin neutral hairlines
157
158     # -----
159     # Styles
160     # -----
161     styles = getSampleStyleSheet()
162
163
164     def style(name, **kw):
165         base = ParagraphStyle(name, parent=styles["Normal"], **kw)
166         return base
167
168
169     BODY = style(
170         "Body",
171         fontName=FONT_REGULAR,
172         fontSize=10,
173         leading=13.5,
174         alignment=TA_JUSTIFY,
175         spaceAfter=6,
176         textColor=BLACK,
177     )
178     BODY_TIGHT = style(
179         "BodyTight",
180         fontName=FONT_REGULAR,
181         fontSize=10,
182         leading=13,
183         alignment=TA_JUSTIFY,
184         spaceAfter=3,
185         textColor=BLACK,
186     )
187     H1 = style(
188         "H1",
189         fontName=FONT_BOLD,
190         fontSize=13,
191         leading=17,

```

```

192         textColor=NAVY,
193         spaceBefore=14,
194         spaceAfter=6,
195     )
196     H2 = style(
197         "H2",
198         fontName=FONT_BOLD,
199         fontSize=11,
200         leading=15,
201         textColor=NAVY,
202         spaceBefore=8,
203         spaceAfter=4,
204     )
205     ABSTRACT = style(
206         "Abstract",
207         fontName=FONT_ITALIC,
208         fontSize=10,
209         leading=13.5,
210         alignment=TA_JUSTIFY,
211         textColor=BLACK,
212         leftIndent=10,
213         rightIndent=10,
214         spaceAfter=6,
215     )
216     CAPTION = style(
217         "Caption",
218         fontName=FONT_ITALIC,
219         fontSize=8.5,
220         leading=11,
221         alignment=TA_CENTER,
222         textColor=DARKGREY,
223         spaceBefore=2,
224         spaceAfter=8,
225     )
226     TITLE = style(
227         "Title2",
228         fontName=FONT_BOLD,
229         fontSize=20,
230         leading=25,
231         alignment=TA_CENTER,
232         textColor=NAVY,
233         spaceAfter=4,
234     )
235     SUBTITLE = style(
236         "Subtitle",
237         fontName=FONT_ITALIC,
238         fontSize=11.5,
239         leading=15,
240         alignment=TA_CENTER,
241         textColor=BLACK,
242         spaceAfter=2,
243     )
244     AUTHORLINE = style(
245         "Author",
246         fontName=FONT_REGULAR,
247         fontSize=9.5,
248         leading=13,
249         alignment=TA_CENTER,
250         textColor=DARKGREY,
251         spaceAfter=14,
252     )
253     TBL_CELL = style("TblCell", fontName=FONT_REGULAR, fontSize=8.6, leading=11,
textColor=BLACK)
254     TBL_HEAD = style(
255         "TblHead",

```

```

256     fontName=FONT_BOLD,
257     fontSize=8.6,
258     leading=11,
259     textColor=BLACK,
260 )
261 KEYBOX = style(
262     "KeyBox",
263     fontName=FONT_REGULAR,
264     fontSize=9.5,
265     leading=13,
266     textColor=BLACK,
267     leftIndent=6,
268     rightIndent=6,
269     spaceAfter=2,
270 )
271
272
273 def P(text, st=BODY):
274     return Paragraph(text, st)
275
276
277 # -----
278 # Matplotlib chart: fusion ablation (uses full-precision JSON floats)
279 # -----
280 def build_chart():
281     # Full-precision values from eval_baselines/experiment_leaderboard.json
282     labels = ["shuttle-only", "+person (P3)", "+person+audio (P4)"]
283     f1 = [0.3068611057861412, 0.2863398846047612, 0.3655826552797436]
284     # 95% bootstrap CI half-widths are only defined for the incremental
285     # deltas, not for absolute F1; we annotate deltas as text instead.
286     # Muted, conservative palette: navy + neutral grey, no saturated colors.
287     bar_colors = ["#1F3A5F", "#9AA7B4", "#1F3A5F"]
288
289     fig, ax = plt.subplots(figsize=(6.6, 3.5), dpi=150)
290     x = range(len(labels))
291     bars = ax.bar(x, f1, color=bar_colors, width=0.58, edgecolor="#222222",
292 linewidth=0.6)
293
294     for i, (b, v) in enumerate(zip(bars, f1)):
295         ax.text(
296             b.get_x() + b.get_width() / 2,
297             v + 0.006,
298             f"{v:.3f}",
299             ha="center",
300             va="bottom",
301             fontsize=10,
302             fontweight="bold",
303             color="#000000",
304         )
305
306     # delta annotations between consecutive bars (neutral black text)
307     ax.annotate(
308         "?F1 = -0.021\n(CI spans 0)",
309         xy=(0.5, 0.33),
310         ha="center",
311         fontsize=8,
312         color="#000000",
313     )
314     ax.annotate(
315         "?F1 = +0.079\n(CI spans 0)",
316         xy=(1.5, 0.39),
317         ha="center",
318         fontsize=8,
319         color="#000000",
320     )

```

```

320
321 # reference line: heuristic LOVO floor (thin dashed grey)
322 ax.axhline(0.480, color="#888888", linestyle="--", linewidth=1.0, alpha=0.9)
323 ax.text(
324     2.42, 0.487, "heuristic LOVO floor 0.480", ha="right", va="bottom",
325     fontsize=7.5, color="#333333",
326 )
327
328 ax.set_xticks(list(x))
329 ax.set_xticklabels(labels, fontsize=9, color="#000000")
330 ax.tick_params(axis="y", colors="#000000")
331 ax.set_ylabel("LOVO F1 (IoU >= 0.5, n=6)", fontsize=9.5, color="#000000")
332 ax.set_ylim(0, 0.55)
333 ax.set_title(
334     "Fusion ablation: incremental cue contribution (honest LOVO)",
335     fontsize=10.5,
336     fontweight="bold",
337     color="#000000",
338 )
339 ax.spines["top"].set_visible(False)
340 ax.spines["right"].set_visible(False)
341 ax.yaxis.grid(True, linestyle=":", alpha=0.4)
342 ax.set_axisbelow(True)
343 fig.tight_layout()
344 fig.savefig(CHART_PATH, bbox_inches="tight", facecolor="white", dpi=200)
345 plt.close(fig)
346
347
348 # -----
349 # Table helper
350 # -----
351 def make_table(data, col_widths, header_rows=1, align_centers=None, font_size=8.3):
352     """data: list of rows; row[i] is either a str or a Paragraph."""
353     wrapped = []
354     for r_idx, row in enumerate(data):
355         new_row = []
356         for c_idx, cell in enumerate(row):
357             if isinstance(cell, Paragraph):
358                 new_row.append(cell)
359             else:
360                 st = TBL_HEAD if r_idx < header_rows else TBL_CELL
361                 new_row.append(Paragraph(str(cell), st))
362         wrapped.append(new_row)
363
364     t = Table(wrapped, colWidths=col_widths, repeatRows=header_rows)
365     last_row = len(wrapped) - 1
366     # Booktabs style: thin top rule, bold-black header on a very light grey
367     # fill, a thin rule under the header, faint hairlines between body rows,
368     # and a thin bottom rule. No heavy colored header, no vertical grid.
369     cmds = [
370         ("BACKGROUND", (0, 0), (-1, header_rows - 1), LIGHT),
371         ("TEXTCOLOR", (0, 0), (-1, header_rows - 1), BLACK),
372         ("VALIGN", (0, 0), (-1, -1), "MIDDLE"),
373         ("TOPPADDING", (0, 0), (-1, -1), 4),
374         ("BOTTOMPADDING", (0, 0), (-1, -1), 4),
375         ("LEFTPADDING", (0, 0), (-1, -1), 5),
376         ("RIGHTPADDING", (0, 0), (-1, -1), 5),
377         # Top rule (above the header).
378         ("LINEABOVE", (0, 0), (-1, 0), 1.0, BLACK),
379         # Thin rule under the header.
380         ("LINEBELOW", (0, header_rows - 1), (-1, header_rows - 1), 0.8, BLACK),
381         # Faint horizontal hairlines between body rows only.
382         ("LINEBELOW", (0, header_rows), (-1, last_row - 1), 0.3, HAIRLINE),
383         # Thin bottom rule (below the last body row).
384         ("LINEBELOW", (0, last_row), (-1, last_row), 1.0, BLACK),

```

```

385         # Extremely faint alternating shade (off-white only).
386         ("ROWBACKGROUNDS", (0, header_rows), (-1, -1), [colors.white, ROW_ALT]),
387     ]
388     if align_centers:
389         for c in align_centers:
390             cmds.append(("ALIGN", (c, 0), (c, -1), "CENTER"))
391     t.setStyle(TableStyle(cmds))
392     return t
393
394
395     # -----
396     # Page decoration: header rule + footer page number
397     # -----
398     def _footer(canvas, doc):
399         w, h = A4
400         canvas.saveState()
401         canvas.setStrokeColor(HAIRLINE)
402         canvas.setLineWidth(0.5)
403         canvas.line(2 * cm, 1.4 * cm, w - 2 * cm, 1.4 * cm)
404         canvas.setFont(FONT_REGULAR, 8)
405         canvas.setFillColor(MIDGREY)
406         canvas.drawCentredString(w / 2.0, 1.0 * cm, f"{doc.page}")
407         canvas.drawString(2 * cm, 1.0 * cm, "badminton-highlight-indexer")
408         canvas.drawRightString(w - 2 * cm, 1.0 * cm, "Internal · honest measured results
only")
409         canvas.restoreState()
410
411
412     def on_page(canvas, doc):
413         canvas.saveState()
414         w, h = A4
415         # Thin, grey running header (small) on pages 2+ ? academic style, no rule
416         # in a strong color. A faint hairline separates header text from the body.
417         canvas.setStrokeColor(HAIRLINE)
418         canvas.setLineWidth(0.5)
419         canvas.line(2 * cm, h - 1.4 * cm, w - 2 * cm, h - 1.4 * cm)
420         canvas.setFont(FONT_ITALIC, 7.5)
421         canvas.setFillColor(MIDGREY)
422         canvas.drawString(2 * cm, h - 1.25 * cm, "Local, Cheap, AI Rally Detection for
Racket Sports")
423         canvas.drawRightString(
424             w - 2 * cm, h - 1.25 * cm, "Project Quality Report ? toward a paper"
425         )
426         canvas.restoreState()
427         _footer(canvas, doc)
428
429
430     def on_first_page(canvas, doc):
431         # Clean academic title page: no colored banner block, no red strip.
432         # Only the bottom footer/page number (the title block in the story carries
433         # the heading, centered, like a real paper's first page).
434         _footer(canvas, doc)
435
436
437     # -----
438     # Build the story
439     # -----
440     def build_story():
441         story = []
442
443         # --- Title block (sits below the banner) ---
444         story.append(Spacer(1, 0.2 * cm))
445         story.append(Paragraph("Local, Cheap, AI Rally Detection<br/>for Racket Sports",
TITLE))
446         story.append(Paragraph("Project report &mdash; toward a paper", SUBTITLE))

```

```

447     story.append(
448         Paragraph(
449             "An honest empirical study of a shuttle-trajectory rally segmenter, "
450             "its disciplined small-N promotion gate, and a fusion ablation",
451             AUTHORLINE,
452         )
453     )
454
455     # --- Abstract ---
456     story.append(Paragraph("Abstract", H1))
457     abstract = (
458         "We present a fully local, low-cost rally-detection system for racket sports "
459         "(badminton first) that segments a match video into rally <i>[start, end]</i> "
460         "intervals. The system is built on a frozen shuttle-trajectory substrate (a WASB
461         CNN "
462         "that emits per-frame shuttle position), from which a velocity signal seeds "
463         "recall-oriented candidate windows that a tiny pure-numpy logistic-regression
464         scorer "
465         "grades. Every additional cue enters as a confidence-weighted feature column
466         that the "
467         "scorer learns to weight &mdash; never as a hand-coded branch &mdash; and ships
468         "
469         "<b>default-OFF</b>, promoted to the served default only on a measured held-out
470         lift. "
471         "We evaluate against a small owned golden corpus of <b>n=6</b> labelled matches,
472         treating "
473         "the labelling vendor (Roora) as an interchangeable <i>oracle</i>: the contract
474         is with the "
475         "golden labels as trustable ground truth, not with their provenance, so
476         evaluation rigor "
477         "is decoupled from the vendor and the corpus scales by adding oracles. Using
478         leave-one- "
479         "video-out cross-validation with bootstrap 95% confidence intervals and a paired
480         sign "
481         "test, our first fused cue &mdash; a net-crossing gate (Gate&nbsp;A) &mdash; is
482         honestly "
483         f"<b>promoted</b> ( {DELTA}F1 = +0.074, 6/6 matches, sign-test p = 0.031, CI
484         [+0.042, +0.104]), "
485         "cutting over-segmentation from 1.68 to 1.01. Two further cues (a person cue,
486         then audio) "
487         "do <i>not</i> clear the bar at n=6 (both CIs span 0) and stay default-OFF. We
488         report these "
489         "negative results deliberately as the honest ablation backbone, frame the
490         architecture "
491         "against standard prior art (temporal action localization / segmentation + late
492         fusion), "
493         "and isolate the genuinely novel piece: a report-first reconciler-as-linter
494         paired with a "
495         "disciplined small-N promotion gate."
496     )
497     story.append(Paragraph(abstract, ABSTRACT))
498
499     # Key-result strip
500     kr = [
501         [P(f"<b>Gemini wrapper (cloud ref.)</b>", TBL_CELL), P("clean 0.93 / multi-court
502         0.66", TBL_CELL)],
503         [P("<b>Gen-0 owned head (LOVO)</b>", TBL_CELL), P("clean 0.744 / multi-court
504         0.561", TBL_CELL)],
505         [P("<b>Gate-A (PROMOTED)</b>", TBL_CELL), P(f"{DELTA}F1 +0.074, p=0.031, CI
506         [+0.042,+0.104]", TBL_CELL)],
507         [P("<b>Golden corpus</b>", TBL_CELL), P("n=6 matches (262 rallies), Proprietary,
508         minors excluded", TBL_CELL)],
509     ]
510     kt = Table(kr, colWidths=[5.2 * cm, 11.3 * cm])
511     kt.setStyle(

```

```

491         TableStyle(
492             [
493                 # Restrained: thin top/bottom rules + faint label-column tint
494                 # and faint row hairlines; no boxed grid, no bright fills.
495                 ("BACKGROUND", (0, 0), (0, -1), LIGHT),
496                 ("LINEABOVE", (0, 0), (-1, 0), 1.0, BLACK),
497                 ("LINEBELOW", (0, -1), (-1, -1), 1.0, BLACK),
498                 ("LINEBELOW", (0, 0), (-1, -2), 0.3, HAIRLINE),
499                 ("VALIGN", (0, 0), (-1, -1), "MIDDLE"),
500                 ("TOPPADDING", (0, 0), (-1, -1), 3),
501                 ("BOTTOMPADDING", (0, 0), (-1, -1), 3),
502                 ("LEFTPADDING", (0, 0), (-1, -1), 6),
503             ]
504         )
505     )
506     story.append(Spacer(1, 0.2 * cm))
507     story.append(kt)
508     story.append(Paragraph("Headline results at a glance (IoU 0.5 vs human golden
509 labels).", CAPTION))
510     # ===== 1. Introduction =====
511     story.append(Paragraph("1&nbsp;&nbsp;&nbsp;Introduction & Problem", H1))
512     story.append(
513         P(
514             "A rally is the unit of play in racket sports: the span from serve contact
515             to the point "
516             "ending. Indexing a long match video into its rallies is the substrate for
517             everything a "
518             "coach or player wants &mdash; highlight reels, &ldquo;show me the longest
519             rally,&rdquo; "
520             "per-rally statistics. Cloud vision-language models can do this well on
521             clean footage, but "
522             "they are expensive per video, non-deterministic, and (critically) <b>cannot
523             be a training "
524             "data source</b> for an owned model under their terms. Our constraint is the
525             opposite end of "
526             "the design space: a system that runs <b>locally, cheaply, and
527             reproducibly</b> on a single "
528             "consumer GPU, with paid LLMs allowed only as an optional inference fallback
529             &mdash; never "
530             "in the training loop."
531         )
532     )
533     story.append(
534         P(
535             "Two failure modes dominate and motivate the entire method. <b>(1) Held-
536             shuttle false "
537             "positives:</b> a player holding or flicking the shuttle between points
538             produces "
539             "player-motion that the old velocity heuristic counted as a rally &mdash;
540             the owner-observed "
541             "precision bug. <b>(2) Over-segmentation:</b> a single true rally gets
542             shredded into several "
543             "short fragments. Crucially, the standard temporal-IoU metric is nearly
544             <i>blind</i> to "
545             "over-segmentation, so a method can look fine on IoU while destroying rally
546             structure. Both "
547             "failure modes are about the boundary between play and non-play, which is
548             exactly what the "
549             "fusion gates in this report target."
550         )
551     )
552     # ===== 2. System & Method =====
553     story.append(Paragraph("2&nbsp;&nbsp;&nbsp;System & Method", H1))

```

```

540     story.append(Paragraph("2.1&nbsp;&nbsp;&nbsp;A two-layer mental model", H2))
541     story.append(
542         P(
543             "<b>Layer 1 &mdash; Detection (&ldquo;where is the shuttle this
frame?&rdquo;).</b> A frozen "
544             "WASB CNN emits a per-frame (x, y, visible) shuttle track. The shuttle is
chosen as the "
545             "camera-<i>invariant</i> primary signal because &ldquo;shuttle in
flight&rdquo; is a near-"
546             "synonym for &ldquo;a rally is happening,&rdquo; unlike player motion which
fooled the old "
547             "heuristic on broadcast pan cameras. <b>Layer 2 &mdash; Segmentation
(&ldquo;when is a "
548             "rally?&rdquo;).</b> From the per-frame track we compute a shuttle
<i>velocity</i> signal, "
549             "which seeds recall-oriented candidate windows; a learned scorer then grades
each window "
550             f"into final [start, end] rallies measured by temporal IoU. The chain is:
frames {ARROW} "
551             f"WASB frozen trajectory {ARROW} velocity {ARROW} shuttle-seeded candidate
windows {ARROW} "
552             "learned scorer weights per-window feature columns {0} graded rallies. The
original v0 "
553             "segmentation was three hand-tuned knobs (velocity threshold, merge gap,
minimum window "
554             "duration); the learned scorer replaces that hand-tuning.".format(ARROW)
555         )
556     )
557     story.append(
558         P(
559             "A key honesty property follows from the data: golden labels are rally
<i>times</i> "
560             "(when), not (x, y) <i>coordinates</i> (where). They therefore cannot
retrain the frozen "
561             "WASB detector &mdash; they can only grade and tune the segmentation layer.
The discipline "
562             "is &ldquo;calibrate on some rallies, measure on held-out ones.&rdquo;"
563         )
564     )
565     story.append(Paragraph("2.2&nbsp;&nbsp;&nbsp;Signals as feature columns, not branches",
H2))
566     story.append(
567         P(
568             "Every producer (detector / analyzer / reconciler) emits a <b>signal =
(value, "
569             "confidence)</b> persisted to a namespaced column <font face=\"{0}\">"
570             "&lt;producer&gt;__&lt;key&gt;</font>. The scoring model consumes those
columns and "
571             "<b>learns to weight</b> them; there are <b>no hand-coded if/else
branches</b> in the "
572             "decision path. Producers propose; the scorer arbitrates; the experiment
harness and the "
573             "golden set decide what is promoted. The scorer itself is deliberately a
tiny pure-numpy "
574             "L2 logistic regression (ADR-004), <i>not</i> a neural net, so it survives
small-N. A cue "
575             "that does not earn its keep simply stays at weight ~0 &mdash; it costs
nothing because it "
576             "is an unused column, not a live branch.".format(FONT_ITALIC)
577         )
578     )
579     story.append(
580         P(
581             "Fusion has two realizations. The <b>decision-level gate</b> (label-free,
ships first) "

```

```

582         f"composes Gate&nbsp;A (net-crossings {GE} 2 &mdash; the structural held-
shuttle / service-"
583         "feed fix) and Gate&nbsp;B (a person cue). The <b>feature-level</b> path
(the primary, "
584         "learned route) lets cues emit a small set of
scale/translation-<i>invariant</i> per-window "
585         "features (counts, ratios, zone-membership, two-sidedness &mdash; never raw
coordinates, "
586         "which would memorize this court and camera) that join the logistic feature
vector. The "
587         "shuttle stays primary; person is a gate-plus-feature, never the sole
trigger, and single-"
588         "cue segmenters remain registered fallbacks."
589     )
590 )
591
592 # ===== 3. Evaluation Methodology (the heart) =====
593 story.append(Paragraph("3&nbsp;&nbsp;Evaluation Methodology", H1))
594 story.append(
595     P(
596         "This section is the heart of the work: the system's credibility rests on
<i>how</i> it is "
597         "measured, not on any single number."
598     )
599 )
600 story.append(Paragraph("3.1&nbsp;&nbsp;The owned golden corpus and the oracle
contract", H2))
601 story.append(
602     P(
603         "Evaluation uses a small <b>owned golden corpus of n=6 labelled matches</b>
"
604         "(<font face=\"{0}\">adarsh_avi_singles, kushagra_singles,
largetest_doubles, gbaaddy, "
605         "testlarge_short, mahadevpura_singles</font>; 262 rallies total), grown by
the owner, "
606         "classified Proprietary, with minors excluded entirely and no
biometric/identity labelling. "
607         "The labelling vendor, <b>Roora</b>, is treated as an interchangeable
<b>oracle label "
608         "generator</b>: the system's contract is with the golden <i>labels</i>
&mdash; assumed-"
609         "trustable ground truth, validated by automated checks plus a ~20% human
spot-check at a "
610         f"{PM}0.3&nbsp;s boundary tolerance &mdash; <b>not</b> with their
provenance. Evaluation "
611         "rigor (interval matching, LOVO, the ship-gate) is therefore
<i>decoupled</i> from the "
612         "vendor, and the corpus scales by <b>adding or swapping oracles</b>; any
oracle's labels "
613         "import directly in the same schema. This is the project's moat: a consented
dataset plus an "
614         "honest eval harness, not the weights.".format(FONT_ITALIC)
615     )
616 )
617 story.append(Paragraph("3.2&nbsp;&nbsp;Leave-one-video-out, with honest statistics",
H2))
618 story.append(
619     P(
620         "Learned variants are scored <b>leave-one-video-out (LOVO)</b>, grouped by
match: train on "
621         "the <i>other</i> videos' candidates, predict the held-out video, score
against its labels. "
622         "We never hold out a window from the <i>same</i> video &mdash; windows
within a video are "
623         "correlated and would flatter the score (the field name is leave-one-

```

```

group/subject-out CV). "
624         "Every tunable knob &mdash; the operating-point threshold, the calibrator
&mdash; is fit on "
625         "the <i>training fold only</i> and never looks at the held-out video.
Because a bare LOVO "
626         f"mean at n {TIMES if False else GE} 6 is not trustworthy on its own, we
attach <b>honest "
627         "statistics</b>: a bootstrap 95% confidence interval per metric, a paired "
628         f"{DELTA}F1 CI, and an exact paired <b>sign test</b>. The promotion rule is
encoded directly: "
629         f"a lift is real only when the {DELTA}F1 CI clears 0 <i>and</i> the sign
test agrees. This is "
630         "necessary because the sign test needs sample size &mdash; at n=6 a ~+0.08
lift cannot reach "
631         "p < 0.05; roughly n=10 with 9 wins gives p &asymp; 0.011."
632     )
633 )
634 story.append(Paragraph("3.3&nbsp;&nbsp; Persist once, re-score forever", H2))
635 story.append(
636     P(
637         "The expensive, non-deterministic per-frame <b>detection</b> (shuttle WASB
trajectory, person "
638         "tracks, audio hits) runs <b>once</b> per video on the GPU/WSL and is
persisted as a small "
639         "replayable feature substrate (one JSON per video of resolution-invariant
shuttle + person "
640         "features per candidate window). The cheap, deterministic <b>decision</b>
(windowing, gates, "
641         "the logistic scorer) then re-scores any number of variants <b>offline</b>,
with no GPU and "
642         "zero production risk. A <i>Variant</i> is a declarative re-scoring recipe,
not a code branch; "
643         "given persisted features it produces the same intervals deterministically.
This is what makes "
644         "every experiment a zero-risk offline re-score &mdash; the reproducibility
thesis of the whole "
645         "harness."
646     )
647 )
648
649 story.append(PageBreak())
650
651 # ===== 4. Results =====
652 story.append(Paragraph("4&nbsp;&nbsp; Results", H1))
653 story.append(Paragraph("4.1&nbsp;&nbsp; Baselines that drive strategy", H2))
654 story.append(
655     P(
656         "Table&nbsp;1 anchors strategy. Two reference footages are reported:
<b>Mahadevpura</b> "
657         "(clean-ish, single court) and <b>Kushagra</b> (multi-court, the hard case,
contrast "
658         f"1.4{TIMES}). The reading is blunt: clean footage is commoditized (a Gemini
serve reaches "
659         "0.93), but multi-court drops the cloud wrapper to <b>0.66</b> &mdash; and
that 0.66 is the "
660         "owned model's ship target. The owned Gen-0 head already reaches 0.561
multi-court at n=6."
661     )
662 )
663 tbl1 = [
664     ["Method", "Mahadevpura (clean)", "Kushagra (multi-court)"],
665     ["Heuristic (velocity windowing)", "0.657", "0.157"],
666     ["Gen-0 owned head (LOVO, n=6)", "0.744", "0.561"],
667     ["Two-stage WASB + Gemini refine", "0.83", "&mdash; (not measured)"],
668     ["Gemini wrapper (gemini-flash-latest)", "0.93", "0.66"],

```

```

669     ]
670     story.append(make_table(tbl1, [7.2 * cm, 4.65 * cm, 4.65 * cm], align_centers=[1,
2]))
671     story.append(
672         Paragraph(
673             "Table 1. Rally-detection F1 at IoU 0.5 vs human golden labels. "
674             "Clean footage is commoditized; multi-court 0.66 is the owned-model ship
target.",
675             CAPTION,
676         )
677     )
678     story.append(
679         P(
680             "Per-video detail on the cloud reference (gemini-flash-latest): on
Mahadevpura, F1 0.93, "
681             "precision 1.00, recall 0.87, mean IoU 0.80, boundary error 0.7&nbsp;s (n_gt
31, n_pred 27; "
682             "3 of 4 false negatives were detected by Gemini but rejected by a 3.0&nbsp;s
minimum-segment "
683             "filter on 2.5&nbsp;s rallies). On the harder Kushagra, F1 0.66, precision
0.69, recall 0.62, "
684             "mean IoU 0.73, boundary error 1.2&nbsp;s (n_gt 32, n_pred 29).".
685         )
686     )
687
688     story.append(Paragraph("4.2&nbsp;&nbsp; Gate-A: the first honestly-promoted cue",
H2))
689     story.append(
690         P(
691             f"Gate&nbsp;A (net-crossings {GE} 2) is the first cue promoted through the
harness, because it "
692             "is data-backed: real rallies on multi-court matches cross the net 2 or more
times <b>97% of "
693             "the time</b> (median 5), whereas a single service feed crosses about once.
The measured "
694             "result on the recall-oriented proposal candidate set is unambiguous
(Table&nbsp;2).".
695         )
696     )
697     tbl2 = [
698         ["Metric", "Result"],
699         [f"{DELTA}F1 (Gate-A vs control)", "<b>+0.074</b>"],
700         ["Golden matches won", "6 / 6"],
701         ["Paired sign-test p", "<b>0.031</b>"],
702         [f"95% CI on {DELTA}F1", "[+0.042, +0.104] (clears 0)"],
703         ["Over-segmentation (segment-count ratio)", f"1.68 {ARROW} 1.01"],
704         ["Recommended config", "<font face=\"%s\">indexing.fusion.min_crossings =
2</font>" % FONT_ITALIC],
705     ]
706     story.append(make_table(tbl2, [8.3 * cm, 8.2 * cm], align_centers=[1]))
707     story.append(
708         Paragraph(
709             "Table 2. Gate-A promotion statistics (2026-06-14). The CI clears 0 and the
sign test agrees "
710             "? the only fusion cue to date that does.",
711             CAPTION,
712         )
713     )
714     story.append(
715         P(
716             "Real-footage corroboration (no golden labels, so this is a divergence
check, not an F1): an "
717             "offline re-score of GX010125_1080p's cached trajectory took 45 candidates
from CONTROL's 45 "
718             "rallies down to Gate-A's 27, dropping 18 low-crossing windows

```

```

(~1.7&nbsp;min) of likely "
719         "held-shuttle / feed false positives. Caveat: this offline win is on the
recall-oriented "
720         "proposal set; flipping the live served default remains owner-gated, since
the production "
721         "fusion segmenter is still mock-tested rather than verified on real video."
722     )
723 )
724
725     story.append(Paragraph("4.3&nbsp;&nbsp;The learned-variant fix (PR #130)", H2))
726     story.append(
727         P(
728             "An early end-to-end A/B of a window-level 5-feature logistic variant
<i>lost</i> to control "
729             f"({DELTA}F1 = &minus;0.081, 2W/4L) and zeroed the testlarge video &mdash;
but this was a "
730             "fixed-0.5-cutoff artifact, not a real defect of the stronger per-frame
prototype. PR&nbsp;#130 "
731             "added an additive, default-OFF threshold-tuning step (operating point
chosen on the train "
732             "fold) plus probability calibration (Platt / isotonic). Validated on the n=6
golden set, mean "
733             "F1 rose <b>0.307 &rarr; 0.423 (+0.116)</b>, testlarge recovered from 0.000
to 0.300, and the "
734             "learned variant now beats CONTROL (0.388). The lesson &mdash; calibrate and
tune the "
735             "operating point <i>inside</i> the LOVO loop &mdash; is now baked into the
harness."
736         )
737     )
738
739     story.append(Paragraph("4.4&nbsp;&nbsp;Fusion ablation: person (P3) and audio (P4)",
H2))
740     story.append(
741         P(
742             "Two further cues were measured incrementally on the same n=6 corpus (honest
LOVO, IoU 0.5). "
743             "Neither clears the promotion bar &mdash; both CIs span 0 and both sign
tests are non-"
744             "significant &mdash; so both stay <b>default-OFF</b>. We report them in full
because negative "
745             "and uncertain results are the honest ablation backbone (Table&nbsp;3,
Figure&nbsp;1). Values "
746             "below use the full-precision leaderboard floats; the doc rounds them."
747         )
748     )
749     tbl3 = [
750         ["Step", "F1", f"({DELTA}F1 vs prior", "95% CI", "Sign test", "Clears 0?"],
751         ["shuttle-only", "0.3069", "&mdash;", "&mdash;", "&mdash;", "&mdash;"],
752         ["+person (P3)", "0.2863", "&minus;0.0205 (&minus;6.7%)", "[&minus;0.165,
+0.162]", "2W/4L, p=0.6875", "No"],
753         ["+person+audio (P4)", "0.3656", "+0.0792 (+27.7%)", "[&minus;0.054, +0.203]",
"4W/2L, p=0.6875", "No"],
754     ]
755     story.append(
756         make_table(
757             tbl3,
758             [3.0 * cm, 1.7 * cm, 3.2 * cm, 3.0 * cm, 2.7 * cm, 1.9 * cm],
759             align_centers=[1, 2, 3, 4, 5],
760             font_size=7.6,
761         )
762     )
763     story.append(
764         Paragraph(
765             "Table 3. Fusion ablation (n=6, honest LOVO, IoU 0.5; full-precision floats

```

```

from "
766         "experiment_leaderboard.json). Both records: IoU 0.5, label_set_hash
50d98ffbc370, n=6.",
767         CAPTION,
768     )
769 )
770     story.append(
771         P(
772             "Person adds noise: without court polygons, the players-in-court count
includes spectators, "
773             "so the cue is a wash (per-video: testlarge +0.40, gbaaddy &minus;0.22).
Audio is the more "
774             "promising lead &mdash; it wins 4/6 (largetest +0.27, mahadevpura +0.27) and
lifts F1 to "
775             "0.366, above the 0.307 shuttle-only baseline &mdash; but n=6 simply cannot
confirm it. The "
776             "discipline holds: promote only on a lift whose CI clears 0."
777         )
778     )
779     # Figure 1
780     if os.path.exists(CHART_PATH):
781         img = Image(CHART_PATH, width=15.2 * cm, height=8.06 * cm)
782         story.append(Spacer(1, 0.1 * cm))
783         story.append(img)
784         story.append(
785             Paragraph(
786                 "Figure 1. Incremental fusion ablation on the n=6 golden corpus. Neither
cue's "
787                 "?F1 confidence interval clears zero; both remain default-OFF. Dashed
line: the "
788                 "heuristic LOVO floor (0.480) the learned scorer must beat.",
789                 CAPTION,
790             )
791         )
792
793     # ===== 5. Related Work =====
794     story.append(Paragraph("5&nbsp;&nbsp; Related Work (cite, don't claim)", H1))
795     story.append(
796         P(
797             "A literature review (multi-agent web search, every citation adversarially
verified &mdash; "
798             "zero fabricated papers across ~40 checks) settles what is standard versus
novel. The "
799             "architecture's <b>skeleton is standard and must be cited, not claimed</b>.
"
800             "Candidate-windows-then-learned-scorer is <i>temporal action
localization</i> (TAL, "
801             "proposal-then-score). The reconciler's over-segmentation fixes are
<i>temporal action "
802             "segmentation</i> (TAS) refinement / post-processing. &ldquo;Signal not
verdict / a learned "
803             "arbiter over cue columns&rdquo; is late / score-level fusion realized as
stacked "
804             "generalization &mdash; roughly 20 years old and already present in racket
sports (CVIU 2009; "
805             "Ghosh et al. 2018 SVM-over-features). In-domain prior art (See et al.
2024/25 rally start/end; "
806             "MonoTrack; Hsu et al. 2024; Goh et al. 2023 service fault; ShuttleSet)
hard-codes the same "
807             "cues as if/else verdicts."
808         )
809     )
810     story.append(
811         P(
812             "Against that backdrop, <b>what is genuinely novel</b> is narrow and we park

```

```

our claims there: "
813         "(1) the <b>report-first reconciler-as-linter</b> bundle &mdash;
contradiction-detection over a "
814         "persisted temporal decision, with auditable structured interval
corrections, per-rule A/B "
815         "lift-gating <i>before</i> auto-apply, idempotency and fixed precedence,
human-in-the-loop, and "
816         "feeding a (value, confidence) scorer &mdash; for which no surveyed paper
bundles all parts; and "
817         "(2) this <b>disciplined small-N promotion gate</b> on a deliberately tiny
golden set. "
818         "Everything else (the fusion mechanism, the persist-once substrate, cross-
check-and-correct, "
819         "the no-regression harness) is textbook and we claim only the specific
discipline, not the idea."
820     )
821 )
822
823     # ===== 6. Limitations =====
824     story.append(Paragraph("6&nbsp;&nbsp;&nbsp;Limitations", H1))
825     lims = [
826         ("Sample size n=6.", "Enough for direction-checks and LOVO sign-tests, not
enough to claim a "
827         "general model. At n=6 a ~+0.08 lift cannot reach p < 0.05; corpus growth is
the highest-R0I "
828         "unblock."),
829         ("CIs span 0 for person and audio.", "Both fusion cues (P3, P4) have ?F1
confidence "
830         "intervals straddling zero and non-significant sign tests, so neither is
promotable yet despite "
831         "audio's encouraging +27.7% point estimate."),
832         ("Court polygons absent.", "Without per-video court masks the person cue counts
spectators, "
833         "which is the mechanical reason +person is a wash. Activating court polygons is
the obvious next "
834         "fix."),
835         ("Served path is mock-tested.", "The production fusion segmenter is verified
offline on cached "
836         "trajectories, not yet on live real video; flipping the served default stays
owner-gated."),
837         ("Detector domain gap unmeasured.", "WASB/TrackNet were trained on other rigs;
detector accuracy "
838         "on our footage is not yet quantified, and every downstream cue inherits that
gap."),
839     ]
840     for head, txt in lims:
841         story.append(P(f"<b>{head}</b> {txt}", BODY_TIGHT))
842
843     # ===== 7. Planned Work =====
844     story.append(Paragraph("7&nbsp;&nbsp;&nbsp;Planned Work", H1))
845     plans = [
846         ("Grow the corpus via the oracle.", "The #1 lever. Priority: multi-court
badminton (the target "
847         "distribution and where the ship target lives), then >=3 matches per new
sport (table tennis "
848         "first &mdash; WASB transfers at F1 0.909). At n=10 with 9 wins, p ≈
0.011 &mdash; "
849         "significance is reachable within a month, adults only."),
850         ("Activate court polygons.", "The intake manifest already collects
court_corners, net_line, and "
851         "singles/doubles per video, so the detector can ignore people outside the court
and require >=2 "
852         "players on court &mdash; the direct fix for the person-cue noise above."),
853         ("Measure audio on shuttle-only directly.", "Audio is parked (E1 probe found
~2.2x "

```

```

854         "discrimination, AUC ~0.6), not in reserve; the planned Fusion-P4 path lands it
flag-gated and "
855         "default-OFF, promoted only on a measured LOVO lift."),
856         ("Shuttle-dropped cue + wire Gate-A to production.", "The cheapest current win
is wiring the "
857         "already-built net-crossing Gate-A into the live segmenter (it is presently
eval-only). A faithful "
858         "serve-contact boundary definition structurally fixes the held-shuttle false
positive; a future "
859         "&ldquo;shuttle-on-floor&rdquo; cue extends the same substrate reasoning."),
860         ("Owned model: Qwen3-VL (Apache-2.0).", "The roadmap starts with the TAL head,
not the VLM, "
861         "because boundary accuracy is the product and fine-tuned VLMS miss strict
boundaries. Base model "
862         "Qwen3-VL (Apache-2.0), InternVL3 (MIT) fallback; commercial-clean licensing
for code, data, and "
863         "weights; never train on paid-LLM outputs."),
864         ("Execution-policy resilience.", "Motivated by a real Gemini hang with no
client-side timeout, a "
865         "per-job declarative ExecutionPolicy (op_timeout_sec, max_wait_sec, self-
scheduling via the DB as "
866         "coordinator) makes every long operation declare how it waits, aborts, and
resumes."),
867     ]
868     for head, txt in plans:
869         story.append(P(f"<b>{head}</b> {txt}", BODY_TIGHT))
870
871     # ===== 8. Engineering & Reproducibility =====
872     story.append(Paragraph("&nbsp;&nbsp;&nbsp;Engineering & Reproducibility", H1))
873     story.append(
874         P(
875             "<b>Telemetry black box.</b> A rotating ~10&nbsp;KB JSONL runlog (atomic tmp
+ os.replace on "
876             "every write, so a hard kill never leaves it half-written) records execution
velocity plus "
877             "machine health, so a post-mortem on an OS/OOM/thermal kill with no Python
traceback is a "
878             "bullseye instead of a guess. Each sample captures GPU
temp/util/power/clocks/VRAM (with a "
879             "derived throttle flag at SM clock &lt; 0.8&times; max or temp &ge;
84&nbsp;C) and CPU/RAM. "
880             "It is the observability companion to the ExecutionPolicy control layer."
881         )
882     )
883     story.append(
884         P(
885             "<b>Experiment leaderboard.</b> Each compare() appends one compact
reproducible row to "
886             "<font face=\"%s\">eval_baselines/experiment_leaderboard.json</font> &mdash;
a tracked location "
887             "that survives a clean checkout. Every row is pinned to (exact recipe via a
12-char "
888             "spec_hash, exact label set via a label_set_hash, exact IoU), so any
regression is traceable "
889             "to a recipe change rather than drift." % FONT_ITALIC
890         )
891     )
892     story.append(
893         P(
894             "<b>CI gates.</b> Four gates run on every PR (Python 3.13): Ruff lint over
the whole repo; a "
895             "lenient-but-green mypy baseline with an explicit ratchet-down quarantine; a
syntax + import "
896             "smoke with no GPU stack (guarding the class of breakage where a module-
level import error "

```

```

897         "ships while the suite stays green); and the full offline mocked suite with
a coverage floor "
898         "(&minus;&minus;cov-fail-under=70, calibrated against the measured 72%).
Combined with default-"
899         "OFF cues and a pinned CONTROL variant, this makes rollback a config flip,
never a git revert."
900     )
901 )
902
903 # ===== Conclusion =====
904 story.append(Paragraph("9&nbsp;&nbsp;Conclusion", H1))
905 story.append(
906     P(
907         "The contribution of this work is less any single F1 number and more a
<b>method for being "
908         "honest at small scale</b>. On a 6-match owned golden corpus, a shuttle-
trajectory substrate "
909         "feeding a tiny logistic scorer, with every cue entering as a default-OFF
feature column and "
910         "promoted only on a held-out lift whose confidence interval clears zero,
produced exactly one "
911         f"honestly-promoted cue ({DELTA}F1 +0.074, p=0.031) and two cues that
&mdash; reported in full "
912         "&mdash; do not yet clear the bar. By treating the labelling vendor as a
swappable oracle, the "
913         "evaluation rigor is decoupled from data provenance, so the cheapest path to
a publishable "
914         "result is simply more labelled matches: at n=10 with 9 wins, significance
is one month away. "
915         "The architecture is deliberately standard where the literature is mature
and claims novelty "
916         "only on the report-first reconciler and the small-N promotion discipline
&mdash; the honest "
917         "ablation backbone this report is built to be."
918     )
919 )
920
921 return story
922
923
924 def main():
925     build_chart()
926
927     doc = BaseDocTemplate(
928         PDF_PATH,
929         pagesize=A4,
930         leftMargin=2 * cm,
931         rightMargin=2 * cm,
932         topMargin=1.8 * cm,
933         bottomMargin=1.8 * cm,
934         title="Local, Cheap, AI Rally Detection for Racket Sports - Quality Report",
935         author="badminton-highlight-indexer",
936     )
937     frame = Frame(
938         doc.leftMargin,
939         doc.bottomMargin,
940         doc.width,
941         doc.height,
942         id="body",
943         leftPadding=0,
944         rightPadding=0,
945         topPadding=0,
946         bottomPadding=0,
947     )
948     first = PageTemplate(id="First", frames=[frame], onPage=on_first_page)

```

```

949     later = PageTemplate(id="Later", frames=[frame], onPage=on_page)
950     doc.addPageTemplates([first, later])
951
952     story = build_story()
953     # Page 1 uses the banner template (First); every page after switches to the
954     # running-header template (Later). NextPageTemplate takes effect at the next
955     # page break, so it must be emitted on page 1.
956     story.insert(0, NextPageTemplate("Later"))
957     # No banner anymore ? leave a modest top gap so the centered title sits a
958     # little below the top margin, like a paper's first page.
959     story.insert(1, Spacer(1, 0.6 * cm))
960
961     doc.build(story)
962
963     size = os.path.getsize(PDF_PATH)
964     print(f"Wrote {PDF_PATH} ({size} bytes)")
965     # cleanup the temp chart? keep it for inspection
966     return size
967
968
969     if __name__ == "__main__":
970         main()
971

```

23. user (2026-06-14T10:10:57.821Z)

```

keys: ['name', 'fps', 'frame_width', 'candidates', 'gts']
cand keys: ['start', 'end', 'shuttle', 'person', 'label', 'audio']
shuttle: ['duration', 'peak_velocity', 'mean_velocity', 'active_ratio', 'net_crossings']
person: ['players_in_court', 'two_sided_motion', 'mean_player_motion', 'in_court_ratio',
'shuttle_player_colocation']
n_cands: 82
n_gts: 48

```

24. assistant (2026-06-14T10:11:02.214Z)

Now I understand the data shape. Key observations:

- The golden JSON candidates have `shuttle`, `person`, **and `audio`** keys.
- `shuttle` features include `net\_crossings` (used by Gate-A's `min\_crossings` gate AND a column).
- The shuttle `SHUTTLE\_FEATURE\_NAMES` = `[duration, peak\_velocity, mean\_velocity, active\_ratio]` (4 features per distill_local). `net\_crossings` is the 5th key but is the structural gate. Let me verify the exact feature name lists and audio feature names.

25. user (2026-06-14T10:11:07.313Z)

```

SHUTTLE distill FEATURE_NAMES: ['duration', 'peak_velocity', 'mean_velocity', 'active_ratio',
'net_crossings']
---PERSON---
PERSON: ['players_in_court', 'two_sided_motion', 'mean_player_motion', 'in_court_ratio',
'shuttle_player_colocation']
---AUDIO---
backend/eval/fusion_audio.py:68:         for fn in af.AUDIO_FEATURE_NAMES:
backend/eval/fusion_audio.py:84:         feature_names=list(SHUTTLE_FEATURE_NAMES) +
list(PERSON_FEATURE_NAMES) + list(af.AUDIO_FEATURE_NAMES),
Binary file backend/eval/__pycache__/fusion_audio.cpython-313.pyc matches
backend/pipeline/detectors/audio_features.py:15: AUDIO_FEATURE_NAMES = ["audio_hit_count",
"audio_hit_rate", "audio_hit_strength_mean"]
backend/pipeline/detectors/audio_features.py:28:         return {k: 0.0 for k in
AUDIO_FEATURE_NAMES}
Binary file backend/pipeline/detectors/__pycache__/audio_features.cpython-313.pyc matches

```

26. user (2026-06-14T10:11:07.520Z)

```
audio keys: ['audio_hit_count', 'audio_hit_rate', 'audio_hit_strength_mean']
shuttle vals: {'duration': 25.408742075408743, 'peak_velocity': 8.80786690537186,
'mean_velocity': 0.595300423032679, 'active_ratio': 0.5810899540380827, 'net_crossings': 12.0}
sample net_crossings across: [12.0, 14.0, 1.0, 2.0, 12.0, 24.0, 36.0, 8.0, 12.0, 16.0]
```

27. assistant (2026-06-14T10:11:12.293Z)

Important detail: `distill\_local.FEATURE\_NAMES` **includes** `net\_crossings` (5 features). So `SHUTTLE\_FEATURE\_NAMES` = `[duration, peak\_velocity, mean\_velocity, active\_ratio, net\_crossings]`. Let me read fusion_audio.py to see how audio is loaded and confirm the load pattern, then look at the QUALITY_ITERATIONS Gate-A entry to understand the exact known result I must reproduce.

28. user (2026-06-14T10:11:12.732Z)

```
1      """Fusion P4 ? measure the INCREMENTAL audio-cue lift on top of the person cue (CPU-
only).
2
3      Reuses the person-feature JSONs from `fusion_golden.py` (no GPU re-detect): `augment`
runs
4      `detect_hits_in_video` once per video (ffmpeg + numpy, CPU) and adds the 3 audio
features per
5      window; `compare_increment` then runs the SHIPPED harness between a shuttle+person
variant
6      and a shuttle+person+audio variant ? so audio is measured as a marginal contribution ON
TOP
7      of person, not against raw shuttle. Audio enters only as columns the learned scorer
weights
8      (no special-casing, default-OFF); the golden-set ?F1 decides whether it earns its keep.
9
10     Run (no GPU):
11         python -m backend.eval.fusion_audio --manifest output/human_lovo_manifest.json
--features-dir output --augment --compare
12     """
13     from __future__ import annotations
14
15     import argparse
16     import glob
17     import json
18     import os
19     from typing import Any, Dict, List
20
21     from backend.eval import experiment
22     from backend.eval.fusion_compare import delta_stats
23     from backend.eval.fusion_golden import PERSON_FEATURE_NAMES, SHUTTLE_FEATURE_NAMES,
_derive_video_path
24     from backend.pipeline.detectors import audio_features as af
25     from backend.pipeline.audio.hit_detector import detect_hits_in_video
26
27
28     def augment(manifest_path: str, features_dir: str, k: float = 2.5) -> List[str]:
29         """Add the 3 audio features to each existing ``<name>_golden_features.json``
window."""
30         with open(manifest_path, encoding="utf-8") as f:
31             specs = json.load(f)
32             written: List[str] = []
33             for spec in specs:
34                 jp = os.path.join(features_dir, f"{spec['name']}_golden_features.json")
35                 if not os.path.isfile(jp):
36                     print(f"[fusion_audio] {spec['name']}: no feature JSON ? run fusion_golden
first; skipping")
37                     continue
38                 video = _derive_video_path(spec)
39                 print(f"[fusion_audio] {spec['name']}: detecting audio hits (k={k})...",
flush=True)
```

```

40         hits = detect_hits_in_video(video, k=k)
41         with open(jp, encoding="utf-8") as f:
42             data = json.load(f)
43             for c in data["candidates"]:
44                 c["audio"] = af.compute_audio_features(hits, c["start"], c["end"])
45             with open(jp, "w", encoding="utf-8") as f:
46                 json.dump(data, f, indent=2)
47             print(f"[fusion_audio] {spec['name']}: {len(hits)} hits over
{len(data['candidates'])} windows", flush=True)
48             written.append(jp)
49         return written
50
51
52     def load_videos_with_audio(features_dir: str) -> List[experiment.VideoCandidates]:
53         """Load the feature JSONs into VideoCandidates including the audio columns (requires
54         `augment` to have run ? raises if a JSON lacks audio)."""
55         videos: List[experiment.VideoCandidates] = []
56         for path in sorted(glob.glob(os.path.join(features_dir, "*_golden_features.json"))):
57             with open(path, encoding="utf-8") as f:
58                 d = json.load(f)
59                 cands = d["candidates"]
60                 if cands and "audio" not in cands[0]:
61                     raise SystemExit(f"{path} has no audio features ? run `--augment` first")
62                 intervals = [(float(c["start"]), float(c["end"])) for c in cands]
63                 features: Dict[str, List[float]] = {}
64                 for fn in SHUTTLE_FEATURE_NAMES:
65                     features[fn] = [float(c["shuttle"][fn]) for c in cands]
66                 for fn in PERSON_FEATURE_NAMES:
67                     features[fn] = [float(c["person"][fn]) for c in cands]
68                 for fn in af.AUDIO_FEATURE_NAMES:
69                     features[fn] = [float(c["audio"][fn]) for c in cands]
70                 gts = [(float(a), float(b)) for a, b in d["gts"]]
71                 videos.append(experiment.VideoCandidates(name=d["name"], intervals=intervals,
72                                                         features=features, gts=gts))
73         return videos
74
75
76     def compare_increment(videos: List[experiment.VideoCandidates], iou: float = 0.5,
77                          threshold: float = 0.5) -> Dict[str, Any]:
78         """Incremental audio lift: shuttle+person vs shuttle+person+audio (honest
79         LOVO)."""
80         person = experiment.Variant(
81             name="shuttle_person",
82             feature_names=list(SHUTTLE_FEATURE_NAMES) + list(PERSON_FEATURE_NAMES),
83             threshold=threshold)
84         person_audio = experiment.Variant(
85             name="shuttle_person_audio",
86             feature_names=list(SHUTTLE_FEATURE_NAMES) + list(PERSON_FEATURE_NAMES) +
87             list(af.AUDIO_FEATURE_NAMES),
88             threshold=threshold)
89         return experiment.compare(videos, person, person_audio, iou=iou, honest=True)
90
91
92     def format_result(res: Dict[str, Any], iou: float) -> str:
93         a, b, d = res["variant_a"]["aggregate"], res["variant_b"]["aggregate"], res["delta"]
94         ds = delta_stats(res)
95         st = ds["sign_test"]
96         lines = [
97             f"=== Fusion P4 incremental AUDIO LOVO (n={res['num_videos']}, IoU>={iou},
98             honest) ===",
99             f" shuttle+person          : F1={a['f1']:.3f}  P={a['precision']:.3f}
100             R={a['recall']:.3f}",
101             f" shuttle+person+audio : F1={b['f1']:.3f}  P={b['precision']:.3f}
102             R={b['recall']:.3f}",
103             f" incremental ?F1 = {d['f1']:.3f}  ({res['delta_pct']['f1']:.1f}%)",

```

```

98         f" paired ?F1 {ds['mean_delta']:+.3f} [95% CI {ds['ci_low']:+.3f},
{ds['ci_high']:+.3f}] "
99         f"| sign test {int(st['wins'])}W/{int(st['losses'])}L (p={st['p_value']:.3f}) "
100        f"| CI clears 0: {ds['ci_excludes_zero']}",
101    ]
102    for pv in res["per_video_delta"]:
103        lines.append(f" {pv['video']:<26s} ?F1={pv['f1']:+.3f}")
104    return "\n".join(lines)
105
106
107    def main() -> None:
108        ap = argparse.ArgumentParser(description="Fusion P4 ? incremental audio lift on the
golden set.")
109        ap.add_argument("--manifest", default="output/human_lovo_manifest.json")
110        ap.add_argument("--features-dir", default="output")
111        ap.add_argument("--k", type=float, default=2.5, help="hit-detector sensitivity")
112        ap.add_argument("--augment", action="store_true", help="add audio features to the
existing JSONs")
113        ap.add_argument("--compare", action="store_true", help="report the incremental audio
LOVO ?F1")
114        ap.add_argument("--iou", type=float, default=0.5)
115        ap.add_argument("--record", action="store_true")
116        args = ap.parse_args()
117
118        if args.augment:
119            augment(args.manifest, args.features_dir, args.k)
120        if args.compare:
121            videos = load_videos_with_audio(args.features_dir)
122            if len(videos) < 2:
123                raise SystemExit(f"need >=2 augmented feature files in {args.features_dir}")
124            res = compare_increment(videos, iou=args.iou)
125            print(format_result(res, args.iou))
126            if args.record:
127                row = experiment.record_experiment(res)
128                print(f"recorded -> eval_baselines/experiment_leaderboard.json
({row['timestamp']}")
129
130
131    if __name__ == "__main__":
132        main()
133

```

29. user (2026-06-14T10:11:14.268Z)

52: promote noise and can't disable a structural guard (the proven Gate-A held-shuttle guard stays on regardless of a thin

70:**Gate-A-only structural + `min\_n = 5`**.

91:### Acting on the first A/B: Gate-A PROMOTED + learned variant FIXED + GX010125 corroboration *(2026-06-14)*

92:**Gate-A (net-crossings ? 2) ? PROMOTED (first cue promoted via the harness).** The measured win

93:(?F1 +0.074, **6/6 golden matches, sign-test p=0.031**, CI [+0.042,+0.104]; over-seg 1.68?1.01 ? the

95:bar. **Recommended config:** `indexing.fusion.min\_crossings = 2` on the fusion path (the eval-only Gate A

100:trajectory (no GPU, isolated `output/GX010125\_run/`) ? 45 candidates ? CONTROL 45 rallies vs **Gate-A 27**;

101:Gate-A dropped **18 low-crossing windows (~1.7 min)** of likely held-shuttle/feed FPs (no golden labels for

111:**Next:** a real fusion-path run to confirm Gate-A served-side; per-video output layout

116:**Gate-A (net-crossings ? 2) is a measured win** ? ?F1 +0.074, 6/6 matches, sign-test **p = 0.031**, CI

122:Next: Gate-A is a candidate for the no-regression promotion gate; the logistic failure is the C2 (calibration)

150:**Hypothesis.** Adjudicating shuttle-seeded windows with the net-crossing **Gate A**

```
(`min_crossings?2`)
164:**Data support for Gate A.** The same archive shows real rallies on the multi-court matches
contain
165:**?2 net-crossings 97% of the time (median 5)** while a single service feed crosses ~once ?
`min_crossings?2`
168:**Shipping shape.** A NEW `fusion` segmenter, **default OFF** (`default_segmenter`
unchanged); Gate A
169:wired into the live trajectory path behind `indexing.min_crossings` (default 0 = OFF ?
wasb/tracknet
186:Gate-A-pruned windows (a P3 redesign, would change the persisted-superset semantics); and
`fusion.velocity_threshold`
234: Gate A leans on this.
240:- **Gate A already existed, just wasn't wired to production** ? the cheapest precision win
is plumbing, not
```

30. assistant (2026-06-14T10:11:19.709Z)

I now have a complete picture. The known Gate-A result to reproduce: ****?F1 +0.074, 6/6 golden matches, sign-test p=0.031, CI [+0.042, +0.104], over-seg 1.68?1.01.****

Critically, Gate-A is the `min\_crossings` gate. To reproduce that exact result, the ablation must be: ****full set with `min\_crossings=2`** vs **full-minus-gate_a with `min\_crossings=0`**.** But the Gate-A measurement in QUALITY_ITERATIONS was Gate-A-vs-control on the recall-oriented candidate set (a static variant, no learned classifier). Let me verify the exact Gate-A measurement recipe so my reproduction matches. Let me check how Gate-A was actually measured.

31. user (2026-06-14T10:11:25.689Z)

```
scratch/ab_eval_demo.py:15:SHUTTLE = ["duration", "peak_velocity", "mean_velocity",
"active_ratio", "net_crossings"]
scratch/ab_eval_demo.py:47:print("\n[1] CURRENT HARNESS A/B ? CONTROL (no gate) vs Gate-A
(net_crossings >= 2)")
scratch/ab_eval_demo.py:49:gateA = Variant(name="gateA", min_crossings=2)
scratch/ab_eval_demo.py:52:print(f"      {'metric':<12}{'CONTROL':>10}{'Gate-A':>10}{'B -
A':>10}")
scratch/ab_eval_demo.py:57:print(f"      segments served: CONTROL={len(preds_ctrl)}
Gate-A={len(preds_gate)} (golden={len(tl.gts)})")
scratch/ab_eval_demo.py:62:for name, preds in (("CONTROL", preds_ctrl), ("Gate-A", preds_gate)):
scratch/ab_n6_lovo.py:96:     gateA = evaluate_variant(videos, Variant(name="gateA",
min_crossings=2), IOU)
scratch/ab_n6_lovo.py:114:     print("\n--- A/B 1: CONTROL vs Gate-A (static) ? C1 honest
aggregation ---")
scratch/gate_ab_ablation.py:1:"""Leave-one-signal-out ablation for the Gate-A/B foundational
decision (CPU, offline).
scratch/gate_ab_ablation.py:18:2. GATE marginal contribution (decision masks, NOT learned
columns): Gate-A (min_crossings>=2)
scratch/gate_ab_ablation.py:125:def gate_ab(videos, variant_a, variant_b, label) -> Dict[str,
Any]:
scratch/gate_ab_ablation.py:175:         ("net_crossings(shuttle-as-feature)",
["net_crossings"]),
scratch/gate_ab_ablation.py:216:     print("\n=== Gate-A ON vs OFF (static CONTROL vs
net_crossings>=2) ===")
scratch/gate_ab_ablation.py:218:     gate_a = experiment.Variant(name="gate_a", min_crossings=2)
scratch/gate_ab_ablation.py:219:     ga = gate_ab(videos, control, gate_a, "Gate-A
(min_crossings>=2)")
scratch/gate_ab_ablation.py:220:     result["gate_a"] = ga
scratch/gate_ab_ablation.py:223:     print(f"      Gate-A  F1={ga['f1_b']:.3f} (P={ga['p_b']:.3f}
R={ga['r_b']:.3f}) "
scratch/gate_ab_ablation.py:233:         r = gate_ab(videos, control, gb, f"Gate-B
(min_players>={n})")
scratch/gate_ab_ablation.py:241:     # ---- Gate-A AND Gate-B stacked (does B add on top of A?)
----
scratch/gate_ab_ablation.py:242:     print("\n=== Gate-A + Gate-B stacked vs Gate-A alone ===")
scratch/gate_ab_ablation.py:245:         a = experiment.Variant(name="gate_a", min_crossings=2)
scratch/gate_ab_ablation.py:246:         b = experiment.Variant(name=f"gate_ab{n}",
```

```

min_crossings=2, min_players=n)
scratch/gate_ab_ablation.py:258:     result["gate_ab_stacked"] = gab_rows
scratch/gate_ab_ablation.py:260:     out = os.path.join("scratch",
"gate_ab_ablation_result.json")
scratch/gate_ab_detail.py:2:d = json.load(open('scratch/gate_ab_ablation_result.json',
encoding='utf-8'))
scratch/gate_ab_detail.py:3:ga = d['gate_a']
scratch/gate_ab_detail.py:4:print('Gate-A per-video DeltaF1 (B-A):')
scratch/gate_ab_detail.py:9:print('Gate-A  F1@k:', {k: round(v, 3) for k, v in
ga['f1k_b'].items()})
scratch/quality_report.py:486:         [P("<b>Gate-A (PROMOTED)</b>", TBL_CELL), P(f"{DELTA}F1
+0.074, p=0.031, CI [+0.042,+0.104]", TBL_CELL)],
scratch/quality_report.py:688:     story.append(Paragraph("4.2   Gate-A: the first
honestly-promoted cue", H2))
===LEADERBOARD===
shuttle_only                vs shuttle_person_fusion      ?f1=-0.02052122118138 | honest
mean_delta=-0.02052122118137999 ci=[-0.16462806145542527,0.16224446866060574] sign={'wins': 2.0,
'losses': 4.0, 'ties': 0.0, 'n_effective': 6.0, 'p_value': 0.6875}
shuttle_person              vs shuttle_person_audio      ?f1=0.07924277067498242 | honest
mean_delta=0.07924277067498244 ci=[-0.053508315003560535,0.2029045239922802] sign={'wins': 4.0,
'losses': 2.0, 'ties': 0.0, 'n_effective': 6.0, 'p_value': 0.6875}

```

32. assistant (2026-06-14T10:11:32.638Z)

There's a very relevant existing scratch script: `scratch/gate\_ab\_ablation.py`. The Gate-A measurement is: `**CONTROL (static, no gate, no classifier)**` vs `**Gate-A (`min_crossings=2`, static)**`. Let me read that scratch ablation to see exactly how the +0.074 was produced, so my runner reproduces it.

33. user (2026-06-14T10:11:33.185Z)

```

1      """Leave-one-signal-out ablation for the Gate-A/B foundational decision (CPU, offline).
2
3      Reuses ONLY the shipped harness (experiment.compare + eval_stats) over the 6 persisted
4      golden feature JSONs (output/*_golden_features.json). No new scoring math, no GPU, no
Gemini.
5
6      Two kinds of question, two mechanisms (mirrors the repo's own distinction):
7
8      1. LEARNED-FUSION marginal contribution (leave-one-signal-out): the "full" learned
variant
9          uses ALL 13 feature columns (shuttle 5 + person 5 + audio 3). For each signal GROUP
(and
10         each individual shuttle feature where sensible) we drop those columns and measure
held-out
11         ?F1 = full - leave_one_out, with bootstrap 95% CI + paired sign test. ?F1 > 0 means
the
12         signal carries marginal information the learned scorer used.
13
14         NOTE the sign convention: we report delta = FULL - DROPPED, so a POSITIVE delta means
the
15         signal HELPS (dropping it hurts). experiment.compare reports B - A, so we set
16         variant_a = leave_one_out (dropped), variant_b = full -> B-A = full - dropped. Good.
17
18         2. GATE marginal contribution (decision masks, NOT learned columns): Gate-A
(min_crossings>=2)
19         and Gate-B (min_players>=N) are re-scoring masks over the recall-oriented CONTROL
proposal
20         set. Measured as static-variant A/B: CONTROL vs CONTROL+gate. Reports F1, ?F1
(CI+sign),
21         and the segment-count / over-seg ratio (C4) that tIoU hides.
22
23         All numbers printed are ACTUAL measured values. Writes a JSON blob to scratch/ for the
doc.
24         """

```

```

25     from __future__ import annotations
26
27     import json
28     import os
29     from typing import Any, Dict, List, Sequence
30
31     from backend.eval import experiment
32     from backend.eval.eval_stats import paired_delta_ci
33     from backend.eval import segmentation_metrics as segm
34     from backend.eval.metrics import score
35     from backend.pipeline.detectors import audio_features as af
36     from backend.pipeline.detectors import fusion_features as ff
37     from backend.eval import distill_local
38
39     SHUTTLE = list(distill_local.FEATURE_NAMES)          # 5
40     PERSON = list(ff.PERSON_FEATURE_NAMES)              # 5
41     AUDIO = list(af.AUDIO_FEATURE_NAMES)                # 3
42     ALL_FEATURES = SHUTTLE + PERSON + AUDIO             # 13
43
44     FEATURES_DIR = "output"
45     IOU = 0.5
46     THRESHOLD = 0.5
47
48
49     def load_videos() -> List[experiment.VideoCandidates]:
50         import glob
51         videos: List[experiment.VideoCandidates] = []
52         for path in sorted(glob.glob(os.path.join(FEATURES_DIR, "*_golden_features.json"))):
53             with open(path, encoding="utf-8") as f:
54                 d = json.load(f)
55                 cands = d["candidates"]
56                 intervals = [(float(c["start"]), float(c["end"])) for c in cands]
57                 feats: Dict[str, List[float]] = {}
58                 for fn in SHUTTLE:
59                     feats[fn] = [float(c["shuttle"][fn]) for c in cands]
60                 for fn in PERSON:
61                     feats[fn] = [float(c["person"][fn]) for c in cands]
62                 for fn in AUDIO:
63                     feats[fn] = [float(c["audio"][fn]) for c in cands]
64                 gts = [(float(a), float(b)) for a, b in d["gts"]]
65                 videos.append(experiment.VideoCandidates(name=d["name"], intervals=intervals,
66                                                         features=feats, gts=gts))
67         return videos
68
69
70     def paired_from_compare(res: Dict[str, Any], metric: str = "f1") -> Dict[str, Any]:
71         """Pair B-A per-video held-out scores -> mean delta + CI + sign test."""
72         by_a = {p["video"]: p[metric] for p in res["variant_a"]["per_video"]}
73         by_b = {p["video"]: p[metric] for p in res["variant_b"]["per_video"]}
74         vids = [p["video"] for p in res["variant_a"]["per_video"]]
75         return paired_delta_ci([by_a[v] for v in vids], [by_b[v] for v in vids])
76
77
78     def learned_lovo(videos, feature_names) -> Dict[str, Any]:
79         """Run a learned LOVO variant over `feature_names`; return aggregate + per-video
80 f1."""
81         v = experiment.Variant(name="v", feature_names=list(feature_names),
82 threshold=THRESHOLD)
83         ev = experiment.evaluate_variant(videos, v, iou=IOU, honest=True)
84         return ev
85
86
87     def leave_one_out_row(videos, full_features, drop, label) -> Dict[str, Any]:
88         """full (all features) as B; leave-one-out (full minus `drop`) as A.
89         delta = B - A = full - dropped -> positive means the dropped signal HELPS."""

```

```

88     kept = [f for f in full_features if f not in set(drop)]
89     a = experiment.Variant(name=f"drop_{label}", feature_names=kept,
threshold=THRESHOLD)
90     b = experiment.Variant(name="full", feature_names=list(full_features),
threshold=THRESHOLD)
91     res = experiment.compare(videos, a, b, iou=IOU, honest=True) # B-A = full - dropped
92     pd = paired_from_compare(res)
93     st = pd["sign_test"]
94     return {
95         "signal": label,
96         "dropped_cols": list(drop),
97         "f1_full": res["variant_b"]["aggregate"]["f1"],
98         "f1_dropped": res["variant_a"]["aggregate"]["f1"],
99         "marginal_dF1": pd["mean_delta"], # full - dropped
100        "ci_low": pd["ci_low"], "ci_high": pd["ci_high"],
101        "ci_excludes_zero": pd["ci_excludes_zero"],
102        "wins": int(st["wins"]), "losses": int(st["losses"]),
103        "p_value": st["p_value"],
104        "per_video_full": {p["video"]: p["f1"] for p in res["variant_b"]["per_video"]},
105        "per_video_dropped": {p["video"]: p["f1"] for p in
res["variant_a"]["per_video"]},
106    }
107
108
109 def static_predict_segcount(variant, videos) -> Dict[str, float]:
110     """Mean F1@k + segment_count_ratio for a static variant across videos."""
111     overlaps = segm.DEFAULT_OVERLAPS
112     f1k = {ov: [] for ov in overlaps}
113     ratios = []
114     for vc in videos:
115         preds = experiment.predict(variant, vc)
116         got = segm.f1_at_overlaps(preds, vc.gts or [], overlaps)
117         for ov in overlaps:
118             f1k[ov].append(got[ov])
119         ratios.append(segm.segment_count_ratio(preds, vc.gts or []))
120     out = {f"f1@{int(ov*100)}": (sum(v)/len(v) if v else 0.0) for ov, v in f1k.items()}
121     out["segment_count_ratio"] = sum(ratios)/len(ratios) if ratios else 0.0
122     return out
123
124
125 def gate_ab(videos, variant_a, variant_b, label) -> Dict[str, Any]:
126     """Static A/B (no learned filter): CONTROL vs CONTROL+gate. Reports F1, ?F1
(CI+sign),
127     and over-seg ratio for both."""
128     res = experiment.compare(videos, variant_a, variant_b, iou=IOU, honest=True)
129     pd = paired_from_compare(res)
130     st = pd["sign_test"]
131     seg_a = static_predict_segcount(variant_a, videos)
132     seg_b = static_predict_segcount(variant_b, videos)
133     # count of predicted segments per variant (for over-seg narrative)
134     def total_preds(v):
135         return sum(len(experiment.predict(v, vc)) for vc in videos)
136     return {
137         "label": label,
138         "f1_a": res["variant_a"]["aggregate"]["f1"],
139         "f1_b": res["variant_b"]["aggregate"]["f1"],
140         "p_a": res["variant_a"]["aggregate"]["precision"],
141         "p_b": res["variant_b"]["aggregate"]["precision"],
142         "r_a": res["variant_a"]["aggregate"]["recall"],
143         "r_b": res["variant_b"]["aggregate"]["recall"],
144         "dF1": pd["mean_delta"], "ci_low": pd["ci_low"], "ci_high": pd["ci_high"],
145         "ci_excludes_zero": pd["ci_excludes_zero"],
146         "wins": int(st["wins"]), "losses": int(st["losses"]), "p_value": st["p_value"],
147         "segratio_a": seg_a["segment_count_ratio"], "segratio_b":
seg_b["segment_count_ratio"],

```

```

148         "n_pred_a": total_preds(variant_a), "n_pred_b": total_preds(variant_b),
149         "per_video_dF1": {p["video"]: p["f1"] for p in res["per_video_delta"]},
150         "f1k_a": seg_a, "f1k_b": seg_b,
151     }
152
153
154     def main() -> None:
155         videos = load_videos()
156         print(f"Loaded {len(videos)} golden videos: {[v.name for v in videos]}")
157         result: Dict[str, Any] = {"n_videos": len(videos), "iou": IOU, "threshold":
THRESHOLD}
158
159         # ---- Full learned variant (all 13 cols) reference ----
160         full_ev = learned_lovo(videos, ALL_FEATURES)
161         result["full_learned"] = {
162             "feature_names": ALL_FEATURES,
163             "aggregate": full_ev["aggregate"],
164             "per_video": {p["video"]: p["f1"] for p in full_ev["per_video"]},
165         }
166         print("\n=== FULL learned LOVO (all 13 feature columns) ===")
167         print(f"    F1={full_ev['aggregate']['f1']:.3f}
P={full_ev['aggregate']['precision']:.3f}    "
168             f"R={full_ev['aggregate']['recall']:.3f}")
169
170         # ---- Leave-one-SIGNAL-GROUP-out ----
171         print("\n=== Leave-one-signal-out marginal contribution (delta = full - dropped)
===")
172         groups = [
173             ("person", PERSON),
174             ("audio", AUDIO),
175             ("net_crossings(shuttle-as-feature)", ["net_crossings"]),
176             ("shuttle_velocity_block", ["peak_velocity", "mean_velocity", "active_ratio"]),
177             ("duration", ["duration"]),
178         ]
179         rows = []
180         for label, drop in groups:
181             row = leave_one_out_row(videos, ALL_FEATURES, drop, label)
182             rows.append(row)
183             print(f"    drop {label:<36s} ?F1(full-drop)={row['marginal_dF1']:+.3f}    "
184                 f"CI[{row['ci_low']:+.3f},{row['ci_high']:+.3f}]    "
185                 f"{row['wins']}W/{row['losses']}L p={row['p_value']:.3f}    "
186                 f"clears0={row['ci_excludes_zero']}")
187         result["leave_one_out"] = rows
188
189         # ---- Also: incremental ADD-each-group-onto-shuttle (matches P3/P4 framing) ----
190         print("\n=== Incremental add-onto-shuttle (B - A) ===")
191         add_rows = []
192         incr = [
193             ("shuttle->+person", SHUTTLE, SHUTTLE + PERSON),
194             ("shuttle->+audio", SHUTTLE, SHUTTLE + AUDIO),
195             ("shuttle+person->+audio", SHUTTLE + PERSON, SHUTTLE + PERSON + AUDIO),
196             ("shuttle->+person+audio", SHUTTLE, ALL_FEATURES),
197         ]
198         for label, a_f, b_f in incr:
199             a = experiment.Variant(name="a", feature_names=a_f, threshold=THRESHOLD)
200             b = experiment.Variant(name="b", feature_names=b_f, threshold=THRESHOLD)
201             res = experiment.compare(videos, a, b, iou=IOU, honest=True)
202             pd = paired_from_compare(res)
203             st = pd["sign_test"]
204             r = {"label": label, "f1_a": res["variant_a"]["aggregate"]["f1"],
205                 "f1_b": res["variant_b"]["aggregate"]["f1"], "dF1": pd["mean_delta"],
206                 "ci_low": pd["ci_low"], "ci_high": pd["ci_high"],
207                 "ci_excludes_zero": pd["ci_excludes_zero"],
208                 "wins": int(st["wins"]), "losses": int(st["losses"]), "p_value":
st["p_value"]}

```

```

209         add_rows.append(r)
210         print(f"  {label:<28s} F1 {r['f1_a']:.3f}->{r['f1_b']:.3f}  ?F1={r['dF1']:+.3f}
"
211             f"CI[{r['ci_low']:+.3f},{r['ci_high']:+.3f}]  {r['wins']]W/{r['losses']]L
"
212             f"p={r['p_value']:.3f} clears0={r['ci_excludes_zero']}")
213     result["incremental_add"] = add_rows
214
215     # ---- GATE-A ON vs OFF (static, the foundational decision) ----
216     print("\n=== Gate-A ON vs OFF (static CONTROL vs net_crossings>=2) ===")
217     control = experiment.Variant(name="control") # merge-all, no gate
218     gate_a = experiment.Variant(name="gate_a", min_crossings=2)
219     ga = gate_ab(videos, control, gate_a, "Gate-A (min_crossings>=2)")
220     result["gate_a"] = ga
221     print(f"  CONTROL F1={ga['f1_a']:.3f} (P={ga['p_a']:.3f} R={ga['r_a']:.3f})  "
222           f"segratio={ga['segratio_a']:.2f}  n_pred={ga['n_pred_a']}")
223     print(f"  Gate-A F1={ga['f1_b']:.3f} (P={ga['p_b']:.3f} R={ga['r_b']:.3f})  "
224           f"segratio={ga['segratio_b']:.2f}  n_pred={ga['n_pred_b']}")
225     print(f"  ?F1={ga['dF1']:+.3f} CI[{ga['ci_low']:+.3f},{ga['ci_high']:+.3f}]  "
226           f"{ga['wins']]W/{ga['losses']]L p={ga['p_value']:.3f}
clears0={ga['ci_excludes_zero']}")
227
228     # ---- GATE-B ON vs OFF (static; min_players sweep). No court polygons -> caveat.
229     print("\n=== Gate-B ON vs OFF (static CONTROL vs players_in_court>=N) ===")
230     gb_rows = []
231     for n in (1, 2, 3, 4):
232         gb = experiment.Variant(name=f"gate_b{n}", min_players=n)
233         r = gate_ab(videos, control, gb, f"Gate-B (min_players>={n})")
234         gb_rows.append(r)
235         print(f"  N>={n}: CONTROL F1={r['f1_a']:.3f} -> Gate-B F1={r['f1_b']:.3f}  "
236               f"?F1={r['dF1']:+.3f} CI[{r['ci_low']:+.3f},{r['ci_high']:+.3f}]  "
237               f"{r['wins']]W/{r['losses']]L p={r['p_value']:.3f}
clears0={r['ci_excludes_zero']}  "
238               f"n_pred {r['n_pred_a']}->{r['n_pred_b']}")
239     result["gate_b"] = gb_rows
240
241     # ---- Gate-A AND Gate-B stacked (does B add on top of A?) ----
242     print("\n=== Gate-A + Gate-B stacked vs Gate-A alone ===")
243     gab_rows = []
244     for n in (1, 2):
245         a = experiment.Variant(name="gate_a", min_crossings=2)
246         b = experiment.Variant(name=f"gate_ab{n}", min_crossings=2, min_players=n)
247         res = experiment.compare(videos, a, b, iou=IOU, honest=True)
248         pd = paired_from_compare(res)
249         st = pd["sign_test"]
250         r = {"n": n, "f1_a": res["variant_a"]["aggregate"]["f1"],
251              "f1_b": res["variant_b"]["aggregate"]["f1"], "dF1": pd["mean_delta"],
252              "ci_low": pd["ci_low"], "ci_high": pd["ci_high"],
253              "ci_excludes_zero": pd["ci_excludes_zero"],
254              "wins": int(st["wins"]), "losses": int(st["losses"]), "p_value":
st["p_value"]}
255         gab_rows.append(r)
256         print(f"  GateA + B(N>={n}) vs GateA: F1 {r['f1_a']:.3f}->{r['f1_b']:.3f}
?F1={r['dF1']:+.3f}  "
257               f"CI[{r['ci_low']:+.3f},{r['ci_high']:+.3f}] {r['wins']]W/{r['losses']]L
p={r['p_value']:.3f}")
258     result["gate_ab_stacked"] = gab_rows
259
260     out = os.path.join("scratch", "gate_ab_ablation_result.json")
261     with open(out, "w", encoding="utf-8") as f:
262         json.dump(result, f, indent=2)
263     print(f"\nWrote {out}")
264
265

```

```
266     if __name__ == "__main__":
267         main()
268
```

34. assistant (2026-06-14T10:11:38.972Z)

This scratch script is the exact prototype. The Gate-A measurement that produces `**+0.074, 6/6, p=0.031**` is: ``gate_ab(videos, control, gate_a)` = `compare(CONTROL, CONTROL+min_crossings=2)` ?` `**static variants**` (no learned classifier). Let me run it to confirm the exact numbers I must reproduce, then design the production runner around this.

35. user (2026-06-14T10:11:40.843Z)

Traceback (most recent call last):

```
File "C:\Users\avidu\Projects\badminton-highlight-indexer\scratch\gate_ab_ablation.py", line
31, in <module>
    from backend.eval import experiment
ModuleNotFoundError: No module named 'backend'
```

36. assistant (2026-06-14T10:11:43.618Z)

Need to set PYTHONPATH / run as a module. Let me run it properly.

37. user (2026-06-14T10:12:00.302Z)

```
GATE-A static CONTROL vs min_crossings=2:
marginal dF1 (full-ablated) = +0.0739
CI=[+0.0417,+0.1045] clears0=True
sign 6W/0L p=0.0312
segratio CONTROL=1.675 -> Gate-A=1.014
```

38. assistant (2026-06-14T10:12:11.804Z)

Confirmed the exact reproduction target:

```
- **marginal ?F1 = +0.0739** (? +0.074)
- **CI = [+0.0417, +0.1045]** (? [+0.042, +0.104])
- **sign 6W/0L, p=0.0312** (? 0.031)
- **over-seg 1.675 ? 1.014** (? 1.68 ? 1.01)
```

This matches QUALITY_ITERATIONS exactly. The Gate-A ablation semantics: `**full = CONTROL+gate (min_crossings=2)**`, `**ablated = full with min_crossings=0**`, and `marginal = full ? ablated`.

Now I understand the design clearly. Let me design the runner. The key insight on how to unify Gate-A (a gate knob) with feature-column signals (person, audio, shuttle) into ONE ``full`` reference:

The "full set" is a Variant with all gate knobs ON. For each signal:

```
- **Feature-column signal** (person, audio, shuttle columns): ablated = full minus those
columns.
- **Gate signal** (gate_a): ablated = full with `min_crossings=0`.
```

But there's a subtlety: the Gate-A litmus uses a `**static**` full (no classifier). The plan §2 example shows ``full = Variant(feature_names=ALL_COLUMNS, min_crossings=2)``. For the litmus to reproduce `+0.074`, the Gate-A ablation must be measured against the SAME baseline used historically: CONTROL (static) vs CONTROL+gate.

The cleanest design: the runner takes ``cfg.full`` as the reference Variant. For Gate-A's row, ``ablated`` = full with ``min_crossings=0``. If ``cfg.full`` is the static CONTROL+gate variant (no feature_names, min_crossings=2), then Gate-A ablation = CONTROL+gate vs CONTROL = exactly the litmus. Let me verify this exact equivalence and also confirm that when full has min_crossings=2 but no feature_names, the per-signal feature drops are no-ops (good ? we expose a separate learned-full for column signals).

Let me mark a chapter and start building.

39. user (2026-06-14T10:12:12.502Z)

Chapter marked: "Build ablation runner". Continue your current work.

40. assistant (2026-06-14T10:12:16.481Z)

Now let me check the worktree's existing test structure and how tests/run_all_tests work, and confirm reportlab/pypdf availability.

41. user (2026-06-14T10:12:20.004Z)

```
Exit code 2
__init__.py
test_annotation_providers.py
test_api_endpoints.py
test_async_jobs.py
test_audio_features.py
test_automated_provider.py
test_base.py
test_calibrate_local.py
test_calibrate_wasb.py
test_calibration.py
test_cleanup_caches.py
test_compiler.py
test_config.py
test_config_wasb.py
test_database.py
test_detector_keying.py
test_distill_local.py
test_eval_annotations.py
test_eval_batch.py
test_eval_classifier.py
test_eval_harness.py
test_eval_metrics.py
test_eval_partial_labels.py
test_eval_regression.py
test_eval_stats.py
test_eval_training.py
test_execution_policy.py
test_experiment.py
test_experiment_honest.py
test_export_wasb_segments.py
test_fusion_compare.py
test_fusion_features.py
test_fusion_golden_telemetry.py
test_fusion_hybrid.py
test_gemini.py
test_gemini_refine.py
test_geometry.py
test_gt_loader.py
test_hashing.py
test_hit_detector.py
===DEPS===
reportlab 4.5.1
pypdf 6.13.0
pytest 9.0.3
===CFG===
ls: cannot access 'pyproject.toml': No such file or directory
ls: cannot access 'setup.cfg': No such file or directory
pytest.ini
run_all_tests.py
```

42. assistant (2026-06-14T10:12:24.948Z)

reportlab, pypdf, pytest all available. Let me check the test_fusion_compare.py and test_experiment_honest.py for patterns (how they build synthetic VideoCandidates), plus check ruff/mypy config and the CI workflow.

43. user (2026-06-14T10:12:26.595Z)

```
1      """GPU-free tests for the fusion-P3 measurement (fusion_golden extractor +
fusion_compare LOVO).
2
3      The LOVO driver is exercised on a synthetic 6-video set where the SHUTTLE features are
4      non-separating (constant) but a PERSON feature cleanly separates rallies ? so the
harness must
5      report a positive fusion ?F1. The extractor is exercised with a stubbed detector (no
GPU).
6      """
7      import json
8      import os
9      from types import SimpleNamespace
10
11     import pytest
12
13     from backend.eval import fusion_compare
14     from backend.eval.fusion_compare import PERSON_FEATURE_NAMES, SHUTTLE_FEATURE_NAMES
15
16
17     def _write_video(d, name):
18         """A video whose shuttle features are identical for pos/neg (no signal) but whose
19         players_in_court separates rallies (4) from junk (0)."""
20         cand = []
21         for (s, e), pos in [(0.0, 10.0), 1), (20.0, 30.0), 1), (40.0, 41.0), 0), (50.0,
51.0), 0)]:
22             shuttle = {fn: 1.0 for fn in SHUTTLE_FEATURE_NAMES} # constant
-> non-separating
23             person = {fn: 0.0 for fn in PERSON_FEATURE_NAMES}
24             person["players_in_court"] = 4.0 if pos else 0.0 # the
separating signal
25             person["in_court_ratio"] = 1.0 if pos else 0.0
26             cand.append({"start": s, "end": e, "shuttle": shuttle, "person": person,
"label": pos})
27             data = {"name": name, "fps": 30.0, "frame_width": 1920.0,
28                     "candidates": cand, "gts": [[0.0, 10.0], [20.0, 30.0]]}
29             with open(os.path.join(d, f"{name}_golden_features.json"), "w", encoding="utf-8") as
f:
30                 json.dump(data, f)
31
32
33     def test_fusion_compare_reports_person_driven_lift(tmp_path):
34         d = str(tmp_path)
35         for k in range(6):
36             _write_video(d, f"vid{k}")
37         videos = fusion_compare.load_videos(d)
38         assert len(videos) == 6
39         res = fusion_compare.compare(videos, iou=0.5)
40         assert res["num_videos"] == 6
41         assert res["variant_b"]["honest_lovo"] is True # LOVO-honest learned
variant
42         # Fusion (with the separating person feature) must beat shuttle-only (no signal).
43         assert res["delta"]["f1"] > 0.0
44         assert res["variant_b"]["aggregate"]["f1"] > res["variant_a"]["aggregate"]["f1"]
45         # format_result must not crash and must mention both variants.
46         out = fusion_compare.format_result(res, 0.5)
47         assert "shuttle-only" in out and "+person" in out
48
49
50     def test_fusion_compare_delta_stats_and_ci(tmp_path):
```

```

51         """delta_stats yields a paired bootstrap CI + sign test; format_result reports
them."""
52         d = str(tmp_path)
53         for k in range(6):
54             _write_video(d, f"vid{k}")
55             res = fusion_compare.compare(fusion_compare.load_videos(d), iou=0.5)
56             ds = fusion_compare.delta_stats(res)
57             assert set(ds) >= {"mean_delta", "ci_low", "ci_high", "sign_test",
"ci_excludes_zero"}
58             assert ds["ci_low"] - 1e-9 <= ds["mean_delta"] <= ds["ci_high"] + 1e-9 # CI
brackets mean (±fp)
59             assert ds["sign_test"]["wins"] == 6 # fusion wins on every video (person
feature separates)
60             assert ds["ci_excludes_zero"] is True
61             out = fusion_compare.format_result(res, 0.5)
62             assert "CI" in out and "sign test" in out
63
64
65     def test_fusion_compare_needs_two_videos(tmp_path):
66         _write_video(str(tmp_path), "only")
67         videos = fusion_compare.load_videos(str(tmp_path))
68         assert len(videos) == 1 # compare() / experiment.compare needs >=2 for honest LOVO
69
70
71     # ----- extractor assembly
72
73     def test_extract_video_assembles_shuttle_and_person(monkeypatch):
74         from backend.eval import fusion_golden
75         from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
76
77         intervals = [(0.0, 10.0), (40.0, 41.0)]
78         x_shuttle = [[10.0, 0.3, 0.2, 0.5, 4.0], [1.0, 0.1, 0.05, 0.2, 0.0]]
79         points = [SimpleNamespace(frame=i, visible=True, x=500.0, y=(100.0 if i % 2 else
900.0))
80                     for i in range(10)]
81
82         monkeypatch.setattr(fusion_golden.distill_local, "build_candidates",
83                             lambda traj, fps, fw: (intervals, x_shuttle))
84         monkeypatch.setattr(TrackNetRunner, "parse_trajectory_csv", staticmethod(lambda
traj: points))
85         monkeypatch.setattr(fusion_golden, "load_golden_gts", lambda labels: [(0.0, 10.0)])
86
87         class _StubDetector:
88             def detections_over_windows(self, video, windows, frame_skip, progress_cb=None):
89                 return [{"frames": {sf: [(1, (10.0, 10.0, 30.0, 40.0))]},
90                             "fps": 30.0, "frame_width": 1920.0, "frame_height": 1080.0}
91                         for (sf, ef) in windows]
92
93         spec = {"name": "t", "trajectory": "bar.csv", "fps": 30.0, "frame_width": 1920.0,
94                "labels": "foo.rallies.csv"}
95         data = fusion_golden.extract_video(spec, detector=_StubDetector(), frame_skip=3,
iou=0.5)
96
97         assert data["name"] == "t"
98         assert data["gts"] == [(0.0, 10.0)]
99         assert len(data["candidates"]) == 2
100        c0 = data["candidates"][0]
101        assert set(c0["shuttle"]) == set(SHUTTLE_FEATURE_NAMES)
102        assert set(c0["person"]) == set(PERSON_FEATURE_NAMES)
103        assert c0["label"] == 1 # (0,10) matches the gt -> positive
104        assert data["candidates"][1]["label"] == 0 # (40,41) -> negative
105        assert all(isinstance(v, float) for v in c0["person"].values())
106
107
108     def test_fusion_golden_run_is_resumable(tmp_path, monkeypatch):

```

```

109         """run() skips a video whose JSON already exists (resume a long run); --fresh redoes
all."""
110         from backend.eval import fusion_golden
111         out_dir = str(tmp_path / "out")
112         os.makedirs(out_dir)
113         manifest = [
114             {"name": "alpha", "trajectory": "a.csv", "fps": 30.0, "frame_width": 1920.0,
"labels": "a.rallies.csv"},
115             {"name": "beta", "trajectory": "b.csv", "fps": 30.0, "frame_width": 1920.0,
"labels": "b.rallies.csv"},
116         ]
117         manifest_path = str(tmp_path / "m.json")
118         with open(manifest_path, "w", encoding="utf-8") as f:
119             json.dump(manifest, f)
120         # a prior run already produced alpha's JSON
121         with open(os.path.join(out_dir, "alpha_golden_features.json"), "w",
encoding="utf-8") as f:
122             json.dump({"name": "alpha", "fps": 30.0, "frame_width": 1920.0, "candidates":
[], "gts": []}, f)
123
124         extracted = []
125
126         def _stub(spec, **kw):
127             extracted.append(spec["name"])
128             return {"name": spec["name"], "fps": 30.0, "frame_width": 1920.0, "candidates":
[], "gts": []}
129
130         monkeypatch.setattr(fusion_golden, "extract_video", _stub)
131
132         fusion_golden.run(manifest_path, out_dir, fresh=False)
133         assert extracted == ["beta"] # alpha skipped (already extracted)
134
135         extracted.clear()
136         fusion_golden.run(manifest_path, out_dir, fresh=True)
137         assert extracted == ["alpha", "beta"] # --fresh redoes both
138
139
140     def test_fusion_golden_smallest_first(tmp_path, monkeypatch):
141         """--smallest-first orders by trajectory size; default keeps manifest order."""
142         from backend.eval import fusion_golden
143         out_dir = str(tmp_path / "out")
144         os.makedirs(out_dir)
145         (tmp_path / "big.csv").write_text("x" * 2000)
146         (tmp_path / "small.csv").write_text("x" * 20)
147         manifest = [
148             {"name": "big", "trajectory": str(tmp_path / "big.csv"), "fps": 30.0,
"frame_width": 1920.0, "labels": "b.rallies.csv"},
149             {"name": "small", "trajectory": str(tmp_path / "small.csv"), "fps": 30.0,
"frame_width": 1920.0, "labels": "s.rallies.csv"},
150         ]
151         mp = str(tmp_path / "m.json")
152         with open(mp, "w", encoding="utf-8") as f:
153             json.dump(manifest, f)
154
155         order = []
156
157         def _stub(spec, **kw):
158             order.append(spec["name"])
159             return {"name": spec["name"], "fps": 30.0, "frame_width": 1920.0, "candidates":
[], "gts": []}
160
161         monkeypatch.setattr(fusion_golden, "extract_video", _stub)
162
163         fusion_golden.run(mp, out_dir, smallest_first=True)
164         assert order == ["small", "big"] # smaller trajectory first

```

```

165
166         order.clear()
167         fusion_golden.run(mp, out_dir, smallest_first=False, fresh=True)
168         assert order == ["big", "small"]          # manifest order preserved
169
170
171     def test_derive_video_path():
172         from backend.eval.fusion_golden import _derive_video_path
173         assert _derive_video_path({"labels": "X/Adarsh and Avi.rallies.csv"}) == "X/Adarsh
and Avi.MP4"
174         assert _derive_video_path({"video_path": "a.mp4", "labels": "b.rallies.csv"}) ==
"a.mp4"
175         with pytest.raises(ValueError):
176             _derive_video_path({"name": "n", "labels": "weird.csv"})
177

```

44. user (2026-06-14T10:12:27.507Z)

[pytest]

When `pytest` is invoked without an explicit path, only collect from tests/.scratch/ holds debugging scripts that hit real APIs (Gemini upload, etc.) and should never run as part of the regular test suite.

```

testpaths = tests
===RUFF/MYPY cfg===
ls: cannot access '.ruff.toml': No such file or directory
ls: cannot access '.mypy.ini': No such file or directory
mypy.ini
ruff.toml
===CI===
ci.yml

```

45. assistant (2026-06-14T10:12:30.280Z)

Let me check the CI workflow (the 4 lanes), ruff config, and mypy config to ensure my code complies.

46. user (2026-06-14T10:12:31.255Z)

```

1     name: CI
2
3     on:
4         push:
5             branches: [master]
6         pull_request:
7
8     permissions:
9         contents: read
10
11     concurrency:
12         group: ci-${{ github.ref }}
13         cancel-in-progress: true
14
15     jobs:
16         # Lint gate (ruff.toml at repo root): pyflakes/pycodestyle + bugbear.
17         # Also the repo-wide syntax guard ? ruff reports invalid-syntax on every
18         # tracked .py, including scripts/ and training/, which the backend-only
19         # py_compile smoke sweep does not reach.
20         lint:
21             name: Ruff lint
22             runs-on: ubuntu-latest
23             steps:
24                 - uses: actions/checkout@v6

```

```

25         - uses: actions/setup-python@v6
26         with:
27             python-version: "3.13"
28         - name: Install ruff
29           run: python -m pip install ruff
30         - name: Ruff check
31           run: ruff check . --output-format github
32
33     # Type gate (mypy.ini at repo root): lenient green baseline with an explicit
34     # per-module quarantine list ? ratchet by removing entries as modules clean
35     # up. Typed core deps are installed so mypy sees their real signatures
36     # (without them ignore_missing_imports would Any-stub half the checks).
37     typecheck:
38       name: Mypy (lenient baseline)
39       runs-on: ubuntu-latest
40       steps:
41         - uses: actions/checkout@v6
42         - uses: actions/setup-python@v6
43           with:
44             python-version: "3.13"
45         - name: Install mypy + typed core deps
46           run: python -m pip install mypy pydantic fastapi uvicorn python-dotenv google-
genai numpy
47         - name: Mypy
48           run: mypy backend training
49
50     # Fast guard against the 7fbb0 class of breakage: a SyntaxError or
51     # module-level import error shipping while the suite stays green.
52     # Deliberately does NOT install cv2/torch/numpy ? the smoke tests must hold
53     # on machines without the GPU stack (test_import_smoke stubs what's absent).
54     import-smoke:
55       name: Syntax sweep + import smoke (no GPU stack)
56       runs-on: ubuntu-latest
57       steps:
58         - uses: actions/checkout@v6
59         - uses: actions/setup-python@v6
60           with:
61             python-version: "3.13"
62         - name: Install minimal deps
63           run: python -m pip install pytest fastapi uvicorn pydantic python-dotenv google-
genai
64         - name: Run import smoke tests
65           run: python -m pytest tests/test_import_smoke.py -v
66
67     full-suite:
68       name: Full test suite (offline, mocked)
69       runs-on: ubuntu-latest
70       steps:
71         - uses: actions/checkout@v6
72         - uses: actions/setup-python@v6
73           with:
74             python-version: "3.13"
75             cache: pip
76         - name: System libs for opencv-python
77           run: |
78             sudo apt-get update
79             sudo apt-get install -y libgl1 libglu1-mesa-dev || sudo apt-get install -y
libgl1 libglu1-mesa-dev
80         - name: Install CPU-only torch (no CUDA wheels on a CPU runner)
81           run: python -m pip install torch torchvision --index-url
https://download.pytorch.org/whl/cpu
82         - name: Install requirements
83           run: python -m pip install -r requirements.txt pytest pytest-cov httpx # httpx:
fastapi TestClient
84         # obsreport (private, soft dep) is absent here: reporting tests skip via

```

```

85         # pytest.importorskip ? run_all_tests.py prints which. The coverage floor
86         # is therefore calibrated against THIS lane's number (local runs with
87         # obsreport installed measure a little higher).
88         # Floor calibrated 2026-06-11 against this lane's measured 72% (local
89         # with obsreport: 73%). Ratchet it UP as coverage grows; never down
90         # without an owner decision.
91         - name: Run full suite (with coverage floor)
92           run: python run_all_tests.py --cov=backend --cov=training --cov-report=term
--cov-fail-under=70
93

```

47. user (2026-06-14T10:12:32.006Z)

```

1      # Lint-only lane (no `ruff format` enforcement ? a whole-repo reformat would
2      # bury blame history for zero behavior value).
3      #
4      # Selected: pyflakes + pycodestyle errors (ruff defaults) + pycodestyle
5      # warnings (W) + bugbear (B). Deferred with intent: I (import-sort) and UP
6      # (pyupgrade) ? both are large mechanical diffs; adopt via a dedicated PR.
7      target-version = "py38"
8
9      # scratch/ = throwaway debugging scripts that may hit real APIs; it is also
10     # excluded from pytest collection (see pytest.ini).
11     extend-exclude = ["scratch"]
12
13     [lint]
14     extend-select = ["W", "B"]
15
16     [lint.per-file-ignores]
17     # Entrypoint scripts deliberately run setup (load_dotenv, logging config,
18     # sys.path) before importing backend modules ? import order is load-bearing.
19     "backend/main.py" = ["E402"]
20     "scripts/run_phase3_eval.py" = ["E402"]
21     "scripts/run_process_and_stitch.py" = ["E402"]
22

```

48. user (2026-06-14T10:12:32.053Z)

```

1      [mypy]
2      # Lenient, GREEN baseline (quality sweep AI-4): the gate protects the clean
3      # modules and pins an explicit quarantine list below ? not a typing crusade.
4      # Ratchet: when you touch a quarantined module, try removing its entry.
5      ignore_missing_imports = True
6      # backend/ is a namespace package (no root __init__.py) ? required so mypy
7      # maps files to backend.* module names instead of duplicating top-level names.
8      explicit_package_bases = True
9
10     # --- Quarantine (known-loose modules, each with a reason) ---
11
12     # Optional module globals (db_instance/global_config set only in main());
13     # properly fixed by the deferred app-factory/lifespan refactor (BACKLOG).
14     [mypy-backend.main]
15     ignore_errors = True
16
17     # Legacy demo segmenter (fabricated metadata; coverage-omitted too).
18     [mypy-backend.pipeline.segmenters.motion]
19     ignore_errors = True
20
21     # yolo_hybrid: the fusion-P1 extraction moved its loose torchvision/cv2 inline code
22     # to detectors/person_detector.py (which IS type-clean, no quarantine). What remains
23     # here is pre-existing orchestration debt unrelated to that move (implicit-Optional
24     # process_video defaults, the Success|Failure ABC-override mismatch, a Future[...]
25     # annotation) ? ratchet those out in a focused typing pass, not here.
26     # (wasb_hybrid/tracknet_hybrid left the quarantine with the AI-7 collapse.)
27     [mypy-backend.pipeline.segmenters.yolo_hybrid]

```

```

28     ignore_errors = True
29
30     [mypy-backend.pipeline.segmenters.gemini]
31     ignore_errors = True
32
33     # Registry builds cls(None, {}) probes; typing it honestly needs a Protocol.
34     [mypy-backend.pipeline.segmenters.base]
35     ignore_errors = True
36

```

49. assistant (2026-06-14T10:12:39.117Z)

I have all the context. The 4 CI lanes: Ruff lint, Mypy, import-smoke, full-suite (coverage floor 70%). Note: `**mypy runs on `backend training`**`, ruff ``target-version = py38`` (so no ``X | Y`` unions in runtime annotations unless ``from __future__ import annotations``, which the codebase uses). Let me check `run_all_tests.py` to confirm how tests are invoked, then write the module.

50. user (2026-06-14T10:12:40.096Z)

```

1     #!/usr/bin/env python
2     """Run the FULL test suite for the badminton-highlight-indexer.
3
4     A single, discoverable entry point so you (or a reviewer / CI) can verify the
5     whole suite before opening a PR:
6
7         python run_all_tests.py           # full suite, quiet
8         python run_all_tests.py -v       # verbose (passthrough to pytest)
9         python run_all_tests.py -k config # filter by keyword (passthrough)
10        python run_all_tests.py --cov=backend --cov-report=term-missing # coverage
11
12    Anything after the script name is passed straight through to pytest.
13
14    Notes
15    -----
16    - The suite is fully mocked (no GPU, no network, no WSL) and runs offline.
17    - The observability **reporting** tests skip cleanly if the optional `obsreport`
18      package isn't installed (they use `pytest.importorskip`). To exercise them too:
19        pip install -e ../sports-obsreport --no-build-isolation
20    - Exit code is pytest's own, so this is CI / pre-PR friendly (0 = all green).
21    """
22    from __future__ import annotations
23
24    import importlib.util
25    import subprocess
26    import sys
27
28
29    def _installed(module: str) -> bool:
30        try:
31            return importlib.util.find_spec(module) is not None
32        except (ImportError, ValueError):
33            return False
34
35
36    def main() -> int:
37        args = list(sys.argv[1:])
38        # Default to quiet unless the caller chose a verbosity.
39        if not any(a in ("-q", "-v", "-vv", "-vvv", "--quiet", "--verbose") for a in args):
40            args = ["-q", *args]
41        cmd = [sys.executable, "-m", "pytest", *args]
42
43        obs = "available" if _installed("obsreport") else "NOT installed ? reporting tests
will SKIP"
44        bar = "=" * 72
45        print(bar)

```

```

46         print(" Full test suite: " + " ".join(cmd))
47         print(f" obsreport (optional reporting dep): {obs}")
48         print(bar, flush=True)
49
50         rc = subprocess.run(cmd).returncode
51
52         print(bar)
53         print(" RESULT: " + ("PASS - all tests green" if rc == 0 else f"FAIL - pytest exit
{rc}"))
54         print(bar)
55         return rc
56
57
58     if __name__ == "__main__":
59         raise SystemExit(main())
60

```

51. assistant (2026-06-14T10:12:47.227Z)

Now I have everything. Let me write `backend/eval/ablation.py`. Design decisions, distilled from the plan:

1. `Signal` dataclass: `name`, `group`, `columns`, `gate\_knob` (e.g. `"min_crossings"`), `structural` flag.
2. `AblationConfig`: `full` Variant (B reference), `signals` inventory, `granularity` ("signal"|"group"), `iou`, `honest`, `seed`, `p\_threshold`.
3. `ablate_one`: builds `without\_s` by subtracting columns / zeroing the gate knob, calls `experiment.compare(without_s, full)`, reads the `honest` block ? one `AblationRow`.
4. `run_ablation`: loops inventory (LOSO or LOGO), returns `AblationReport`.
5. `Verdict logic`: `earns\_keep` (CI clears 0 + sign agrees), `suggestive`, `inert`, `structural\_protected`.
6. `Default inventory` matching the golden JSON shape.
7. `Markdown/ASCII table` sorted by ?F1 desc.
8. `PDF exporter` (reportlab).
9. `CLI` `--features-dir` + `--pdf`.

Let me write it.

52. assistant (2026-06-14T10:14:51.217Z)

"""Leave-one-signal-out / leave-one-group-out ablation runner (Phase 0).

A *systematic, repeatable* version of the one-off ablation (`scratch/gate_ab_ablation.py`): run it and it EXCLUDES a single signal, measures whether quality dropped, and reports ? per signal ? the *marginal contribution* ?F1 (full set vs full-minus-that-signal) with a bootstrap 95% CI + paired sign test + the C4 over-segmentation ratio. See `scratch/ABLATION_FRAMEWORK_PLAN.md` (the design) and `docs/ABLATION_LAYER.md` (the shipped note).

Nothing here is new scoring/stats math. It is a *pure driver* over the shipped `experiment.compare` + `eval_stats` engine, exactly as `compare` was a driver over `metrics`/`calibration`. For each signal `s` the marginal contribution is:

$$\Delta(s) = \text{score}(\text{full set } S) - \text{score}(S \setminus \{s\}) \quad \# \text{ leave-one-out drop, ?F1 primary}$$

operationally `experiment.compare(videos, variant_a=without_s, variant_b=full)` ? A is the ablated (smaller) variant, B is full, so the reported $\Delta(s)$, a per-video-paired ?F1 with bootstrap CI + exact sign test, LOVO-honest for learned variants.

Two ablation granularities fall out of one inventory:

- `Leave-one-signal-out (LOSO)` ? remove ONE column at a time (fine-grained).
- `Leave-one-GROUP-out (LOGO)` ? remove a whole producer's columns at once (the owner's "is this old SIGNAL still relevant?" question at cue granularity).

A signal can be a **feature column** (person/audio/shuttle columns the learned scorer weights) or a **gate knob** (Gate-A's `min_crossings` threshold ? ablated by zeroing the knob, NOT by dropping a column). The runner handles both uniformly: `without_s` subtracts the columns AND/OR zeroes the gate knob from `cfg.full`.

Honest small-N contract (inherited from `eval_stats` / `per_user_gate`; §6 of the plan):

- never a bare mean ? every row ships a CI + the exact sign test;
- a signal **earns its keep** only when removal drops F1 with **CI** excluding 0 AND the sign test agreeing (the `per_user_gate` bar, in reverse);
- **structural signals** are exempt from auto-demotion ? a guard whose value is in failure modes

the golden corpus may lack (Gate-A vs held-shuttle FPs) is marked `structural` and is NEVER flagged dead weight on "no measured lift"; its row reports the (possibly ?0) golden ? honestly but tags `structural_protected`;

- the runner emits `earns_keep` / `suggestive` / `inert` / `structural_protected` ? it MUST NOT print "irrelevant" or "proven useless" (one wash at n=6 is noise, not proof).

Pure + offline + GPU-free. Reuses the persisted golden feature substrate (`fusion_golden.py`) writes one `<name>_golden_features.json` per video). CLI::

```
python -m backend.eval.ablation --features-dir output [--pdf out.pdf]
"""
from __future__ import annotations

import argparse
import glob
import json
import logging
import os
from dataclasses import dataclass, field
from typing import Any, Dict, List, Optional, Sequence, Tuple

from backend.eval import experiment
from backend.eval.experiment import Variant, VideoCandidates

logger = logging.getLogger(__name__)
```

--- The signal inventory (the things the runner ablates), grouped by producer. -----
Source: distill_local.FEATURE_NAMES (shuttle, includes `net_crossings`), `fusion_features` (person), `audio_features` (audio). The shuttle group's velocity block + duration are the **learned-column shuttle signals**; `net_crossings` is **BOTH** a column and the Gate-A gate

```
SHUTTLE_COLUMNS: Tuple[str, ...] = ("duration", "peak_velocity", "mean_velocity",
                                     "active_ratio", "net_crossings")
PERSON_COLUMNS: Tuple[str, ...] = ("players_in_court", "two_sided_motion", "mean_player_motion",
                                   "in_court_ratio", "shuttle_player_colocation")
AUDIO_COLUMNS: Tuple[str, ...] = ("audio_hit_count", "audio_hit_rate",
                                  "audio_hit_strength_mean")
```

Verdict vocabulary (the no-over-claim contract ? never "irrelevant"/"proven useless").

```
EARNS_KEEP = "earns_keep"           # CI clears 0 AND sign test agrees ? removing it hurts
SUGGESTIVE = "suggestive"           # point estimate leans positive but CI spans 0
INERT = "inert"                     # wash this run (?0, CI spans 0) ? candidate, not proof
STRUCTURAL_PROTECTED = "structural_protected" # a guard; never auto-demoted on "no lift"
```

#: Human-readable one-liners for each verdict (used in the PDF + table legend).
VERDICT_LABEL = {

```

    EARNNS_KEEP: "clears-bar",
    SUGGESTIVE: "suggestive",
    INERT: "inert",
    STRUCTURAL_PROTECTED: "structural-protected",
}

```

```
@dataclass(frozen=True)
```

```
class Signal:
```

```
    """One unit of ablation ? a feature column (or a named group) and/or a gate knob.
```

```

    - ``name`` ? table/row identity ("person.colocation" | "gate_a" | "audio").
    - ``group`` ? producer ("person" | "gate_a" | "audio" | "shuttle"). LOGO ablates a whole
group.
    - ``columns`` ? the persisted feature columns this signal contributes (may be empty for a
pure gate signal like Gate-A whose column is shared with the shuttle group).
    - ``gate_knob`` ? e.g. ``"min_crossings"`` for the Gate-A threshold; ablated by zeroing it
on
    the full variant rather than dropping a column.
    - ``structural`` ? a guard whose value is in failure modes the golden corpus may lack; NEVER
flagged dead weight on "no measured lift" (mirrors ``per_user_gate`` ``structural=True``).
    """

```

```

    name: str
    group: str
    columns: Tuple[str, ...] = ()
    gate_knob: Optional[str] = None
    structural: bool = False

```

```
@dataclass
```

```
class AblationConfig:
```

```
    """How to run an ablation over a corpus.
```

```

    - ``full`` ? the full-set reference variant (B). For the litmus Gate-A reproduction this is
the STATIC ``Variant(min_crossings=2)`` (no classifier); for a learned ablation it carries
``feature_names`` = all columns and the gates ON.
    - ``signals`` ? the inventory to ablate.
    - ``granularity`` ? ``"signal"`` (LOSO, one column-set at a time) or ``"group"`` (LOGO).
    - ``iou`` ? temporal-IoU match threshold.
    - ``honest`` ? LOVO for learned variants (the number to trust).
    - ``seed`` ? threaded into the ``eval_stats`` bootstrap (determinism).
    - ``p_threshold`` ? sign-test significance bar for the ``earns_keep`` verdict.
    """

```

```

    full: Variant
    signals: List[Signal]
    granularity: str = "group"
    iou: float = 0.5
    honest: bool = True
    seed: int = 0
    p_threshold: float = 0.05

```

```
@dataclass
```

```
class AblationRow:
```

```
    """One signal's marginal-contribution row (?s) + CI + sign test + C4 + a verdict)."""
```

```

    signal: str
    group: str
    structural: bool
    dropped_columns: List[str]
    gate_knob: Optional[str]
    n: int
    delta_f1: float

```

```

ci_low: float
ci_high: float
ci_excludes_zero: bool
sign_wins: int
sign_losses: int
sign_p_value: float
f1_full: float
f1_ablated: float
seg_ratio_full: float
seg_ratio_ablated: float
verdict: str
structural_protected: bool

def as_dict(self) -> Dict[str, Any]:
    return {
        "signal": self.signal,
        "group": self.group,
        "structural": self.structural,
        "dropped_columns": list(self.dropped_columns),
        "gate_knob": self.gate_knob,
        "n": self.n,
        "delta_f1": self.delta_f1,
        "ci_low": self.ci_low,
        "ci_high": self.ci_high,
        "ci_excludes_zero": self.ci_excludes_zero,
        "sign_test": {"wins": self.sign_wins, "losses": self.sign_losses,
                     "p_value": self.sign_p_value},
        "f1_full": self.f1_full,
        "f1_ablated": self.f1_ablated,
        "segment_count_ratio_full": self.seg_ratio_full,
        "segment_count_ratio_ablated": self.seg_ratio_ablated,
        "verdict": self.verdict,
        "structural_protected": self.structural_protected,
    }

```

@dataclass

class AblationReport:

"""All signals' rows + the run's provenance ? a structured, JSON-able ablation result."""

```

granularity: str
iou: float
num_videos: int
honest_lovo: bool
full_set_signals: List[str]
full_spec_hash: str
label_set_hash: str
rows: List[AblationRow] = field(default_factory=list)

```

def sorted_rows(self) -> List[AblationRow]:

"""Rows by marginal ?F1 desc ? the contribution ranking (the plan's table order)."""
 return sorted(self.rows, key=lambda r: r.delta_f1, reverse=True)

def as_dict(self) -> Dict[str, Any]:

```

    return {
        "granularity": self.granularity,
        "iou": self.iou,
        "num_videos": self.num_videos,
        "honest_lovo": self.honest_lovo,
        "full_set_signals": list(self.full_set_signals),
        "full_spec_hash": self.full_spec_hash,
        "label_set_hash": self.label_set_hash,
        "rows": [r.as_dict() for r in self.sorted_rows()],
    }

```

----- the driver

```
def _ablated_variant(full: Variant, columns: Sequence[str], gate_knob: Optional[str]) -> Variant:
    """Build the leave-one-out (smaller) variant from ``full``: drop ``columns`` from its
    ``feature_names`` and/or zero the named gate knob. Pure ? derives a new Variant, never
    mutates ``full``."""
    cols = set(columns)
    fn = full.feature_names
    ablated_fn = [c for c in fn if c not in cols] if fn is not None else fn
    overrides: Dict[str, Any] = {"feature_names": ablated_fn}
    if gate_knob == "min_crossings":
        overrides["min_crossings"] = 0
    elif gate_knob == "min_players":
        overrides["min_players"] = 0
    elif gate_knob is not None:
        raise ValueError(f"unknown gate_knob {gate_knob!r} (expected
        'min_crossings'/'min_players'")
    dropped_label = ", ".join(sorted(cols)) or (gate_knob or "?")
    return experiment.Variant.from_dict(**full.to_dict(), **overrides,
                                       "name": f"?{dropped_label}")

def _seg_ratio(variant: Variant, videos: Sequence[VideoCandidates]) -> float:
    """Mean predicted-to-GT segment-count ratio across videos (C4 over-seg diagnostic).

    For a learned variant we can't score without a model, so this is only meaningful for the
    static-gate case the litmus uses; learned-variant rows still record it best-effort and fall
    back to 0.0 when a model is required (the CI/sign verdict does not depend on it)."""
    from backend.eval import segmentation_metrics as segm
    ratios: List[float] = []
    for vc in videos:
        try:
            preds = experiment.predict(variant, vc)
        except experiment.ExperimentError:
            return 0.0
        ratios.append(segm.segment_count_ratio(preds, vc.gts or []))
    return sum(ratios) / len(ratios) if ratios else 0.0

def _classify(structural: bool, ci_excludes_zero: bool, sign_wins: int, sign_losses: int,
              sign_p: float, p_threshold: float) -> Tuple[str, bool]:
    """The honest verdict (no over-claim). Returns ``(verdict, structural_protected)``.

    A structural guard is reported but never demoted on "no lift". Otherwise: removing the
    signal *earns its keep* when the ?F1 CI clears 0 AND the sign test agrees (majority worse on
    removal, ``p <= p_threshold``); a positive-leaning but CI-spanning row is ``suggestive``; a
    wash this run is ``inert`` (a candidate, never "proven useless" at n=6)."""
    if structural:
        return STRUCTURAL_PROTECTED, True
    sign_agrees = (sign_p <= p_threshold) and (sign_wins > sign_losses)
    if ci_excludes_zero and sign_agrees:
        return EARNS_KEEP, False
    if sign_wins > sign_losses:          # leans toward "removal hurts" but evidence is thin
        return SUGGESTIVE, False
    return INERT, False

def ablate_one(videos: Sequence[VideoCandidates], cfg: AblationConfig,
               signal: Signal) -> AblationRow:
    """One ablation row: marginal ?(s) + CI + sign test + C4 + a verdict.

    Drops ``signal``'s columns / zeroes its gate knob from ``cfg.full`` to form the ablated
    variant A, then calls ``experiment.compare(A, full)`` so the reported B?A ?F1 IS ?(s).
```

```

Reuses ``compare`` verbatim ? learned variants stay LOVO-honest, the C1+C4 honest stats
come for free."""
ablated = _ablated_variant(cfg.full, signal.columns, signal.gate_knob)
res = experiment.compare(videos, ablated, cfg.full, iou=cfg.iou, honest=cfg.honest,
                        honest_stats=True)

honest = res["honest"]
# Re-pair the per-video ?F1 with the configured bootstrap seed for determinism (the default
# honest block uses seed=0; thread cfg.seed explicitly so a run is reproducible).
from backend.eval.eval_stats import paired_delta_ci
by_a = {p["video"]: p["f1"] for p in res["variant_a"]["per_video"]}
by_b = {p["video"]: p["f1"] for p in res["variant_b"]["per_video"]}
vids = [p["video"] for p in res["variant_a"]["per_video"]]
pd = paired_delta_ci([by_a[v] for v in vids], [by_b[v] for v in vids], seed=cfg.seed)
st = pd["sign_test"]
sign_wins, sign_losses = int(st["wins"]), int(st["losses"])
sign_p = float(st["p_value"])
verdict, protected = _classify(signal.structural, bool(pd["ci_excludes_zero"]),
                              sign_wins, sign_losses, sign_p, cfg.p_threshold)

seg_full = res["honest"]["segmentation"]["variant_b"]["segment_count_ratio"]
seg_abl = res["honest"]["segmentation"]["variant_a"]["segment_count_ratio"]

return AblationRow(
    signal=signal.name,
    group=signal.group,
    structural=signal.structural,
    dropped_columns=list(signal.columns),
    gate_knob=signal.gate_knob,
    n=int(pd["n"]),
    delta_f1=float(pd["mean_delta"]),
    ci_low=float(pd["ci_low"]),
    ci_high=float(pd["ci_high"]),
    ci_excludes_zero=bool(pd["ci_excludes_zero"]),
    sign_wins=sign_wins,
    sign_losses=sign_losses,
    sign_p_value=sign_p,
    f1_full=float(res["variant_b"]["aggregate"]["f1"]),
    f1_ablated=float(res["variant_a"]["aggregate"]["f1"]),
    seg_ratio_full=float(seg_full),
    seg_ratio_ablated=float(seg_abl),
    verdict=verdict,
    structural_protected=protected,
)

```

```

def _collapse_to_groups(signals: Sequence[Signal]) -> List[Signal]:
    """Collapse a LOSO inventory into one Signal per producer GROUP (LOGO): union the columns,
    OR the gate knob, OR the structural flag. First-seen group order is preserved."""
    order: List[str] = []
    by_group: Dict[str, List[Signal]] = {}
    for s in signals:
        if s.group not in by_group:
            order.append(s.group)
            by_group[s.group] = []
        by_group[s.group].append(s)
    out: List[Signal] = []
    for g in order:
        members = by_group[g]
        cols: Tuple[str, ...] = tuple(dict.fromkeys(c for m in members for c in m.columns))
        knob = next((m.gate_knob for m in members if m.gate_knob), None)
        structural = any(m.structural for m in members)
        out.append(Signal(name=g, group=g, columns=cols, gate_knob=knob, structural=structural))
    return out

```

```

def run_ablation(videos: Sequence[VideoCandidates], cfg: AblationConfig) -> AblationReport:
    """Loop the inventory at the configured granularity ? an ``AblationReport`` (sorted table).

    A pure driver: each row is one ``experiment.compare(without_s, full)``.
    ``granularity="group"``
    (LOGO) ablates a whole producer's columns at once; ``"signal"`` (LOSO) one column-set at a
    time. The full reference is scored once implicitly per row (cheap re-scoring; detection is
    already persisted)."""
    if len(videos) < 1:
        raise experiment.ExperimentError("run_ablation needs at least one labeled video")
    if cfg.granularity not in ("signal", "group"):
        raise ValueError(f"granularity must be 'signal' or 'group', got {cfg.granularity!r}")

    inventory = cfg.signals if cfg.granularity == "signal" else _collapse_to_groups(cfg.signals)
    rows = [ablate_one(videos, cfg, s) for s in inventory]

    # Provenance: a full-set evaluation gives the honest-LOVO flag + the label-set fingerprint.
    full_eval = experiment.evaluate_variant(videos, cfg.full, iou=cfg.iou, honest=cfg.honest)
    from backend.eval.regression import gt_hash
    all_gts = [iv for vc in videos for iv in (vc.gts or [])]
    return AblationReport(
        granularity=cfg.granularity,
        iou=cfg.iou,
        num_videos=len(videos),
        honest_lovo=bool(full_eval["honest_lovo"]),
        full_set_signals=[s.group for s in _collapse_to_groups(cfg.signals)],
        full_spec_hash=cfg.full.spec_hash(),
        label_set_hash=gt_hash(all_gts),
        rows=rows,
    )

```

----- default inventories

```

def default_signals() -> List[Signal]:
    """The current 4-group inventory (LOSO granularity): shuttle (velocity block + duration),
    gate_a (the structural net-crossings gate AND its column), person, audio.

    Gate-A is marked ``structural`` (the held-shuttle FP guard) so it is reported but never
    flagged dead weight on "no measured lift". It ablates via the ``min_crossings`` knob AND by
    dropping the ``net_crossings`` column, so the leave-one-out genuinely removes the whole
    cue."""
    return [
        Signal("shuttle.duration", "shuttle", ("duration",)),
        Signal("shuttle.velocity", "shuttle", ("peak_velocity", "mean_velocity",
"active_ratio")),
        Signal("gate_a", "gate_a", ("net_crossings",), gate_knob="min_crossings",
structural=True),
        Signal("person", "person", PERSON_COLUMNS),
        Signal("audio", "audio", AUDIO_COLUMNS),
    ]

def static_gate_a_signal() -> Signal:
    """The Gate-A-only structural signal (a pure gate knob over the recall-oriented proposal
    set). Ablating it against the static ``Variant(min_crossings=2)`` full reproduces the
    hand-measured Gate-A result (?F1 ? +0.074, 6/6, p ? 0.031) ? the framework's litmus test."""
    return Signal("gate_a", "gate_a", (), gate_knob="min_crossings", structural=True)

def litmus_config() -> AblationConfig:
    """The Gate-A reproduction config: full = STATIC ``Variant(min_crossings=2)`` (no learned
    classifier), inventory = the single Gate-A gate signal. Ablating it = CONTROL+gate vs

```

CONTROL

```
? exactly the historical measurement (`scratch/gate_ab_ablation.py`)."
full = experiment.Variant(name="control_gate_a", min_crossings=2)
return AblationConfig(full=full, signals=[static_gate_a_signal()], granularity="signal")
```

----- corpus I/O

```
def load_videos(features_dir: str) -> List[VideoCandidates]:
    """Load every ``*_golden_features.json`` into a ``VideoCandidates`` (intervals + shuttle +
    person + audio columns + golden gts). Audio columns are optional (older JSONs predate the
    audio augment) ? a missing block defaults those columns to 0.0 so the loader never crashes,
    mirroring ``experiment._feature_column``'s tolerance."""
    videos: List[VideoCandidates] = []
    for path in sorted(glob.glob(os.path.join(features_dir, "*_golden_features.json"))):
        with open(path, encoding="utf-8") as f:
            d = json.load(f)
            cands = d["candidates"]
            intervals = [(float(c["start"]), float(c["end"])) for c in cands]
            features: Dict[str, List[float]] = {}
            for fn in SHUTTLE_COLUMNS:
                features[fn] = [float(c["shuttle"][fn]) for c in cands]
            for fn in PERSON_COLUMNS:
                features[fn] = [float(c["person"][fn]) for c in cands]
            for fn in AUDIO_COLUMNS:
                features[fn] = [float((c.get("audio") or {}).get(fn, 0.0)) for c in cands]
            gts = [(float(a), float(b)) for a, b in d["gts"]]
            videos.append(VideoCandidates(name=d["name"], intervals=intervals,
                                         features=features, gts=gts))
    return videos

def learned_full_variant(*, threshold: float = 0.5, min_crossings: int = 0) -> Variant:
    """The learned full-set reference: all 13 columns (shuttle 5 + person 5 + audio 3) weighted
    by the LOVO logistic scorer. Used for the LOSO/LOGO contribution table over the learned path
    (distinct from the static Gate-A litmus)."""
    cols = list(SHUTTLE_COLUMNS) + list(PERSON_COLUMNS) + list(AUDIO_COLUMNS)
    return experiment.Variant(name="full_learned", feature_names=cols,
                              threshold=threshold, min_crossings=min_crossings)
```

----- table rendering

```
def format_table(report: AblationReport) -> str:
    """Sorted ASCII/markdown ablation table (by marginal ?F1 desc ? the contribution
    ranking)."""
    rows = report.sorted_rows()
    header = (f"Ablation report ? full set = {{{', '.join(report.full_set_signals)}}}"
              f" . n={report.num_videos} . IoU {report.iou}"
              f" . {'LOVO-honest' if report.honest_lovo else 'static'}"
              f" . {report.granularity.upper()}")
    cols = ["signal", "?F1(full?ablated)", "95% CI", "sign test", "CI?0", "verdict"]
    widths = [18, 17, 18, 14, 5, 22]

    def _fmt_row(cells: Sequence[str]) -> str:
        return " ".join(c.ljust(w) for c, w in zip(cells, widths))

    lines = [header, "", _fmt_row(cols), _fmt_row(["-" * w for w in widths])]
    for r in rows:
        ci = f"[{r.ci_low:+.3f}, {r.ci_high:+.3f}]"
        sign = f"{r.sign_wins}W/{r.sign_losses}L p={r.sign_p_value:.3f}"
```

```

        verdict = VERDICT_LABEL.get(r.verdict, r.verdict)
        lines.append(_fmt_row([r.signal, f"{r.delta_f1:+.3f}", ci, sign,
                                "yes" if r.ci_excludes_zero else "no", verdict]))
    return "\n".join(lines)

```

----- PDF export

```

def export_pdf(report: AblationReport, path: str) -> str:
    """Render the ``AblationReport`` to a clean reportlab PDF: a table of each signal's marginal
    contribution + CI + sign test + a verdict column (clears-bar / suggestive / inert /
    structural-protected). Reuses the muted booktabs aesthetic of ``scratch/quality_report.py``.

    Returns the written path. ``reportlab`` is an offline, pure-Python dependency (no GPU)."""
    from reportlab.lib import colors
    from reportlab.lib.enums import TA_CENTER, TA_LEFT
    from reportlab.lib.pagesizes import A4
    from reportlab.lib.styles import ParagraphStyle, getSampleStyleSheet
    from reportlab.lib.units import cm
    from reportlab.platypus import (Paragraph, SimpleDocTemplate, Spacer, Table, TableStyle)

    navy = colors.HexColor("#1F3A5F")
    black = colors.HexColor("#000000")
    darkgrey = colors.HexColor("#333333")
    light = colors.HexColor("#F0F0F0")
    hairline = colors.HexColor("#BFBFBF")
    row_alt = colors.HexColor("#F7F7F7")

    base = getSampleStyleSheet()["Normal"]
    title = ParagraphStyle("Title2", parent=base, fontName="Helvetica-Bold", fontSize=16,
                           leading=20, alignment=TA_CENTER, textColor=navy, spaceAfter=4)
    subtitle = ParagraphStyle("Sub", parent=base, fontName="Helvetica-Oblique", fontSize=9.5,
                              leading=13, alignment=TA_CENTER, textColor=darkgrey,
spaceAfter=10)
    h2 = ParagraphStyle("H2", parent=base, fontName="Helvetica-Bold", fontSize=11, leading=15,
                        textColor=navy, spaceBefore=10, spaceAfter=4)
    body = ParagraphStyle("Body", parent=base, fontName="Helvetica", fontSize=9, leading=12.5,
                          alignment=TA_LEFT, textColor=black, spaceAfter=5)
    cell = ParagraphStyle("Cell", parent=base, fontName="Helvetica", fontSize=8.2, leading=10.5,
                          textColor=black)
    cell_b = ParagraphStyle("CellB", parent=base, fontName="Helvetica-Bold", fontSize=8.2,
                            leading=10.5, textColor=black)
    head = ParagraphStyle("Head", parent=base, fontName="Helvetica-Bold", fontSize=8.2,
                          leading=10.5, textColor=black)

    rows = report.sorted_rows()
    story: List[Any] = [
        Paragraph("Signal Ablation & Feature-Enablement Report", title),
        Paragraph(
            f"Leave-one-{'group' if report.granularity == 'group' else 'signal'}-out marginal "
            f"contribution & n={report.num_videos} golden matches, IoU {report.iou}, "
            f"{'honest LOVO' if report.honest_lovo else 'static re-score'}. "
            "Honest measured results only.",
            subtitle),
    ]

    full_set = ", ".join(report.full_set_signals)
    story.append(Paragraph("Run provenance", h2))
    story.append(Paragraph(
        f"Full signal set: <b>{{{full_set}}}</b>. Variant spec_hash "
        f"<font face='Courier'>{report.full_spec_hash}</font>, label-set hash "
        f"<font face='Courier'>{report.label_set_hash}</font>. Each row is the held-out "
        "&Delta;F1 of removing one signal from the full set (full &minus; ablated), with a "

```

```

        "bootstrap 95% CI and an exact paired sign test &mdash; a signal earns its keep only "
        "when removing it drops F1 with the CI clearing 0 <i>and</i> the sign test agreeing.",
        body))

story.append(Paragraph("Marginal contribution per signal", h2))
table_data: List[List[Any]] = [[
    Paragraph("Signal", head), Paragraph("Group", head),
    Paragraph("&Delta;F1 (full&minus;abl.)", head), Paragraph("95% CI", head),
    Paragraph("Sign test", head), Paragraph("CI&ne;0", head),
    Paragraph("Over-seg", head), Paragraph("Verdict", head),
]]
for r in rows:
    ci = f"[{r.ci_low:+.3f}, {r.ci_high:+.3f}]"
    sign = f"{r.sign_wins}W/{r.sign_losses}L p={r.sign_p_value:.3f}"
    overseg = f"{r.seg_ratio_full:.2f}&rarr;{r.seg_ratio_ablated:.2f}"
    verdict = VERDICT_LABEL.get(r.verdict, r.verdict)
    df1_style = cell_b if r.verdict == EARNNS_KEEP else cell
    table_data.append([
        Paragraph(r.signal, cell), Paragraph(r.group, cell),
        Paragraph(f"{r.delta_f1:+.3f}", df1_style), Paragraph(ci, cell),
        Paragraph(sign, cell), Paragraph("yes" if r.ci_excludes_zero else "no", cell),
        Paragraph(overseg, cell), Paragraph(verdict, cell),
    ])
col_widths = [2.7 * cm, 1.7 * cm, 2.4 * cm, 2.7 * cm, 2.6 * cm, 1.1 * cm, 1.9 * cm, 2.6 *
cm]
tbl = Table(table_data, colWidths=col_widths, repeatRows=1)
last = len(table_data) - 1
tbl.setStyle(TableStyle([
    ("BACKGROUND", (0, 0), (-1, 0), light),
    ("VALIGN", (0, 0), (-1, -1), "MIDDLE"),
    ("TOPPADDING", (0, 0), (-1, -1), 4),
    ("BOTTOMPADDING", (0, 0), (-1, -1), 4),
    ("LEFTPADDING", (0, 0), (-1, -1), 4),
    ("RIGHTPADDING", (0, 0), (-1, -1), 4),
    ("LINEABOVE", (0, 0), (-1, 0), 1.0, black),
    ("LINEBELOW", (0, 0), (-1, 0), 0.8, black),
    ("LINEBELOW", (0, 1), (-1, max(1, last - 1)), 0.3, hairline),
    ("LINEBELOW", (0, last), (-1, last), 1.0, black),
    ("ROWBACKGROUNDS", (0, 1), (-1, -1), [colors.white, row_alt]),
]))
story.append(tbl)
story.append(Spacer(1, 0.3 * cm))

story.append(Paragraph("Verdict key", h2))
story.append(Paragraph(
    "<b>clears-bar</b> &mdash; removing the signal drops F1 with the &Delta;F1 CI clearing 0
"
    "and the sign test agreeing (the signal earns its keep). "
    "<b>suggestive</b> &mdash; the point estimate leans toward 'removal hurts' but the CI "
    "spans 0 (keep, watch as the corpus grows). "
    "<b>inert</b> &mdash; removal is a wash this run; a candidate for retirement only after
a "
    "trend across multiple runs, never on one wash at small n. "
    "<b>structural-protected</b> &mdash; a guard whose value is in failure modes the golden
"
    "corpus may lack (e.g. Gate-A vs held-shuttle false positives); reported honestly but "
    "never auto-flagged dead weight on 'no measured lift'.",
    body))

doc = SimpleDocTemplate(path, pagesize=A4, leftMargin=2 * cm, rightMargin=2 * cm,
    topMargin=1.8 * cm, bottomMargin=1.8 * cm,
    title="Signal Ablation & Feature-Enablement Report",
    author="badminton-highlight-indexer")
doc.build(story)
logger.info("wrote ablation PDF -> %s", path)

```

```
return path
```

----- CLI

```
def main() -> None:
    ap = argparse.ArgumentParser(
        description="LOSO/LOGO ablation runner over the golden feature substrate (pure driver "
        "over experiment.compare + eval_stats; no new scoring math).")
    ap.add_argument("--features-dir", default="output",
        help="dir of *_golden_features.json (the persisted golden substrate)")
    ap.add_argument("--granularity", choices=("signal", "group"), default="group",
        help="leave-one-SIGNAL-out (column) or leave-one-GROUP-out (producer)")
    ap.add_argument("--iou", type=float, default=0.5)
    ap.add_argument("--seed", type=int, default=0, help="bootstrap seed (determinism)")
    ap.add_argument("--litmus", action="store_true",
        help="reproduce the static Gate-A measurement (CONTROL vs min_crossings=2) "
        "instead of the learned full-set ablation")
    ap.add_argument("--pdf", default=None, help="also write a PDF ablation report to this path")
    ap.add_argument("--json", default=None, help="also write the structured report JSON here")
    args = ap.parse_args()

    videos = load_videos(args.features_dir)
    if len(videos) < 2:
        raise SystemExit(f"need >=2 *_golden_features.json in {args.features_dir} "
            f"(found {len(videos)})")

    if args.litmus:
        cfg = litmus_config()
        cfg.iou, cfg.seed = args.iou, args.seed
    else:
        cfg = AblationConfig(full=learned_full_variant(), signals=default_signals(),
            granularity=args.granularity, iou=args.iou, seed=args.seed)

    report = run_ablation(videos, cfg)
    print(format_table(report))

    if args.json:
        with open(args.json, "w", encoding="utf-8") as f:
            json.dump(report.as_dict(), f, indent=2)
        print(f"\nwrote report JSON -> {args.json}")
    if args.pdf:
        export_pdf(report, args.pdf)
        print(f"wrote PDF report -> {args.pdf}")

if __name__ == "__main__":
    main()
```

53. user (2026-06-14T10:14:51.236Z)

File created successfully at: C:\Users\avidu\Projects\bhi-ablation\backend\eval\ablation.py
(file state is current in your context ? no need to Read it back)

54. assistant (2026-06-14T10:14:59.153Z)

Now let me run the litmus reproduction against the real golden JSONs from the main repo's output/ dir to confirm the runner reproduces +0.074. I'll copy the module path into the main repo's PYTHONPATH context ? actually, the cleanest is to run the worktree module but point `--features-dir` at the main repo's output. Let me test the litmus directly.

55. user (2026-06-14T10:15:02.971Z)

Ablation report ? full set = {gate_a} · n=6 · IoU 0.5 · static · SIGNAL

signal	?F1(full?ablated)	95% CI	sign test	CI?0	verdict
gate_a	+0.074	[+0.042, +0.105]	6W/0L p=0.031	yes	structural-protected

56. assistant (2026-06-14T10:15:07.880Z)

****LITMUS PASSES.**** The runner reproduces Gate-A exactly:
- ?F1 = ****+0.074**** ? (target ~+0.074)
- CI = ****[+0.042, +0.105]**** ? (target [+0.042, +0.104])
- sign ****6W/0L, p=0.031**** ?
- verdict = structural-protected (Gate-A is structural, so reported but never demoted) ?

Now let me run the full LOGO and LOSO ablation tables over the learned full set, and generate a PDF, to confirm the full path works.

57. user (2026-06-14T10:15:18.528Z)

=== LOGO (group) ===
Ablation report ? full set = {shuttle, gate_a, person, audio} · n=6 · IoU 0.5 · LOV0-honest · GROUP

signal	?F1(full?ablated)	95% CI	sign test	CI?0	verdict
audio	+0.079	[-0.054, +0.203]	4W/2L p=0.688	no	suggestive
shuttle	+0.002	[-0.013, +0.019]	2W/2L p=1.000	no	inert
person	-0.009	[-0.074, +0.055]	2W/4L p=0.688	no	inert
gate_a	-0.014	[-0.083, +0.027]	4W/1L p=0.375	no	structural-protected

=== LOSO (signal) ===
Ablation report ? full set = {shuttle, gate_a, person, audio} · n=6 · IoU 0.5 · LOV0-honest · SIGNAL

signal	?F1(full?ablated)	95% CI	sign test	CI?0	verdict
audio	+0.079	[-0.054, +0.203]	4W/2L p=0.688	no	suggestive
shuttle.velocity	+0.002	[-0.010, +0.018]	2W/3L p=1.000	no	inert
shuttle.duration	-0.001	[-0.004, +0.000]	0W/2L p=0.500	no	inert
person	-0.009	[-0.074, +0.055]	2W/4L p=0.688	no	inert
gate_a	-0.014	[-0.083, +0.027]	4W/1L p=0.375	no	structural-protected

wrote PDF report -> C:/Users/avidu/Projects/bhi-ablation/scratch_ablation_sample.pdf

58. assistant (2026-06-14T10:15:24.698Z)

Both LOGO and LOSO work, and the PDF is generated. The results are coherent with QUALITY_ITERATIONS (audio is the suggestive lead at +0.079; person and shuttle are washes in the learned full set). Note: in the ****learned**** full set, gate_a's `net\_crossings` column is largely subsumed by the learned scorer, so its marginal is ?0 ? but it's `structural-protected` so it's never flagged dead weight. This is exactly the design.

Let me verify the PDF is valid (page count > 0) with pypdf, then clean up the scratch PDF.

59. user (2026-06-14T10:15:28.085Z)

pages: 1
bytes: 4087
contains 'Ablation': True

```
contains 'clears-bar': True
contains 'structural-protected': True
contains 'audio': True
contains 'gate_a': True
```

60. assistant (2026-06-14T10:15:34.143Z)

PDF is valid (1 page, 4KB, contains all expected content). Now let me write the comprehensive test suite. I'll build deterministic synthetic candidates where I control the ground truth, plus the golden litmus reproduction (skip-if-absent), structural-guard exemption, honest-stats fields, and a valid PDF.

Let me first verify a synthetic construction where a known signal cleanly earns its keep ? I'll design videos where a feature column perfectly separates rallies so dropping it tanks F1 (earns_keep), and another where it's noise (inert). Let me prototype the synthetic data behavior.

61. user (2026-06-14T10:15:44.158Z)

```
non-structural gate row:
  dF1 0.333 ci 0.333 0.333 wins 6 losses 0 p 0.031
  verdict earns_keep protected False
  seg full 1.0 abl 2.0
structural verdict: structural_protected protected True (dF1 same: 0.333 )
```

62. assistant (2026-06-14T10:15:51.191Z)

The synthetic deterministically produces:

- **earns_keep** when non-structural and the gate cleanly separates (?F1 +0.333, 6W/0L, p=0.031, CI clears 0).
- **structural_protected** for the identical signal marked structural (the exemption works ? same ?F1, but never demoted/promoted to earns_keep).
- The over-seg ratio drops 2.0 ? 1.0 when the gate is on (full).

Now let me write the test suite.

63. assistant (2026-06-14T10:17:03.938Z)

```
"""GPU-free tests for the LOSO/LOGO ablation runner (`backend.eval.ablation`).
```

Covers, all on deterministic synthetic candidate rows (no GPU, no real video):

- LOSO / LOGO correctness ? a cleanly-separating signal *earns its keep*, a noise signal is *inert*; LOGO collapses a group's columns; the ablated variant subtracts columns / zeroes the gate knob without mutating `full`.
- the structural-guard exemption ? the SAME signal marked `structural=True` is reported (honest ?) but verdict is always `structural_protected`, never auto-demoted on "no lift".
- the honest-stats contract ? every row ships a CI + sign test, and no verdict is stronger than the stats support (the no-over-claim contract).
- a valid non-trivial PDF is produced (asserted via pypdf page count > 0).
- the golden litmus reproduction ? on the 6 real `output/*_golden_features.json` the runner reproduces the hand-measured Gate-A result (?F1 ? +0.074, 6/6 wins, p ? 0.031). Skips cleanly when the golden JSONs are absent (CI), runs when present (owner box).

```
"""
```

```
from __future__ import annotations
```

```
import glob
```

```
import json
```

```
import os
```

```
import pytest
```

```
from backend.eval import ablation
from backend.eval.ablation import (
```

```

    AblationConfig,
    AblationReport,
    Signal,
    ablate_one,
    run_ablation,
)
from backend.eval.experiment import Variant, VideoCandidates

```

----- synthetic fixtures

```

def _gate_videos(n: int = 6, *, separating: bool = True) -> list[VideoCandidates]:
    """`n` videos with a `net_crossings` gate signal.

    `separating=True`: rallies have net_crossings=5, junk has 0 ? so a `min_crossings=2`
gate
    perfectly removes the junk and keeping it is load-bearing. `separating=False`: every
window
    has net_crossings=5, so the gate is a no-op (ablating it changes nothing ? inert)."""
    videos: list[VideoCandidates] = []
    for k in range(n):
        intervals = [(0.0, 10.0), (20.0, 30.0), (40.0, 41.0), (50.0, 51.0)]
        junk_val = 5.0 if not separating else 0.0
        feats = {"net_crossings": [5.0, 5.0, junk_val, junk_val]}
        gts = [(0.0, 10.0), (20.0, 30.0)]
        videos.append(VideoCandidates(name=f"v{k}", intervals=intervals, features=feats,
gts=gts))
    return videos

def _learned_videos(n: int = 6) -> list[VideoCandidates]:
    """`n` videos where a PERSON feature (`players_in_court`) cleanly separates rallies from
junk while the shuttle/audio columns are constant (non-separating). So a learned full-set
ablation must show person as the load-bearing group and the others as washes ? a
deterministic
    LOGO/LOSO correctness fixture (mirrors `test_fusion_compare`'s construction)."""
    videos: list[VideoCandidates] = []
    for k in range(n):
        intervals = [(0.0, 10.0), (20.0, 30.0), (40.0, 41.0), (50.0, 51.0)]
        labels = [1, 1, 0, 0]
        feats: dict[str, list[float]] = {}
        for fn in ablation.SHUTTLE_COLUMNS:
            feats[fn] = [1.0, 1.0, 1.0, 1.0] # constant ? non-separating
        for fn in ablation.PERSON_COLUMNS:
            feats[fn] = [0.0, 0.0, 0.0, 0.0]
        feats["players_in_court"] = [4.0, 4.0, 0.0, 0.0] # the separating signal
        feats["in_court_ratio"] = [1.0, 1.0, 0.0, 0.0]
        for fn in ablation.AUDIO_COLUMNS:
            feats[fn] = [0.0, 0.0, 0.0, 0.0]
        gts = [(0.0, 10.0), (20.0, 30.0)]
        videos.append(VideoCandidates(name=f"v{k}", intervals=intervals, features=feats,
gts=gts,
                                ))
        videos[-1].gts = gts
        _ = labels
    return videos

```

----- LOSO correctness

```

def test_separating_gate_earns_keep():

```

```

"""A non-structural gate that cleanly separates rallies must EARN ITS KEEP: removing it
drops F1 with the ?F1 CI clearing 0 and the sign test agreeing (6W/0L)."""
videos = _gate_videos(6, separating=True)
sig = Signal("gate_x", "gate_x", (), gate_knob="min_crossings", structural=False)
cfg = AblationConfig(full=Variant(name="full", min_crossings=2), signals=[sig],
                      granularity="signal")
row = ablate_one(videos, cfg, sig)
assert row.delta_f1 > 0.0
assert row.ci_excludes_zero is True
assert row.sign_wins == 6 and row.sign_losses == 0
assert row.sign_p_value <= 0.05
assert row.verdict == ablation.EARNS_KEEP
assert row.structural_protected is False
# C4: the gate cuts over-segmentation (more junk segments without it).
assert row.seg_ratio_full < row.seg_ratio_ablated

def test_noop_gate_is_inert():
    """A gate whose threshold removes nothing (all windows pass) is a wash ? INERT, never a
    stronger claim. Honest: a single wash at n=6 is a candidate, not 'proven useless'."""
    videos = _gate_videos(6, separating=False)
    sig = Signal("gate_x", "gate_x", (), gate_knob="min_crossings", structural=False)
    cfg = AblationConfig(full=Variant(name="full", min_crossings=2), signals=[sig],
                          granularity="signal")
    row = ablate_one(videos, cfg, sig)
    assert abs(row.delta_f1) < 1e-9
    assert row.ci_excludes_zero is False
    assert row.verdict == ablation.INERT

def test_ablated_variant_does_not_mutate_full():
    """Building the ablated variant must derive a new Variant, never mutate ``full`` in
    place."""
    full = Variant(name="full", feature_names=["a", "b", "c"], min_crossings=2)
    before = full.to_dict()
    ablated = ablation._ablated_variant(full, ("b",), "min_crossings")
    assert ablated.feature_names == ["a", "c"]
    assert ablated.min_crossings == 0
    assert full.to_dict() == before # full untouched
    assert full.feature_names == ["a", "b", "c"]

def test_unknown_gate_knob_raises():
    full = Variant(name="full", feature_names=["a"])
    with pytest.raises(ValueError):
        ablation._ablated_variant(full, (), "min_bogus")

```

----- LOGO correctness

```

def test_logo_collapses_groups():
    """LOGO (`granularity='group'`) ablates a whole producer's columns at once: the default
    5-signal LOSO inventory collapses to 4 producer groups, and the union of columns is
    dropped."""
    groups = ablation._collapse_to_groups(ablation.default_signals())
    names = [g.name for g in groups]
    assert names == ["shuttle", "gate_a", "person", "audio"]
    shuttle = next(g for g in groups if g.name == "shuttle")
    # the shuttle group unions duration + the velocity block
    assert set(shuttle.columns) == {"duration", "peak_velocity", "mean_velocity",
    "active_ratio"}
    gate = next(g for g in groups if g.name == "gate_a")
    assert gate.gate_knob == "min_crossings"

```

```

assert gate.structural is True                                # structural flag OR-ed up

def test_logo_finds_separating_group():
    """On the learned fixture where PERSON cleanly separates, the LOGO person row must be the
    load-bearing one (largest positive marginal ?F1) and the constant groups must be ~washes."""
    videos = _learned_videos(6)
    cfg = AblationConfig(full=ablation.learned_full_variant(),
signals=ablation.default_signals(),
                        granularity="group")
    report = run_ablation(videos, cfg)
    rows = {r.signal: r for r in report.rows}
    assert set(rows) == {"shuttle", "gate_a", "person", "audio"}
    top = report.sorted_rows()[0]
    assert top.signal == "person"
    assert rows["person"].delta_f1 > 0.0
    # the constant audio group carries no information ? marginal ? 0
    assert abs(rows["audio"].delta_f1) < 0.05

def test_run_ablation_both_granularities():
    """LOSO yields one row per signal; LOGO yields one per group; both honest-LOVO on a learned
    full set."""
    videos = _learned_videos(6)
    loso = run_ablation(videos, AblationConfig(full=ablation.learned_full_variant(),
signals=ablation.default_signals(),
                        granularity="signal"))
    logo = run_ablation(videos, AblationConfig(full=ablation.learned_full_variant(),
signals=ablation.default_signals(),
                        granularity="group"))
    assert len(loso.rows) == len(ablation.default_signals()) # 5 LOSO rows
    assert len(logo.rows) == 4 # 4 producer groups
    assert loso.honest_lovo is True and logo.honest_lovo is True

def test_run_ablation_rejects_bad_granularity():
    videos = _learned_videos(2)
    cfg = AblationConfig(full=ablation.learned_full_variant(),
signals=ablation.default_signals(),
                        granularity="bogus")
    with pytest.raises(ValueError):
        run_ablation(videos, cfg)

def test_run_ablation_needs_a_video():
    cfg = AblationConfig(full=ablation.learned_full_variant(),
signals=ablation.default_signals())
    from backend.eval.experiment import ExperimentError
    with pytest.raises(ExperimentError):
        run_ablation([], cfg)

```

----- structural-guard exemption

```

def test_structural_guard_never_demoted():
    """A structural guard with the SAME measured ? as a non-structural one that EARNs its keep
    must report verdict ``structural_protected`` (never auto-demoted), but record the honest
    ?."""
    videos = _gate_videos(6, separating=True)
    non = Signal("gate_x", "gate_x", (), gate_knob="min_crossings", structural=False)
    struct = Signal("gate_x", "gate_x", (), gate_knob="min_crossings", structural=True)
    full = Variant(name="full", min_crossings=2)
    cfg = AblationConfig(full=full, signals=[non], granularity="signal")

```

```

r_non = ablate_one(videos, cfg, non)
r_struct = ablate_one(videos, cfg, struct)
# identical honest measurement?
assert pytest.approx(r_non.delta_f1) == r_struct.delta_f1
assert (r_non.sign_wins, r_non.sign_losses) == (r_struct.sign_wins, r_struct.sign_losses)
# ?but the verdict differs: non-structural earns keep, structural is protected.
assert r_non.verdict == ablation.EARNS_KEEP
assert r_struct.verdict == ablation.STRUCTURAL_PROTECTED
assert r_struct.structural_protected is True

def test_structural_guard_protected_even_when_inert():
    """A structural guard that is a measured wash is STILL reported as ``structural_protected``
    (never flagged dead weight on 'no measured lift')."""
    videos = _gate_videos(6, separating=False)
    struct = Signal("gate_x", "gate_x", (), gate_knob="min_crossings", structural=True)
    cfg = AblationConfig(full=Variant(name="full", min_crossings=2), signals=[struct],
                          granularity="signal")
    row = ablate_one(videos, cfg, struct)
    assert abs(row.delta_f1) < 1e-9 # honestly a wash
    assert row.verdict == ablation.STRUCTURAL_PROTECTED
    assert row.structural_protected is True

def test_default_gate_a_is_structural():
    """The shipped Gate-A signal must carry ``structural=True`` (the held-shuttle FP guard)."""
    sig = next(s for s in ablation.default_signals() if s.group == "gate_a")
    assert sig.structural is True
    assert sig.gate_knob == "min_crossings"

```

----- honest-stats fields

```

def test_row_carries_full_honest_evidence():
    """Every row ships the honest evidence: n, CI bounds, the sign test, and a verdict in the
    closed vocabulary (no over-claim)."""
    videos = _learned_videos(6)
    cfg = AblationConfig(full=ablation.learned_full_variant(),
                          signals=ablation.default_signals(),
                          granularity="group")
    report = run_ablation(videos, cfg)
    valid = {ablation.EARNS_KEEP, ablation.SUGGESTIVE, ablation.INERT,
             ablation.STRUCTURAL_PROTECTED}
    for r in report.rows:
        d = r.as_dict()
        assert {"delta_f1", "ci_low", "ci_high", "ci_excludes_zero", "sign_test", "n",
                "verdict"} <= set(d)
        assert d["n"] == 6
        assert d["sign_test"]["wins"] + d["sign_test"]["losses"] <= 6
        # CI brackets the mean ( $\pm$ fp) ? never a bare mean detached from its interval.
        assert r.ci_low - 1e-9 <= r.delta_f1 <= r.ci_high + 1e-9
        assert r.verdict in valid

def test_no_over_claim_vocabulary():
    """The verdict vocabulary must NOT contain 'irrelevant'/'useless' (attribution contract)."""
    forbidden = {"irrelevant", "useless", "dead", "remove"}
    for v in ablation.VERDICT_LABEL.values():
        assert not any(bad in v.lower() for bad in forbidden)

def test_report_round_trips_json():
    """The structured report is JSON-able and sorted by ?F1 desc (contribution ranking)."""

```

```

videos = _learned_videos(6)
report = run_ablation(videos, AblationConfig(full=ablation.learned_full_variant(),
                                             signals=ablation.default_signals(),
                                             granularity="group"))

blob = json.dumps(report.as_dict())          # must not raise
back = json.loads(blob)
deltas = [r["delta_f1"] for r in back["rows"]]
assert deltas == sorted(deltas, reverse=True)  # sorted desc
assert back["num_videos"] == 6
assert back["label_set_hash"]                # provenance present

def test_seed_is_deterministic():
    """Two runs with the same seed produce identical CIs (deterministic bootstrap)."""
    videos = _learned_videos(6)
    cfg = AblationConfig(full=ablation.learned_full_variant(),
                         signals=ablation.default_signals(),
                         granularity="group", seed=7)
    r1 = run_ablation(videos, cfg)
    r2 = run_ablation(videos, cfg)
    assert [r.ci_low for r in r1.rows] == [r.ci_low for r in r2.rows]
    assert [r.ci_high for r in r1.rows] == [r.ci_high for r in r2.rows]

```

----- table + PDF

```

def test_format_table_is_sorted_and_readable():
    videos = _learned_videos(6)
    report = run_ablation(videos, AblationConfig(full=ablation.learned_full_variant(),
                                                  signals=ablation.default_signals(),
                                                  granularity="group"))

    out = ablation.format_table(report)
    assert "Ablation report" in out
    assert "?F1(full?ablated)" in out
    assert "verdict" in out
    # the highest-?F1 signal name appears before the lowest in the rendered order
    top = report.sorted_rows()[0].signal
    bottom = report.sorted_rows()[-1].signal
    assert out.index(top) < out.index(bottom)

def test_export_pdf_is_valid_nontrivial(tmp_path):
    """A valid PDF with at least one page is produced, and it contains the verdict legend."""
    pypdf = pytest.importorskip("pypdf")
    videos = _learned_videos(6)
    report = run_ablation(videos, AblationConfig(full=ablation.learned_full_variant(),
                                                  signals=ablation.default_signals(),
                                                  granularity="group"))

    out = str(tmp_path / "ablation.pdf")
    ablation.export_pdf(report, out)
    assert os.path.isfile(out)
    assert os.path.getsize(out) > 1000          # non-trivial
    reader = pypdf.PdfReader(out)
    assert len(reader.pages) > 0
    text = "".join(p.extract_text() for p in reader.pages)
    assert "Ablation" in text
    assert "clears-bar" in text                 # the verdict key rendered

```

----- corpus loader (no GPU)

```

def test_load_videos_tolerates_missing_audio(tmp_path):
    """An older feature JSON without an ``audio`` block still loads ? audio columns default to
    0.0 (loader never crashes on a pre-audio JSON)."""
    cand = {"start": 0.0, "end": 10.0,
            "shuttle": {fn: 1.0 for fn in ablation.SHUTTLE_COLUMNS},
            "person": {fn: 0.0 for fn in ablation.PERSON_COLUMNS},
            "label": 1}
    data = {"name": "noaudio", "fps": 30.0, "frame_width": 1920.0,
            "candidates": [cand], "gts": [[0.0, 10.0]]}
    with open(tmp_path / "noaudio_golden_features.json", "w", encoding="utf-8") as f:
        json.dump(data, f)
    videos = ablation.load_videos(str(tmp_path))
    assert len(videos) == 1
    assert videos[0].features["audio_hit_count"] == [0.0]          # defaulted, not crashed

```

----- the golden litmus

```

GOLDEN_DIR = os.path.join(
    os.path.dirname(os.path.dirname(os.path.abspath(__file__))), "output")

def _golden_present() -> bool:
    return len(glob.glob(os.path.join(GOLDEN_DIR, "*_golden_features.json"))) >= 6

@pytest.mark.skipif(not _golden_present(),
                    reason="golden output/*_golden_features.json absent (run fusion_golden
first)")
def test_litmus_reproduces_gate_a():
    """LITMUS: on the 6 real golden feature JSONs the runner reproduces the hand-measured Gate-A
    result ? marginal ? F1 ? +0.074, 6/6 wins, p ? 0.031, CI ? [+0.042, +0.104], within
tolerance.
    This validates the whole framework against the recorded ground truth. Skips cleanly in CI
    where the golden substrate is absent; runs on the owner box where it is present."""
    videos = ablation.load_videos(GOLDEN_DIR)
    assert len(videos) == 6
    cfg = ablation.litmus_config()
    report = run_ablation(videos, cfg)
    assert len(report.rows) == 1
    row = report.rows[0]
    assert row.signal == "gate_a"
    # The recorded QUALITY_ITERATIONS truth (2026-06-14), reproduced within tolerance.
    assert row.delta_f1 == pytest.approx(0.074, abs=0.005)
    assert row.sign_wins == 6 and row.sign_losses == 0
    assert row.sign_p_value == pytest.approx(0.031, abs=0.005)
    assert row.ci_low == pytest.approx(0.042, abs=0.01)
    assert row.ci_high == pytest.approx(0.104, abs=0.01)
    assert row.ci_excludes_zero is True
    # Gate-A is structural ? reported but never auto-promoted to earns_keep.
    assert row.verdict == ablation.STRUCTURAL_PROTECTED
    # C4: over-segmentation drops 1.68 ? 1.01 when Gate-A is on (full).
    assert row.seg_ratio_full == pytest.approx(1.01, abs=0.05)
    assert row.seg_ratio_ablated == pytest.approx(1.68, abs=0.05)

@pytest.mark.skipif(not _golden_present(),
                    reason="golden output/*_golden_features.json absent")
def test_litmus_report_is_well_formed():
    """The litmus report is a proper AblationReport (provenance + a sorted single row)."""
    report = run_ablation(ablation.load_videos(GOLDEN_DIR), ablation.litmus_config())
    assert isinstance(report, AblationReport)
    assert report.num_videos == 6
    assert report.full_set_signals == ["gate_a"]

```

```
assert report.label_set_hash
out = ablation.format_table(report)
assert "gate_a" in out
```

64. user (2026-06-14T10:17:03.954Z)

File created successfully at: C:\Users\avidu\Projects\bhi-ablation\tests\test_ablation.py (file state is current in your context ? no need to Read it back)

65. assistant (2026-06-14T10:17:07.450Z)

I have a leftover `\_\_ = labels` and a redundant `videos[-1].gts = gts` in `\_learned\_videos` that's clumsy. Let me clean that up.

66. user (2026-06-14T10:17:15.211Z)

The file C:\Users\avidu\Projects\bhi-ablation\tests\test_ablation.py has been updated successfully. (file state is current in your context ? no need to Read it back)

67. assistant (2026-06-14T10:17:22.782Z)

Note: the test file uses `list[...]` / `dict[...]` builtins generics in annotations. The repo targets py38 in ruff but uses `from \_\_future\_\_ import annotations` (which I added), so those are fine at runtime. But the CI runs Python 3.13, so runtime is fine regardless. However, the synthetic gate fixture (no `feature\_names`, just a static gate) ? in `ablate\_one`, the `compare` call with static variants works. Let me run the test suite. But first, the golden JSONs are in the MAIN repo's output/, NOT the worktree. The litmus test computes `GOLDEN\_DIR` relative to the test file's parent's parent = the worktree root's `output/`, which won't exist. So the litmus will SKIP in the worktree (correct CI behavior), but I want to actually RUN it to prove reproduction. Let me run tests in the worktree (litmus skips), then run the litmus separately against the main repo's output.

68. user (2026-06-14T10:17:39.378Z)

===== test session starts =====

```
platform win32 -- Python 3.13.13, pytest-9.0.3, pluggy-1.6.0 --
C:\Users\avidu\AppData\Local\Programs\Python\Python313\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\avidu\Projects\bhi-ablation
configfile: pytest.ini
plugins: anyio-4.13.0, cov-7.1.0
collecting ... collected 21 items
```

tests/test_ablation.py::test_separating_gate_earns_keep	PASSED	[4%]
tests/test_ablation.py::test_noop_gate_is_inert	PASSED	[9%]
tests/test_ablation.py::test_ablated_variant_does_not_mutate_full	PASSED	[14%]
tests/test_ablation.py::test_unknown_gate_knob_raises	PASSED	[19%]
tests/test_ablation.py::test_logo_collapses_groups	PASSED	[23%]
tests/test_ablation.py::test_logo_finds_separating_group	PASSED	[28%]
tests/test_ablation.py::test_run_ablation_both_granularities	PASSED	[33%]
tests/test_ablation.py::test_run_ablation_rejects_bad_granularity	PASSED	[38%]
tests/test_ablation.py::test_run_ablation_needs_a_video	PASSED	[42%]
tests/test_ablation.py::test_structural_guard_never_demoted	PASSED	[47%]
tests/test_ablation.py::test_structural_guard_protected_even_when_inert	PASSED	[52%]
tests/test_ablation.py::test_default_gate_a_is_structural	PASSED	[57%]
tests/test_ablation.py::test_row_carries_full_honest_evidence	PASSED	[61%]
tests/test_ablation.py::test_no_over_claim_vocabulary	PASSED	[66%]
tests/test_ablation.py::test_report_round_trips_json	PASSED	[71%]
tests/test_ablation.py::test_seed_is_deterministic	PASSED	[76%]
tests/test_ablation.py::test_format_table_is_sorted_and_readable	PASSED	[80%]
tests/test_ablation.py::test_export_pdf_is_valid_nontrivial	PASSED	[85%]
tests/test_ablation.py::test_load_videos_tolerates_missing_audio	PASSED	[90%]
tests/test_ablation.py::test_litmus_reproduces_gate_a	SKIPPED (golde...)	[95%]
tests/test_ablation.py::test_litmus_report_is_well_formed	SKIPPED (g...)	[100%]

===== 19 passed, 2 skipped in 14.28s =====

69. assistant (2026-06-14T10:17:42.914Z)

19 passed, 2 litmus tests skipped (golden absent in worktree ? correct CI behavior). Now let me prove the litmus actually reproduces by temporarily symlinking/copying the golden JSONs into the worktree's output/, running the 2 litmus tests, then removing them (so they don't get committed ? output/ is gitignored anyway). Let me copy them and run.

70. user (2026-06-14T10:17:47.571Z)

```
6
===== test session starts =====
platform win32 -- Python 3.13.13, pytest-9.0.3, pluggy-1.6.0 --
C:\Users\avidu\AppData\Local\Programs\Python\Python313\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\avidu\Projects\bhi-ablation
configfile: pytest.ini
plugins: anyio-4.13.0, cov-7.1.0
collecting ... collected 21 items / 19 deselected / 2 selected

tests/test_ablation.py::test_litmus_reproduces_gate_a PASSED [ 50%]
tests/test_ablation.py::test_litmus_report_is_well_formed PASSED [100%]

===== 2 passed, 19 deselected in 1.89s =====
```

71. assistant (2026-06-14T10:17:53.467Z)

Both litmus tests **PASS** against the real golden JSONs ? the framework reproduces Gate-A (?F1 +0.074, 6/6, p=0.031, CI [+0.042,+0.104], over-seg 1.68?1.01) within tolerance. Now let me clean up the copied golden JSONs from the worktree (gitignored, but keep the tree clean), and verify `output/` is gitignored so nothing rides in.

72. user (2026-06-14T10:18:23.059Z)

```
?? backend/eval/ablation.py
?? tests/test_ablation.py
===gitignore check===
output/foo.json
===coverage of ablation.py===
tests\test_ablation.py .....ss [100%]
backend\eval\ablation.py 263 40 85% 222, 236-244, 261, 390, 397-398, 588-625, 629
TOTAL 263 40 85%
===== 19 passed, 2 skipped in 25.51s =====
```

73. assistant (2026-06-14T10:18:27.811Z)

Clean tree (only my 2 new files), `output/` is gitignored, and ablation.py has **85% coverage**. The uncovered lines are mostly the CLI `main()` (588-625), the `\_seg\_ratio` fallback path, and some defensive branches. Let me check what those uncovered lines are ? I can boost coverage by testing the CLI main and the litmus seg_ratio path. Let me look at the specific uncovered lines.

74. user (2026-06-14T10:18:28.501Z)

```
216 fn = full.feature_names
217 ablated_fn = [c for c in fn if c not in cols] if fn is not None else fn
218 overrides: Dict[str, Any] = {"feature_names": ablated_fn}
219 if gate_knob == "min_crossings":
220     overrides["min_crossings"] = 0
221 elif gate_knob == "min_players":
222     overrides["min_players"] = 0
223 elif gate_knob is not None:
224     raise ValueError(f"unknown gate_knob {gate_knob!r} (expected
'min_crossings'/'min_players')")
```

```

225         dropped_label = ",".join(sorted(cols)) or (gate_knob or "?")
226         return experiment.Variant.from_dict(**full.to_dict(), **overrides,
227                                           "name": f"?{dropped_label}")
228
229
230     def _seg_ratio(variant: Variant, videos: Sequence[VideoCandidates]) -> float:
231         """Mean predicted-to-GT segment-count ratio across videos (C4 over-seg diagnostic).
232
233         For a learned variant we can't score without a model, so this is only meaningful for
234         the
235         static-gate case the litmus uses; learned-variant rows still record it best-effort
236         and fall
237         back to 0.0 when a model is required (the CI/sign verdict does not depend on it)."""
238         from backend.eval import segmentation_metrics as segm
239         ratios: List[float] = []
240         for vc in videos:
241             try:
242                 preds = experiment.predict(variant, vc)
243             except experiment.ExperimentError:
244                 return 0.0
245             ratios.append(segm.segment_count_ratio(preds, vc.gts or []))
246         return sum(ratios) / len(ratios) if ratios else 0.0

```

75. assistant (2026-06-14T10:18:32.663Z)

I notice `\_seg\_ratio` is actually **dead code** ? I compute the seg ratios from the `honest` block (`res["honest"]["segmentation"]`) inside `ablate\_one`, not via `\_seg\_ratio`. Let me verify it's unused and remove it (dead code is a real cleanup; the honest block already provides the seg ratios).

76. user (2026-06-14T10:18:33.371Z)

```
230: def _seg_ratio(variant: Variant, videos: Sequence[VideoCandidates]) -> float:
```

77. assistant (2026-06-14T10:18:36.509Z)

`\_seg\_ratio` is defined but never called ? the seg ratios come from the honest block. It's dead code. Let me remove it.

78. user (2026-06-14T10:18:40.277Z)

The file C:\Users\avidu\Projects\bhi-ablation\backend\eval\ablation.py has been updated successfully. (file state is current in your context ? no need to Read it back)

79. assistant (2026-06-14T10:18:43.957Z)

Now let me add a CLI test (covers main()) and a couple of edge-case tests to push coverage higher. Let me check what the `Sequence` import is still used for and add a CLI test using monkeypatch/subprocess. I'll add tests that exercise `main()` via monkeypatching sys.argv.

80. user (2026-06-14T10:18:44.853Z)

```

555         "never auto-flagged dead weight on 'no measured lift'.",
556         body))
557
558     doc = SimpleDocTemplate(path, pagesize=A4, leftMargin=2 * cm, rightMargin=2 * cm,
559                           topMargin=1.8 * cm, bottomMargin=1.8 * cm,
560                           title="Signal Ablation & Feature-Enablement Report",
561                           author="badminton-highlight-indexer")
562     doc.build(story)
563     logger.info("wrote ablation PDF -> %s", path)
564     return path
565

```

```

566
567 # -----
568 CLI
569
570 def main() -> None:
571     ap = argparse.ArgumentParser(
572         description="LOSO/LOGO ablation runner over the golden feature substrate (pure
driver "
573         "over experiment.compare + eval_stats; no new scoring math).")
574     ap.add_argument("--features-dir", default="output",
575         help="dir of *_golden_features.json (the persisted golden
substrate)")
576     ap.add_argument("--granularity", choices=("signal", "group"), default="group",
577         help="leave-one-SIGNAL-out (column) or leave-one-GROUP-out
(producer)")
578     ap.add_argument("--iou", type=float, default=0.5)
579     ap.add_argument("--seed", type=int, default=0, help="bootstrap seed (determinism)")
580     ap.add_argument("--litmus", action="store_true",
581         help="reproduce the static Gate-A measurement (CONTROL vs
min_crossings=2) "
582         "instead of the learned full-set ablation")
583     ap.add_argument("--pdf", default=None, help="also write a PDF ablation report to
this path")
584     ap.add_argument("--json", default=None, help="also write the structured report JSON
here")
585     args = ap.parse_args()
586
587     videos = load_videos(args.features_dir)
588     if len(videos) < 2:
589         raise SystemExit(f"need >=2 *_golden_features.json in {args.features_dir} "
590             f"(found {len(videos)})")
591
592     if args.litmus:
593         cfg = litmus_config()
594         cfg.iou, cfg.seed = args.iou, args.seed
595     else:
596         cfg = AblationConfig(full=learned_full_variant(), signals=default_signals(),
597             granularity=args.granularity, iou=args.iou, seed=args.seed)
598
599     report = run_ablation(videos, cfg)
600     print(format_table(report))
601
602     if args.json:
603         with open(args.json, "w", encoding="utf-8") as f:
604             json.dump(report.as_dict(), f, indent=2)
605         print(f"\nwrote report JSON -> {args.json}")
606     if args.pdf:
607         export_pdf(report, args.pdf)
608         print(f"wrote PDF report -> {args.pdf}")
609

```

81. assistant (2026-06-14T10:18:48.326Z)

Let me add CLI tests and an edge-case test (the `min\_players` gate knob, and the `?` dropped-label path) to the test file to cover `main()` and the remaining branches.

82. user (2026-06-14T10:19:06.106Z)

The file C:\Users\avidu\Projects\bhi-ablation\tests\test_ablation.py has been updated successfully. (file state is current in your context ? no need to Read it back)

83. user (2026-06-14T10:19:38.115Z)

```
tests\test_ablation.py .....ss..... [100%]
backend\eval\ablation.py    253      2   99%   244, 612
TOTAL                      253      2   99%
===== 24 passed, 2 skipped in 26.91s =====
```

84. assistant (2026-06-14T10:19:43.873Z)

****99% coverage**** of ablation.py (only 2 lines uncovered). 24 passed, 2 litmus skipped (golden absent in worktree). Let me check the 2 remaining uncovered lines (244, 612) to decide if worth covering.

85. user (2026-06-14T10:19:44.940Z)

```
    if sign_wins > sign_losses:          # leans toward "removal hurts" but evidence is thin
        return SUGGESTIVE, False
    return INERT, False

if __name__ == "__main__":
    main()
```

86. assistant (2026-06-14T10:19:50.183Z)

Line 612 is `if \_\_name\_\_ == "\_\_main\_\_"` (un-coverable). Line 244 is the `INERT` return when `sign\_wins <= sign\_losses` ? wait, that IS covered by `test\_noop\_gate\_is\_inert`. Let me check: line 243 is `if sign\_wins > sign\_losses: return SUGGESTIVE`. Actually line 244 is the `return SUGGESTIVE` ? that needs a non-structural row where `sign\_wins > sign\_losses` but CI does NOT exclude zero AND not earns_keep. The `audio` group hits SUGGESTIVE only with golden data. Let me add a tiny targeted test for `\_classify` directly to cover the SUGGESTIVE branch deterministically.

87. user (2026-06-14T10:19:56.124Z)

The file C:\Users\avidu\Projects\bhi-ablation\tests\test_ablation.py has been updated successfully. (file state is current in your context ? no need to Read it back)

88. assistant (2026-06-14T10:19:58.905Z)

Now let me run the full ablation suite again with coverage, and also run ruff + mypy on the new files.

89. user (2026-06-14T10:20:28.591Z)

```
tests\test_ablation.py .....ss..... [100%]
backend\eval\ablation.py    253      1   99%   612
TOTAL                      253      1   99%
===== 25 passed, 2 skipped in 26.48s =====
```

90. assistant (2026-06-14T10:20:31.792Z)

99% coverage (the only uncovered line is `if \_\_name\_\_ == "\_\_main\_\_"`, which is normal). Now ruff + mypy on the new code.

91. user (2026-06-14T10:20:44.945Z)

```
=== RUFF ===
F841 Local variable `honest` is assigned to but never used
--> backend\eval\ablation.py:259:5
    |
257 |     res = experiment.compare(videos, ablated, cfg.full, iou=cfg.iou, honest=cfg.honest,
258 |                               honest_stats=True)
259 |     honest = res["honest"]
    |     ^^^^^^
260 |     # Re-pair the per-video ?F1 with the configured bootstrap seed for determinism (the
```

```
default
261 |      # honest block uses seed=0; thread cfg.seed explicitly so a run is reproducible).
    |
help: Remove assignment to unused variable `honest`
```

Found 1 error.

No fixes available (1 hidden fix can be enabled with the `--unsafe-fixes` option).

=== MYPY ===

```
backend\annotations\base.py:85: note: By default the bodies of untyped functions are not
checked, consider using --check-untyped-defs [annotation-unchecked]
backend\sports\__init__.py:7: note: By default the bodies of untyped functions are not checked,
consider using --check-untyped-defs [annotation-unchecked]
backend\providers\__init__.py:7: note: By default the bodies of untyped functions are not
checked, consider using --check-untyped-defs [annotation-unchecked]
Success: no issues found in 1 source file
```

92. assistant (2026-06-14T10:20:48.496Z)

Mypy is clean. Ruff found one real issue: `honest = res["honest"]` is assigned but unused (I read it via `res["honest"]["segmentation"]` later). Let me fix that.

93. user (2026-06-14T10:20:50.006Z)

```
255         come for free."""
256         ablated = _ablated_variant(cfg.full, signal.columns, signal.gate_knob)
257         res = experiment.compare(videos, ablated, cfg.full, iou=cfg.iou, honest=cfg.honest,
258                                 honest_stats=True)
259         honest = res["honest"]
260         # Re-pair the per-video ?F1 with the configured bootstrap seed for determinism (the
default
261         # honest block uses seed=0; thread cfg.seed explicitly so a run is reproducible).
262         from backend.eval.eval_stats import paired_delta_ci
263         by_a = {p["video"]: p["f1"] for p in res["variant_a"]["per_video"]}
264         by_b = {p["video"]: p["f1"] for p in res["variant_b"]["per_video"]}
265         vids = [p["video"] for p in res["variant_a"]["per_video"]]
266         pd = paired_delta_ci([by_a[v] for v in vids], [by_b[v] for v in vids],
seed=cfg.seed)
267         st = pd["sign_test"]
268         sign_wins, sign_losses = int(st["wins"]), int(st["losses"])
269         sign_p = float(st["p_value"])
270         verdict, protected = _classify(signal.structural, bool(pd["ci_excludes_zero"]),
271                                       sign_wins, sign_losses, sign_p, cfg.p_threshold)
272
273         seg_full = res["honest"]["segmentation"]["variant_b"]["segment_count_ratio"]
274         seg_abl = res["honest"]["segmentation"]["variant_a"]["segment_count_ratio"]
```

94. user (2026-06-14T10:20:55.506Z)

The file C:\Users\avidu\Projects\bhi-ablation\backend\eval\ablation.py has been updated successfully. (file state is current in your context ? no need to Read it back)

95. assistant (2026-06-14T10:20:59.755Z)

Now let me also verify mypy on the test file and confirm `Sequence` import is still used in ablation.py (after removing `\_seg\_ratio`). Also re-run ruff.

96. user (2026-06-14T10:21:22.822Z)

```
=== RUFF (full repo to catch any new issue) ===
All checks passed!
ruff clean
=== Sequence usage ===
6
=== MYPY backend training ===
```

```
backend\providers\__init__.py:7: note: By default the bodies of untyped functions are not
checked, consider using --check-untyped-defs [annotation-unchecked]
backend\pipeline\validators\base.py:36: note: By default the bodies of untyped functions are not
checked, consider using --check-untyped-defs [annotation-unchecked]
Success: no issues found in 94 source files
```

97. assistant (2026-06-14T10:21:27.542Z)

Ruff clean, mypy clean (94 source files, success). Now let me write the minimal
`docs/ABLATION\_LAYER.md` note, then run the full test suite to confirm nothing else broke.

98. assistant (2026-06-14T10:21:46.625Z)

Ablation layer (Phase 0) ? per-signal marginal contribution

A *systematic, repeatable* version of the one-off ablation (`scratch/gate\_ab\_ablation.py`):
`backend/eval/ablation.py` runs it and it EXCLUDES one signal, measures whether quality dropped,
and reports each signal's *marginal ?F1* (full set vs full-minus-that-signal) with a bootstrap
95% CI + paired sign test + the C4 over-segmentation ratio. The full design (the ledger time
series, interaction handling, per-user ablation, the phased plan) lives in
[`../scratch/ABLATION\_FRAMEWORK\_PLAN.md`](../scratch/ABLATION_FRAMEWORK_PLAN.md); this note
covers
only the shipped Phase-0 runner.

What it is (and is not)

A pure driver. No new scoring/stats math ? every number defers to the shipped
`experiment.compare` + `eval\_stats` engine, exactly as `compare` was a driver over
`metrics`/`calibration`. For a signal `s`:

```
...
```

```
?(s) = score(full set S) ? score(S \ {s})          # leave-one-out drop, ?F1 primary
...
```

is `experiment.compare(videos, variant\_a=without\_s, variant\_b=full)` ? A is the ablated
(smaller)
variant, B is full, so the reported *B?A* ?F1 IS ?(s): a per-video-paired ?F1 with bootstrap
CI
+ exact sign test, LOVO-honest for learned variants.

A signal is either a *feature column* (person/audio/shuttle columns the learned scorer
weights,
ablated by dropping the column) or a *gate knob* (Gate-A's `min\_crossings` threshold, ablated
by
zeroing the knob). The runner handles both uniformly.

Two granularities

- *LOSO* (`--granularity signal`) ? leave one column-set out at a time (fine-grained).
- *LOGO* (`--granularity group`) ? leave a whole producer's columns out at once (the owner's
"is this old SIGNAL still relevant?" question at cue granularity). Default.

The honest small-N contract

Inherited from `eval\_stats` / `per\_user\_gate` (no new statistics):

- *Never a bare mean* ? every row ships a CI + the exact sign test.
- A signal *earns its keep* only when removal drops F1 with the ?F1 *CI* excluding 0 AND the
sign test agreeing (the `per\_user\_gate` bar, in reverse).
- *Structural guards are exempt from auto-demotion.* Gate-A (the held-shuttle false-positive
killer) is marked `structural`; it is reported honestly but verdict is always
`structural-protected`, never flagged dead weight on "no measured lift" ? its value is in

failure modes the golden corpus may not contain.

- The verdict vocabulary is closed and never over-claims: `clears-bar` / `suggestive` / `inert` / `structural-protected`. One wash at n=6 is `inert` (a candidate), never "irrelevant".

Run it

```
```bash
LOGO contribution table over the learned full set (shuttle+gate_a+person+audio):
```

```
python -m backend.eval.ablation --features-dir output [--granularity signal] \
 [--pdf ablation.pdf] [--json ablation.json]
```

**Reproduce the hand-measured Gate-A result (the framework's litmus):**

```
python -m backend.eval.ablation --features-dir output --litmus
```
```

The corpus is the 6 persisted golden feature JSONs (`output/\*\_golden\_features.json`, written by `fusion\_golden.py` ? persist once on the GPU, re-score forever offline). `--pdf` writes a clean reportlab report (table of marginal contribution + CI + sign test + a verdict column).

Litmus (regression guard on the math)

`tests/test\_ablation.py::test\_litmus\_reproduces\_gate\_a` asserts that, on the 6 real golden JSONs, the runner reproduces the recorded `QUALITY\_ITERATIONS` Gate-A result within tolerance:

```
metric	recorded (2026-06-14)	reproduced
marginal ?F1	+0.074	+0.074
sign test	6W/0L, p=0.031	6W/0L, p=0.031
95% CI	[+0.042, +0.104]	[+0.042, +0.105]
over-seg ratio	1.68 ? 1.01	1.68 ? 1.01
```

The test skips cleanly when the golden substrate is absent (CI) and runs when present (owner box).
The rest of the suite runs GPU-free on deterministic synthetic candidate rows.

99. user (2026-06-14T10:21:46.640Z)

File created successfully at: C:\Users\avidu\Projects\bhi-ablation\docs\ABLATION_LAYER.md (file state is current in your context ? no need to Read it back)

100. assistant (2026-06-14T10:21:49.255Z)

Now let me run the full test suite (with coverage floor) to ensure the whole suite is green and nothing regressed.

101. user (2026-06-14T10:23:02.141Z)

| | | | |
|----------------------------------|-----|----|------|
| backend\reporting\run_signals.py | 14 | 0 | 100% |
| backend\results.py | 22 | 0 | 100% |
| backend\sports__init__.py | 17 | 1 | 94% |
| backend\sports\badminton.py | 13 | 1 | 92% |
| backend\sports\base.py | 12 | 3 | 75% |
| backend\tools__init__.py | 0 | 0 | 100% |
| backend\tools\export.py | 75 | 2 | 97% |
| backend\tools\rally_slicer.py | 104 | 47 | 55% |
| backend\utils__init__.py | 0 | 0 | 100% |
| backend\utils\geometry.py | 29 | 0 | 100% |
| backend\utils\hashing.py | 9 | 0 | 100% |
| backend\utils\logging_setup.py | 11 | 0 | 100% |
| backend\utils\paths.py | 27 | 3 | 89% |
| backend\utils\run_telemetry.py | 126 | 12 | 90% |

| | | | |
|---------------------------|-----|----|------|
| backend\utils\video.py | 25 | 3 | 88% |
| training__init__.py | 0 | 0 | 100% |
| training\gen0__init__.py | 0 | 0 | 100% |
| training\gen0\harness.py | 108 | 11 | 90% |

| | | | |
|-------|------|------|-----|
| TOTAL | 6511 | 1244 | 81% |
|-------|------|------|-----|

Required test coverage of 70% reached. Total coverage: 80.89%

603 passed, 2 skipped, 4 warnings in 69.11s (0:01:09)

=====

RESULT: PASS - all tests green

=====

102. assistant (2026-06-14T10:23:06.146Z)

Full suite green: ****603 passed, 2 skipped**** (the litmus tests, golden absent in worktree), total coverage ****80.89%**** (well above the 70% floor; my ablation.py is 99%).

Let me verify the 4 warnings aren't from my code, then check the import-smoke lane will pass (my module imports reportlab lazily inside `export\_pdf`, so the module-level import is clean ? let me confirm).

103. user (2026-06-14T10:23:22.888Z)

=== import-smoke lane ===

.....

[100%]

5 passed in 12.24s

=== module imports without reportlab/pypdf at top level? ===

top-level imports: ['__future__', 'argparse', 'backend.eval', 'backend.eval.experiment', 'dataclasses', 'glob', 'json', 'logging', 'os', 'typing']

OK: no PDF deps at module top level

104. assistant (2026-06-14T10:23:26.787Z)

All 4 lanes verified locally:

- ****import-smoke****: 5 passed; reportlab/pypdf imported lazily (not at module top level), so the import-smoke lane (no PDF deps installed beyond reportlab which is in requirements) is safe.
- ****full-suite****: 603 passed, 80.89% coverage.
- ****ruff****: clean.
- ****mypy****: clean (94 source files).

Let me check whether reportlab is in requirements.txt (the full-suite lane needs it for `test\_export\_pdf\_is\_valid\_nontrivial`), and pypdf too.

105. user (2026-06-14T10:23:28.098Z)

NOT in requirements.txt

106. assistant (2026-06-14T10:23:35.050Z)

reportlab and pypdf are ****not**** in requirements.txt, so the full-suite CI lane won't have them installed. My PDF tests use `pytest.importorskip("pypdf")`, which handles missing pypdf, but `test\_cli\_main\_writes\_pdf\_and\_json` and `test\_export\_pdf\_is\_valid\_nontrivial` call `export\_pdf` which imports reportlab ? if reportlab is absent, that import fails. The owner has reportlab locally (it's used by scratch/quality_report.py) but CI won't.

I need the PDF tests to skip cleanly if reportlab OR pypdf is absent. Let me check how scratch/quality_report.py handles this and add a guard. The cleanest: add `pytest.importorskip("reportlab")` and `pytest.importorskip("pypdf")` to each PDF-producing test. Let me update the tests.

107. user (2026-06-14T10:23:37.137Z)

The file C:\Users\avidu\Projects\bhi-ablation\tests\test_ablation.py has been updated successfully. (file state is current in your context ? no need to Read it back)

108. user (2026-06-14T10:23:42.592Z)

The file C:\Users\avidu\Projects\bhi-ablation\tests\test_ablation.py has been updated successfully. (file state is current in your context ? no need to Read it back)

109. assistant (2026-06-14T10:23:51.688Z)

Now there's a subtlety: `test\_cli\_main\_writes\_pdf\_and\_json` passes `--pdf`, and if reportlab is skipped, fine. But I should double-check: does the full-suite CI lane install reportlab transitively? Let me verify reportlab is truly absent in a clean CI-like env by checking if anything in requirements pulls it. Since it's not in requirements and tests now `importorskip`, those 3 PDF tests will skip in CI ? which is acceptable (the owner runs them locally with reportlab installed).

However, the task explicitly asks to "assert via pypdf page count > 0" as a healthy-coverage requirement. The owner has both installed locally, so the test runs there. To be safe and make the PDF coverage land in CI too, the better move is to ****add reportlab and pypdf to requirements.txt**** so the full-suite lane exercises the PDF path. reportlab is already a de-facto dependency (scratch/quality_report.py). Let me check requirements.txt and add them.

110. user (2026-06-14T10:23:51.756Z)

```
1      fastapi>=0.111.0
2      uvicorn>=0.30.1
3      opencv-python>=4.9.0.80
4      numpy>=1.26.4
5      pydantic>=2.7.1
6      jinja2>=3.1.4
7      python-dotenv>=1.0.0
8      google-genai>=2.4.0
9      torch>=2.12.0
10     torchvision>=0.27.0
11     supervision>=0.21.0,<0.30.0
12
13     # Per-video observability reports (SOFT dependency: the pipeline degrades to a
14     # no-op if absent). Shared, permissively-licensed report toolkit. Pin a tag once
15     # the private repo is published, e.g.:
16     #   obsreport @ git+ssh://git@github.com/avidullu/sports-obsreport.git@v0.1.3
17     # For local development install it editable from a sibling checkout:
18     #   pip install -e ../sports-obsreport --no-build-isolation
19
```

111. assistant (2026-06-14T10:23:58.129Z)

reportlab isn't a core pipeline dependency ? it's only used by the offline ablation/report PDF exporter. Adding heavy deps to `requirements.txt` would change the production dependency surface, which isn't appropriate for an offline reporting feature. The right call: keep the PDF tests `importorskip`-guarded (so CI stays green and the PDF path is exercised on the owner's box where reportlab/pypdf are installed). This mirrors the existing `obsreport` soft-dependency pattern and the `scratch/quality\_report.py` reportlab usage that's already not in requirements.

The PDF tests will skip in CI but run locally ? consistent with the repo's "offline reporting is a soft dep" posture. Let me verify the full suite still passes locally (where reportlab IS installed, so the PDF tests run) and confirm the final state.

112. user (2026-06-14T10:24:16.891Z)

```
.....SS..... [100%]
25 passed, 2 skipped in 16.93s
```

113. assistant (2026-06-14T10:24:20.234Z)

25 passed, 2 skipped (litmus). The PDF tests run locally (reportlab present). Let me do a final lint+mypy pass on everything, then verify the litmus reproduction one more time end-to-end via

the CLI for the report, and commit.

114. user (2026-06-14T10:24:27.542Z)

=== RUFF whole repo ===

All checks passed!

=== MYPY ===

Success: no issues found in 94 source files

=== LITMUS via CLI ===

| signal | ?F1(full?ablated) | 95% CI | sign test | CI?0 | verdict |
|--------|-------------------|------------------|---------------|------|----------------------|
| gate_a | +0.074 | [+0.042, +0.105] | 6W/0L p=0.031 | yes | structural-protected |

115. assistant (2026-06-14T10:24:31.489Z)

Everything green. Now let me follow branch-safety protocol: verify the branch and HEAD, confirm only my files are staged, and commit with explicit paths.

116. user (2026-06-14T10:24:34.672Z)

=== branch ===

feat/ablation-layer

=== HEAD ===

7c6498f docs(quality): personalization Phase-0 checkpoint ? eval/safety rails + locked structural-guard decision (#145)

=== status ===

?? backend/eval/ablation.py

?? docs/ABLATION_LAYER.md

?? tests/test_ablation.py

=== commits ahead of origin/master ===

117. assistant (2026-06-14T10:24:41.767Z)

On `feat/ablation-layer`, HEAD = origin/master tip (7c6498f), no sibling commits ahead, only my 3 untracked files. origin/master hasn't moved. Let me stage exactly my 3 files and commit.

118. user (2026-06-14T10:24:42.618Z)

warning: in the working copy of 'backend/eval/ablation.py', LF will be replaced by CRLF the next time Git touches it

warning: in the working copy of 'docs/ABLATION_LAYER.md', LF will be replaced by CRLF the next time Git touches it

warning: in the working copy of 'tests/test_ablation.py', LF will be replaced by CRLF the next time Git touches it

A backend/eval/ablation.py

A docs/ABLATION_LAYER.md

A tests/test_ablation.py

119. assistant (2026-06-14T10:24:45.621Z)

All 3 files staged (the LF?CRLF warning is just git's autocrlf on Windows, harmless). Let me commit.

120. user (2026-06-14T10:24:58.637Z)

[feat/ablation-layer ab364c4] feat(eval): LOS0/LOG0 ablation runner ? per-signal marginal ?F1 (Phase 0)

3 files changed, 1175 insertions(+)

create mode 100644 backend/eval/ablation.py

create mode 100644 docs/ABLATION_LAYER.md

create mode 100644 tests/test_ablation.py

ab364c4 feat(eval): LOS0/LOG0 ablation runner ? per-signal marginal ?F1 (Phase 0)

121. assistant (2026-06-14T10:25:00.991Z)

Committed. Now let me push and create the PR.

122. user (2026-06-14T10:25:04.120Z)

```
remote:      https://github.com/avidullu/badminton-highlight-indexer/pull/new/feat/ablation-
layer
remote:
branch 'feat/ablation-layer' set up to track 'origin/feat/ablation-layer'.
To https://github.com/avidullu/badminton-highlight-indexer.git
 * [new branch]      feat/ablation-layer -> feat/ablation-layer
```

123. user (2026-06-14T10:25:30.610Z)

<https://github.com/avidullu/badminton-highlight-indexer/pull/147>

124. assistant (2026-06-14T10:25:34.029Z)

PR #147 created. Now let me wait for the 4 CI lanes to complete before merging.

125. user (2026-06-14T10:25:36.267Z)

```
Full test suite (offline, mocked)      pending 0      https://github.com/avidullu/badminton-
highlight-indexer/actions/runs/27495923073/job/81269855158
Mypy (lenient baseline) pending 0      https://github.com/avidullu/badminton-highlight-
indexer/actions/runs/27495923073/job/81269855134
Ruff lint      pending 0      https://github.com/avidullu/badminton-highlight-
indexer/actions/runs/27495923073/job/81269855187
Syntax sweep + import smoke (no GPU stack)      pending 0
https://github.com/avidullu/badminton-highlight-indexer/actions/runs/27495923073/job/81269855140
```

126. assistant (2026-06-14T10:25:39.231Z)

All 4 lanes are pending. Let me wait for them to complete using a monitoring loop.

127. user (2026-06-14T10:27:45.379Z)

```
Ruff lint      pass    7s      https://github.com/avidullu/badminton-highlight-
indexer/actions/runs/27495923073/job/81269855187
Syntax sweep + import smoke (no GPU stack)      pass    20s
https://github.com/avidullu/badminton-highlight-indexer/actions/runs/27495923073/job/81269855140
Full test suite (offline, mocked)      pass    1m56s      https://github.com/avidullu/badminton-
highlight-indexer/actions/runs/27495923073/job/81269855158
Mypy (lenient baseline) pass    32s      https://github.com/avidullu/badminton-highlight-
indexer/actions/runs/27495923073/job/81269855134
Ruff lint      pass    7s      https://github.com/avidullu/badminton-highlight-
indexer/actions/runs/27495923073/job/81269855187
Syntax sweep + import smoke (no GPU stack)      pass    20s
https://github.com/avidullu/badminton-highlight-indexer/actions/runs/27495923073/job/81269855140
Full test suite (offline, mocked)      pass    1m56s      https://github.com/avidullu/badminton-
highlight-indexer/actions/runs/27495923073/job/81269855158
Mypy (lenient baseline) pass    32s      https://github.com/avidullu/badminton-highlight-
indexer/actions/runs/27495923073/job/81269855134
Ruff lint      pass    7s      https://github.com/avidullu/badminton-highlight-
indexer/actions/runs/27495923073/job/81269855187
Syntax sweep + import smoke (no GPU stack)      pass    20s
https://github.com/avidullu/badminton-highlight-indexer/actions/runs/27495923073/job/81269855140
---FINAL---
Full test suite (offline, mocked)      pass    1m56s      https://github.com/avidullu/badminton-
highlight-indexer/actions/runs/27495923073/job/81269855158
Mypy (lenient baseline) pass    32s      https://github.com/avidullu/badminton-highlight-
indexer/actions/runs/27495923073/job/81269855134
Ruff lint      pass    7s      https://github.com/avidullu/badminton-highlight-
```

indexer/actions/runs/27495923073/job/81269855187
Syntax sweep + import smoke (no GPU stack) pass 20s
<https://github.com/avidullu/badminton-highlight-indexer/actions/runs/27495923073/job/81269855140>

128. assistant (2026-06-14T10:27:49.980Z)

All ****4 CI lanes pass**** (Ruff 7s, import-smoke 20s, mypy 32s, full-suite 1m56s). Per the merging working agreement (owner authorized merging good-engineering-fix PRs on verified-green CI), let me merge with squash and delete the branch.

129. user (2026-06-14T10:27:54.310Z)

failed to run git: fatal: 'master' is already used by worktree at
'C:/Users/avidu/Projects/badminton-highlight-indexer'

130. assistant (2026-06-14T10:28:01.421Z)

The remote merge succeeded but `gh` failed on the local post-merge checkout step (it tried to switch the worktree to `master`, which is checked out by the main repo). Let me verify the PR actually merged and the branch was deleted remotely.

131. user (2026-06-14T10:28:05.023Z)

```
{"mergeCommit":{"oid":"e75ec3570dd2242e154a0f580bc5b49432e3d44d"},"mergedAt":"2026-06-14T10:27:54Z","state":"MERGED"}  
=== remote branch check ===  
(empty above = branch deleted)
```

132. assistant (2026-06-14T10:28:08.551Z)

PR #147 is ****MERGED**** (merge commit `e75ec35`) and the remote branch was deleted. The `gh` error was only the cosmetic local-checkout step. Now let me clean up the worktree, local branch, and prune. First I'll move out of the worktree dir to avoid lock issues, then remove it.

133. user (2026-06-14T10:28:11.088Z)

```
EXIT:0  
Deleted branch feat/ablation-layer (was ab364c4).  
=== remaining worktrees ===  
C:/Users/avidu/Projects/badminton-highlight-indexer 570cf2b [master]
```

134. assistant (2026-06-14T10:28:13.967Z)

Cleanup complete: worktree removed, local branch deleted, prune done ? only the main repo worktree remains. Let me do a final verification that master now contains the merged work and pull it into the main checkout.

135. user (2026-06-14T10:28:17.824Z)

```
e75ec35 feat(eval): LOS0/LOG0 ablation runner ? per-signal marginal ?F1 (Phase 0) (#147)  
570cf2b feat(fusion): flip Gate-A (held-shuttle guard) to the served default ? config-reversible (#146)  
7c6498f docs(quality): personalization Phase-0 checkpoint ? eval/safety rails + locked structural-guard decision (#145)  
=== main checkout state (uncommitted WIP preserved) ===  
master
```

136. assistant (2026-06-14T10:28:39.729Z)

The merge landed on `origin/master` as `e75ec35` (#147). Notably, a ****sibling PR #146**** ("flip Gate-A to the served default") merged concurrently ? confirming the value of the conflict-avoidance constraint (I never touched `QUALITY\_ITERATIONS.md` or the leaderboard, so there was no collision). I left the main checkout's uncommitted WIP untouched (didn't pull into it, to

avoid clobbering the owner's working state).

Task complete. Here's the report.

Summary

Built and **merged** the ablation-layer Phase 0 per `scratch/ABLATION\_FRAMEWORK\_PLAN.md`.

PR: #147 ? <https://github.com/avidullu/badminton-highlight-indexer/pull/147> ? **MERGED**
(squash, commit `e75ec35`), remote+local branches deleted, worktree removed and pruned.

Files (3, +1175 lines):

- `backend/eval/ablation.py` ? the LOS0/LOG0 runner (`Signal` / `AblationConfig` / `AblationRow` / `AblationReport` + `ablate\_one` + `run\_ablation` + `export\_pdf` + CLI). A pure driver over `experiment.compare` + `eval\_stats` (no new scoring/stats math). Gate-A ablates via the `min\_crossings` knob; structural signals are reported but never auto-demoted.
- `tests/test\_ablation.py` ? 27 tests (25 run GPU-free, 2 litmus skip-if-absent).
- `docs/ABLATION\_LAYER.md` ? minimal shipped-runner note.

Litmus result ? reproduced Gate-A exactly ? (on the 6 real `output/\*\_golden\_features.json`):

| | | |
|--------------|------------------|-----------------------------|
| metric | recorded truth | reproduced |
| --- | --- | --- |
| marginal ?F1 | +0.074 | **+0.074** |
| sign test | 6W/0L, p=0.031 | **6W/0L, p=0.031** |
| 95% CI | [+0.042, +0.104] | **[+0.042, +0.105]** |
| over-seg | 1.68 ? 1.01 | **1.68 ? 1.01** |

The litmus test skips cleanly in CI (golden substrate absent) and ran green locally against the owner's golden JSONs.

Sample ablation table (LOG0, learned full set):

...

Ablation report ? full set = {shuttle, gate_a, person, audio} · n=6 · IoU 0.5 · LOV0-honest · GROUP

| signal | ?F1(full?ablated) | 95% CI | sign test | CI?0 | verdict |
|---------|-------------------|------------------|---------------|------|----------------------|
| audio | +0.079 | [-0.054, +0.203] | 4W/2L p=0.688 | no | suggestive |
| shuttle | +0.002 | [-0.013, +0.019] | 2W/2L p=1.000 | no | inert |
| person | -0.009 | [-0.074, +0.055] | 2W/4L p=0.688 | no | inert |
| gate_a | -0.014 | [-0.083, +0.027] | 4W/1L p=0.375 | no | structural-protected |

...

PDF: `export\_pdf` renders a clean reportlab report (marginal contribution + CI + sign test + verdict column: clears-bar / suggestive / inert / structural-protected), wired to `--pdf`. Validated via pypdf page count > 0 (1 page, ~4KB). Path is user-supplied (e.g. `--pdf ablation.pdf`).

Tests / coverage: 25 passing GPU-free tests (+ 2 litmus that run on the owner box), **99%** line coverage of `ablation.py`; full repo suite 603 passed / 80.89% total coverage.

Merge status: all **4** CI lanes green (Ruff 7s, import-smoke 20s, mypy 32s, full-suite 1m56s), merged on green per the merging working agreement.

Constraint honored: did not touch `docs/QUALITY\_ITERATIONS.md` or `eval\_baselines/experiment\_leaderboard.json`; tests use tmp paths / skip-if-absent, never the real leaderboard. A concurrent sibling PR #146 (Gate-A served-default flip) merged in the same window with zero conflict ? the scope separation held.

One note: `reportlab`/`pypdf` aren't in `requirements.txt`, so the PDF tests `importorskip` them (same soft-dependency posture as the existing `obsreport` and `scratch/quality\_report.py`) ? they skip in CI but run on the owner's box where both are installed. If you'd prefer the PDF path exercised in CI too, adding those two to requirements is a one-line follow-up.